

Unity 6000.5 ・ Entities 6.5

Unity DOTS 入門

Entities、Netcode、ECS ワークフロー、job、デバッグ、最適化、移行。



Youngmin Cho (SillyToolValley)

日本語版 ・ 2026

Unity DOTS 入門

Entities 6.5、Netcode for Entities、ECS ワークフロー、最適化、移行を扱う実践ガイド。

Unity DOTS Introduction リポジトリから 2026-05-21 に生成した日本語版。

著者: Youngmin Cho (SillyToolValley) / UCI (Unity Certified Instructor)

Unity、Entities、Netcode for Entities および関連パッケージ名は Unity Technologies の商標または製品名である。本書はコミュニティによる学習用のまとめであり、Unity 公式ドキュメントではない。

公開された範囲での閲覧と引用は許可されている。自動スクレイピング、データセットへの収録、AI 学習、AI 回答への取り込みは、書面による許可なく行うことを許可しない。

Unity DOTS 入門

Entities 6.5、Netcode for Entities、ECS ワークフロー、最適化、マイグレーションの実践ガイド。

本書は Unity DOTS ガイドラインリポジトリを、ひと続きの読み物としてまとめたものだ。あくまで実践重視で、セットアップ、SubScene と Baker のワークフロー、ECS のコンポーネントとシステムのパターン、ジョブスケジューリング、構造変更の安全性、EntityCommandBuffer の使い方、Netcode for Entities、最適化、デバッグ、マイグレーション、用語集までを扱う。

対象とする環境は Unity 6000.5 以降、Entities 6.5.0、Netcode for Entities 6.5.0、そしてエディターに同梱される DOTS コアパッケージである。

本書の読み方

- DOTS が初めてなら、まず Part I を読み、Netcode に進む前に Part II の Chapter 4～15 に目を通してほしい。
- すでに Entities 1.x を使っているなら、コアの章は流し読みでよく、マイグレーションの解説は Part IV にある。
- パフォーマンス改善は ECS ワークフローの主要な章を終えてから取り組むとよい。チャンクレイアウトやプロファイリングは、データモデルに慣れて初めて腹に落ちるからだ。

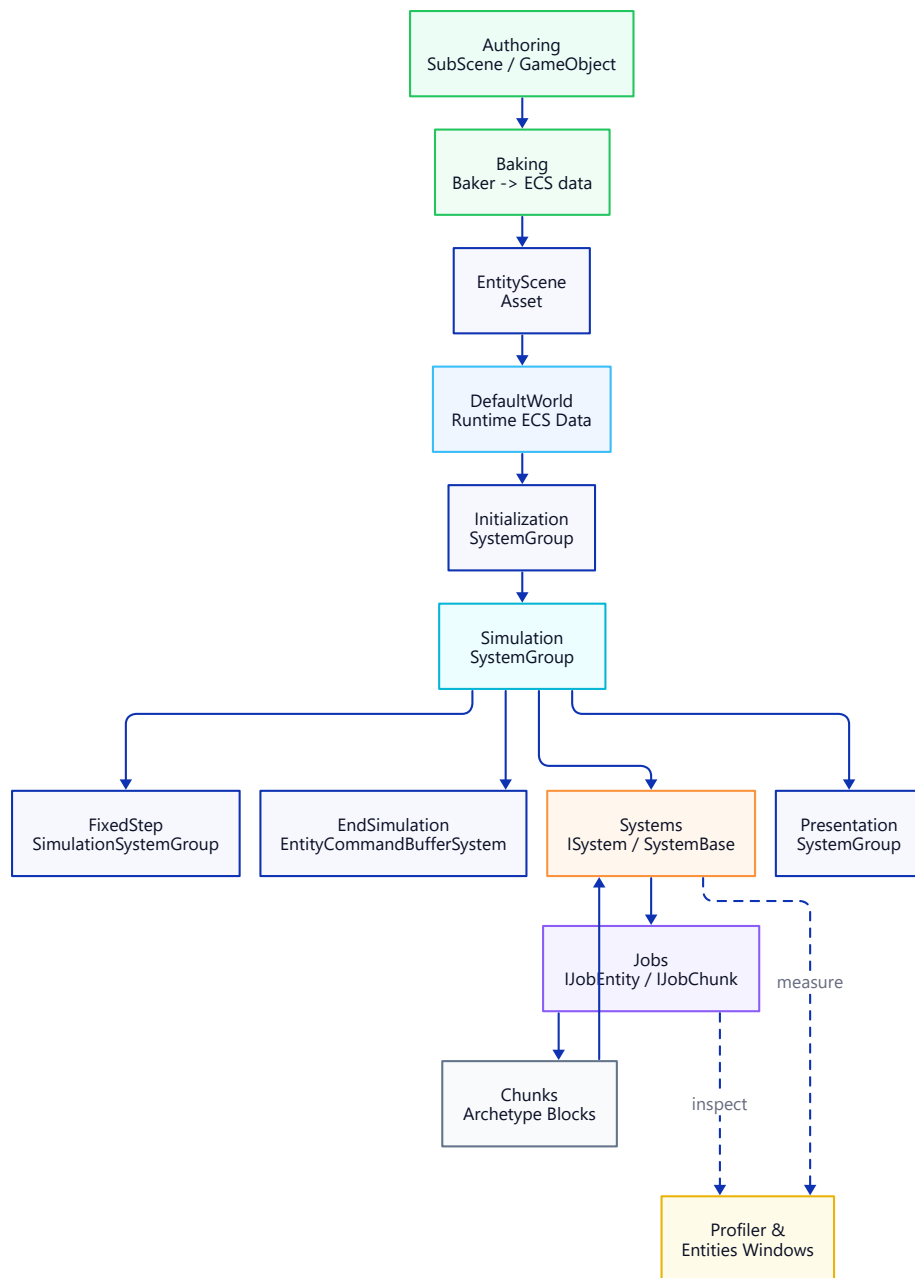


Figure: Unity DOTS アーキテクチャフロー。

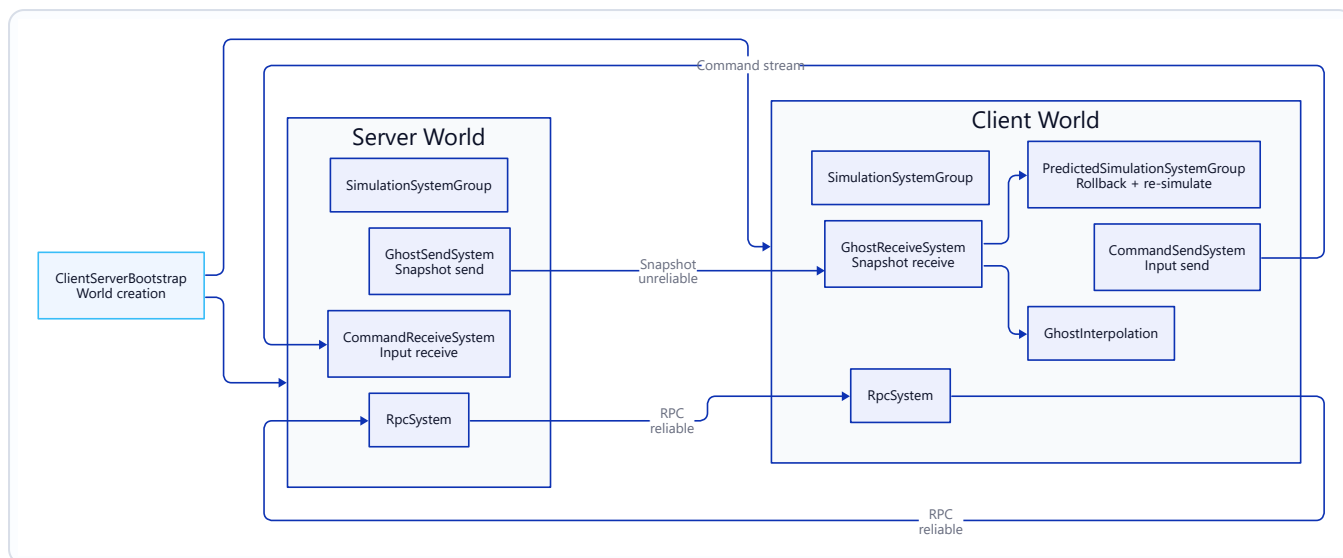


Figure: Netcode for Entities アーキテクチャフロー。

目次

セクション	トピック
Part I. はじめに	
Chapter 1	環境構築 — Unity 6000.5 + Entities 6.5.0
Chapter 2	コアパッケージ解説
Chapter 3	Hello DOTS — はじめての Entity
Part II. DOTS ワークフロー	
Chapter 4	Baker パターン & SubScene
Chapter 5	Spawner サンプル
Chapter 6	ECS コアコンセプト
Chapter 7	Entity 参照 — Entity · EntityPrefabReference · UnityObjectRef
Chapter 8	Component 型
Chapter 9	Enableable Component
Chapter 10	Singleton Component
Chapter 11	System — ISystem vs SystemBase
Chapter 12	System Group & 更新順序
Chapter 13	JobSystem & Burst
Chapter 14	IJobEntity · SystemAPI.Query

セクション	トピック
Chapter 15	IJobEntity vs IJobChunk
Chapter 16	構造変更 & 安全性
Chapter 17	EntityCommandBuffer・遅延 Entity
Chapter 18	ParallelWriter・決定論的再生
Chapter 19	Netcode クライアント・サーバー World & Bootstrap
Chapter 20	Netcode ネットワーク接続 & 承認
Chapter 21	Netcode Ghost スナップショット & 同期
Chapter 22	Netcode 予測 & ロールバック
Chapter 23	Netcode コマンドストリーム & 入力
Chapter 24	Netcode RPC
Chapter 25	Netcode Ghost 最適化・重要度・関連性
Chapter 26	Netcode 物理インテグレーション & ラグ補正
Chapter 27	Netcode Profiler & デバッグ
Part III. 最適化とデバッグ	
Chapter 28	チャンクレイアウト & TypeManager
Chapter 29	Systems・Entity Inspector・Query ウィンドウ
Chapter 30	Profiler・ボトルネック分析
Chapter 31	マネージドオブジェクト参照の監査
Part IV. マイグレーション	
Chapter 32	Entities 1.x → 6.5 概要
Chapter 33	Package Manager → Core Package
Chapter 34	マネージドオブジェクト参照 → UnityObjectRef
Chapter 35	foreach → IJobEntity マイグレーション
Chapter 36	IAAspect 廃止 — Aspect からの移行
Appendix. 変更履歴	
Appendix A	Entities 1.4 → 6.5 主要変更点
Appendix B	Netcode for Entities 1.4 → 6.5 主要変更点
Reference. グLOSSARY	
Glossary	グLOSSARY

Part I. はじめに

Chapter 1. 環境構築 — Unity 6000.5 + Entities 6.5.0

1. 概要

Unity 6000.4 以降、Entities は **Core Package** になった。エディターに同梱され、そのバージョンはエディターのバージョンに固定されるため、Entities のバージョンを自分で選ぶことはもうない。ただし新規プロジェクトに自動で追加されるわけ**ではない**。プロジェクトごとに一度、Package Manager (Unity Registry) から追加する必要がある。本章ではエディターのインストールから、プロジェクトの作成、Entities の追加、そして有効になっているかの確認までを順に追っていく。

動作確認環境： Unity 6000.5.0b2 · Entities 6.5.0 · Collections / Mathematics / Entities Graphics (いずれも Core Package)。

2. Unity 6000.5+ のインストール

1. **Unity Hub** を開く。未導入なら先に Unity Hub をインストールしておく (<https://unity.com/download>)。
2. Hub で **Installs** → **Install Editor** を開く。
3. **6000.5.x** (b 系ベータ以降) のいずれかを選ぶ。Entities 6.5 は 6000.5 以降でしか使えない。
4. モジュールはデフォルトのままでよい。ECS そのものに追加モジュールは要らない。

エディターをインストールするとき、"DOTS" モジュールにチェックを入れる必要は**ない**。Entities パッケージは 6000.4 からエディターに同梱されている。各プロジェクトへの追加はあとで行う (次のステップを参照)。

3. 新規プロジェクトの作成

Hub → **Projects** → **New project** から：

フィールド	値
Editor Version	6000.5.x
Template	Universal 3D （または 3D (Built-in) 、 HDRP — どれでも可）
Project Name	任意

Create project をクリックすると、空のシーンでエディターが立ち上がる。

6000.4 以降では、専用の「DOTS テンプレート」はもう要らない。どのテンプレートでも Entities ワークフローを動かせる。

4. Entities パッケージの追加と有効化の確認

Window → **Package Manager** を開き、スコープを **Unity Registry** に切り替え、**Entities** を選んで **Install** をクリックする（または **+ → Add package by name...** から `com.unity.entities` を入力する）。Entities は Core Package なのでバージョンは選ばない。エディターに同梱されているもの（Unity 6000.5 では Entities 6.5）がインストールされる。Entities をインストールすると、依存関係として **Collections**、**Mathematics**、**Entities Graphics** も一緒に取り込まれる。

続いてスコープを **In Project** に切り替え、**Entities**、**Collections**、**Mathematics**、**Entities Graphics** が並んでいることを確認する。ファイルで確かめたいなら `Packages/manifest.json` を開く。いまは `com.unity.entities` のエントリが存在する（それが取り込む依存関係は自動的に解決され、必ずしもそれぞれ専用のエントリを持つとは限らない）。

続いて ECS のウィンドウを開く。

- **Window** → **Entities** → **Hierarchy** — 実行時の Entity を表示する。
- **Window** → **Entities** → **Systems** — 登録済みの System を表示する。
- **Window** → **Entities** → **Archetypes** — チャンクレイアウトを表示する。

3 つのメニューがすべて揃っていれば、Entities は正しく組み込まれている。

5. スモークテスト — SubScene の作成

1. Hierarchy で右クリック → **New Sub Scene** → **Empty Scene...**
2. MainSubScene.unity という名前で保存する。
3. 新しい SubScene の中に、プリミティブを 1 つ追加する (**3D Object** → **Cube**)。
4. シーンを保存する。SubScene を閉じたとき (アイコンが黄色になる)、またはビルド時に、Cube は Entity へベイクされる。
5. **Play** を押す。**Window** → **Entities** → **Hierarchy** を開いて Cube Entity が見えれば、スモークテストは合格だ。

SubScene はオーサリングの起点だ。その中の GameObject は、保存時とビルド時に Entity へベイクされる。03_Hello DOTS — First Entity.md のチュートリアルも、ここを土台に話を進めていく。

6. トラブルシューティング

症状	原因 / 対処法
Window → Entities メニューが見当たらない	Entities がまだインストールされていない (Package Manager → Unity Registry → Entities から追加する)。あるいはエディターが 6000.4 より古い (アップグレードする)。
Baker<T> / IComponentData が見つからない	アセンブリ定義ファイルが Unity.Entities を参照していない。参照に追加するか、手早く試したいだけなら asmdef を削除する。
SubScene が GameObject のままでベイクされない	SubScene が 閉じた状態 (黄色のアイコン) でないと、ベイク済みの Entity は表示されない。Hierarchy のチェックボックスで開閉する。
Play モードで Entities Hierarchy ウィンドウに何も表示 されない	SubScene が現在のシーンに含まれているか、また Window → Entities → Hierarchy の「World」ドロップダウンが DefaultGameObjectInjectionWorld になっているかを確認する。
ビルドエラー： com.unity.entities not found	パッケージがインストールされていないか、manifest.json がこのエディターには存在しない 1.x のバージョンを固定している。Package Manager から Entities をインストールし、エントリがエディターの Core バージョンに解決されるようにする。Migration/02_Package Manager → Core Package.md を参照。

Chapter 2. コアパッケージ解説

1. 概要

Entities パッケージ 6.4.0 のリリース（Unity Editor 6000.4 に同梱）を境に、主要な DOTS パッケージは **Core Package** になった。これらはエディターに同梱され（リモートレジストリからダウンロードするのではなく、エディターのバンドルからインストールされる）、そのバージョンはエディターに紐づく。パッケージの変更履歴には、こう書かれている。

"Package is now a core package embedded in Unity."

この変更は `com.unity.entities`、`com.unity.collections`、`com.unity.mathematics`、`com.unity.entities.graphics` に及び、Netcode for Entities も対応する 6.5 リリースで同じ扱いになった。インストール方法やバージョンの管理・追跡の仕方は変わるが、変わったのは **配布の仕組み** であって API ではない。

本章で扱うのは次の内容だ。

- どのパッケージが Core Package になり、どれがそうでないか。
- バージョン番号がどう決まるか。
- `manifest.json` と `packages-lock.json` がどう変わるか。
- なぜこの移行が行われたのか。

2. Core Package とは何か

Core Package とは、**エディターに同梱される** パッケージのことだ。他のパッケージと同じように Package Manager からプロジェクトに追加するが、リモートレジストリではなくエディターのバンドルからインストールされ、バージョンをエディターとは別に変えることはできず、リリースサイクルもエディターに従う。（これは **Built-in パッケージ** — Physics のような、有効・無効を切り替えるだけのエンジンモジュール — とは別物だ。）

特性	Package Manager パッケージ	Core Package
インストール手順	UPM 経由で追加	Package Manager から追加 (エディターのバンドルからインストールされる)

特性	Package Manager パッケージ	Core Package
バージョンの決定者	ユーザー (manifest.json で)	Unity (エディターバージョンごと)
リリースサイクル	UPM で独立	エディターのリリースに従う
manifest.json に掲載	あり	あり — インストール時に追加される
packages-lock.json に掲載	あり	あり (builtin としてマーク)
Package Manager に表示	「In Project」の下	インストール時は「Unity Registry」、その後は「In Project」

3. Unity 6000.5+ における DOTS パッケージマップ

パッケージ	Unity 6000.5+ での状態	用途
Entities	Core Package (6.5)	ECS ランタイム : Entity、Component、System、World
Collections	Core Package	ネイティブコンテナ型 (NativeList<T>、NativeHashMap<K,V> など)
Mathematics	Core Package	Burst 最適化済みの float3、quaternion、int4 など
Entities Graphics	Core Package	SRP 経由で Entity ベースのグラフィックスをレンダリング
Netcode for Entities	Core Package (6.5)	Entities 向けクライアント・サーバーネットワーキング
Unity Physics	Package Manager	Entities 向けステートレスで決定論的な物理演算
Character Controller	Package Manager	ECS ベースのキャラクターコントローラー
Burst Compiler	エンジン組み込み機能	ジョブとシステムをタイトなネイティブコードにコンパイル
Job System	エンジン組み込み機能	マルチスレッドジョブスケジューリング

補足 : Job System と Burst はもともと UPM パッケージではなく、エンジンの一部だ。6000.4 の変更が対象としたのは、その上に乗る DOTS パッケージである。

4. 移行後のバージョン番号

Core Package のバージョンは **エディターに合わせて** 付けられる。

エディター	Core Entities
Unity 6000.4	Entities 6.4
Unity 6000.5	Entities 6.5

6000.4 以降にまだ移行できないプロジェクトのために、従来の 1.x ラインも引き続き提供されている。2 つのラインは独立しており、6.x へ上げるにはエディター自体をアップグレードする。

Entities の移行に関する解説は [Changelog/Entities 1.4 → 6.5 Key Changes.md](#)、Netcode 固有の変更点は [Changelog/Netcode for Entities 1.4 → 6.5 Key Changes.md](#) を参照してほしい。

5. プロジェクトファイルへの影響

manifest.json

Entities を追加すると、ここに `com.unity.entities` のエントリが現れる。それが取り込むパッケージ（`com.unity.collections`、`com.unity.mathematics`、`com.unity.entities.graphics`）は依存関係として解決され、通常は専用のエントリを持つ必要はない。1.x プロジェクトをアップグレードするなら、古い固定バージョンを置き換え、Package Manager にエディターの Core バージョンへ解決させる。詳しくは [Migration/02_Package Manager → Core Package.md](#) を参照。

packages-lock.json

Core Package は、バージョン URL ではなく `"source": "builtin"` として記載される。

アセンブリ定義 (.asmdef)

.asmdef ファイルは、これまでどおりアセンブリ名（`Unity.Entities`、`Unity.Collections` など）を参照する。変わったのはパッケージの配布レイヤーだけなので、ここに手を入れる必要はない。

6. 移行の理由

Unity が掲げる狙いは、DOTS の更新を独立したパッケージのスケジュールではなくエディターのリリースに紐づけ、ECS チームが機能をより速く、より小刻みに届けられるようにすることだ。あわせて、エンジンが内部で ECS に依存できるようにもなる。パッケージが任意導入だったころは、これが難しかった。

7. トラブルシューティング

症状	原因 / 対処法
Package Manager で Entities が古い 1.x バージョンでしか提供されない	エディターが 6000.4 より古い。 エディターをアップグレードするか、1.x ラインに留まる。
<code>com.unity.entities</code> のバージョンを変更した後に <code>manifest.json</code> の解決エラーが出る	<code>packages-lock.json</code> を削除し、エディターに再生成させる。
サンプルやアセットストアのパッケージが <code>com.unity.entities@1.x</code> を対象にしている	6000.5 以降では 1.x の API はほぼ置き換えられている。 対応する API は Migration フォルダーで確認する。
2つの Editor インストールで、表示される Entities バージョンが異なる	正常な動作だ。 バージョンはエディターのインストールごとに決まる。

Chapter 3. Hello DOTS — はじめての Entity

1. 作るもの

ECS システムで Y 軸まわりに回転するキューブを作る。この作業ひとつで、コアコンセプトをひとつおり体験できる。**SubScene**、**オーサリング** → **baker**、**IComponentData**、**ISystem**、そして **Entities Hierarchy / Systems** ウィンドウだ。最後にはメインスレッドのシステムを **IJobEntity** に置き換え、並列処理がどう入り込むかを見ていく。

前提：[01_Environment Setup.md](01_Environment Setup (Unity 6000.5 + Entities 6.5.0).md) — SubScene と Entities メニューの動作を確認済みの Unity 6000.5 プロジェクト。

2. SubScene の作成

1. 空のシーンを開く。
2. **Hierarchy** で右クリック → **New Sub Scene** → **Empty Scene...**
3. **HelloDots.unity** という名前で **Assets/Scenes/** に保存する。
4. 新しい SubScene の中で右クリック → **3D Object** → **Cube**。名前を **RotatingCube** に変える。
5. シーンを保存する (**Ctrl/Cmd + S**)。Hierarchy で名前の横のチェックボックスをクリックして SubScene を閉じると、アイコンが黄色になる。これは中身が Entity へベイクされたことを表す。

これ以降、**RotatingCube** のコンポーネントを変更すると、SubScene を開閉したタイミングで自動的に再ベイクが走る。

3. コンポーネントの定義 — **RotationSpeed**

Assets/Scripts/RotationSpeed.cs を作成する：

```
using Unity.Entities;

public struct RotationSpeed : IComponentData
{
```



```
public float RadiansPerSecond;
}
```

注記：

- アンマネージドな struct に IComponentData を付けるのが基本形だ。チャンク内に直接置かれ、Burst と相性がよい。
- フィールドは blittable に保つ（プリミティブ型、Unity.Mathematics の型、ほかのアンマネージド構造体）。

4. オーサリング + baker の定義

Assets/Scripts/RotationSpeedAuthoring.cs を作成する：

```
using Unity.Entities;
using Unity.Mathematics;
using UnityEngine;

public class RotationSpeedAuthoring : MonoBehaviour
{
    public float DegreesPerSecond = 90f;

    class Baker : Baker<RotationSpeedAuthoring>
    {
        public override void Bake(RotationSpeedAuthoring authoring)
        {
            // Dynamic – this entity will have its transform written each frame.
            var entity = GetEntity(TransformUsageFlags.Dynamic);

            AddComponent(entity, new RotationSpeed
            {
                RadiansPerSecond = math.radians(authoring.DegreesPerSecond)
            });
        }
    }
}
```

オーサリングをキューブにアタッチする

1. SubScene 内で RotatingCube を選択する。
2. **Inspector** → **Add Component** → **Rotation Speed Authoring**。
3. DegreesPerSecond = 90 のままにする。

SubScene を閉じた瞬間（アイコンが黄色になる）に baker が走り、キューブは RotationSpeed コンポーネントを持つ Entity へと変換される。

ベイクの確認

- **Window** → **Entities** → **Hierarchy** に RotatingCube が並んでいるはずだ。
- クリックすると、右側の **Inspector** にコンポーネントが出る。RotationSpeed { RadiansPerSecond: ~1.5708 } ($\pi/2$) があるか確かめる。

5. システムの記述 — RotationSystem

Assets/Scripts/RotationSystem.cs を作成する：

```
using Unity.Burst;
using Unity.Entities;
using Unity.Mathematics;
using Unity.Transforms;

[BurstCompile]
public partial struct RotationSystem : ISystem
{
    [BurstCompile]
    public void OnUpdate(ref SystemState state)
    {
        float dt = SystemAPI.Time.DeltaTime;

        foreach (var (transform, speed) in
            SystemAPI.Query<RefRW<LocalTransform>, RefRO<RotationSpeed>>())
        {
            transform.ValueRW = transform.ValueRO.RotateY(speed.ValueRO.RadiansPerSecond * dt);
        }
    }
}
```

注記：

- partial struct + ISystem は **アンマネージドシステム** の書き方だ。Burst コンパイルでき、GC のオーバーヘッドなしで動く。
- SystemAPI.Query<RefRW<A>, RefRO>() は今主流のイテレーション API で、廃止された Entities.ForEach の後継にあたる。
- LocalTransform.RotateY(radians) は回転後のコピーを返すので、transform.ValueRW = ... で書き戻す。

Play を押す

キューブが回り出す。**Window** → **Entities** → **Systems** を見ると、シミュレーショングループの下に RotationSystem がフレームごとの処理時間付きで並んでいるはずだ。

6. 作ったものを確認する

Play モード中に役立つウィンドウを挙げる。

ウィンドウ	表示内容
Entities Hierarchy	Entity のリアルタイム一覧。Entity をクリックすると、そのチャンクとコンポーネントを確認できる。
Systems	実行中の全システムを SystemGroup ごとにまとめて表示する。 システム単位の処理時間もここで分かる。
Archetypes	コンポーネント構成ごとにまとめたチャンク。断片化の発見に役立つ。

次の点を確認しておこう。

- 1. RotatingCube Entity が存在し、LocalTransform + RotationSpeed を持っている。
- 2. RotationSystem が SimulationSystemGroup 配下に存在し、フレーム時間を蓄積している。

7. IJobEntity に切り替える（並列処理）

並列処理がどう組み込まれるかを見るため、RotationSystem.cs をジョブベースの実装に置き換える。

```
using Unity.Burst;
using Unity.Entities;
using Unity.Mathematics;
using Unity.Transforms;

[BurstCompile]
public partial struct RotateJob : IJobEntity
{
    public float DeltaTime;

    void Execute(ref LocalTransform transform, in RotationSpeed speed)
    {
        transform = transform.RotateY(speed.RadiansPerSecond * DeltaTime);
    }
}

[BurstCompile]
public partial struct RotationSystem : ISystem
{
    [BurstCompile]
    public void OnUpdate(ref SystemState state)
    {
        new RotateJob { DeltaTime = SystemAPI.Time.DeltaTime }
            .ScheduleParallel();
    }
}
```

注記：

- `IJobEntity.Execute(ref A, in B)` は、ソースジェネレーターによってチャンク単位のジョブに変換される。パラメーターには、扱うコンポーネントを並べる。
- `.ScheduleParallel()` はチャンクをワーカースレッドに振り分ける。キューブ 1 個では高速化は体感できないが、このコードの形のまま、数千の Entity まで手を加えずにスケールする。
- システムの依存関係は自動で面倒を見てくれる。`ISystem.OnUpdate` から `ScheduleParallel()` を呼べば、`state.Dependency` が自動的に組み合わせられる。

8. 次に読むもの

やりたいこと	参照先
オーサリングが Entity にどうマッピングされるかを詳しく知る	DOTS Workflows/01_Baker Pattern & SubScene.md
実行時に大量の Entity をスポンする	DOTS Workflows/02_Spawner Example.md
Entity / Component / Chunk を理解する	DOTS Workflows/03_ECS Core Concepts.md
ISystem と SystemBase を使い分ける	DOTS Workflows/08_System – ISystem vs SystemBase.md

9. トラブルシューティング

症状	原因 / 対処法
キューブが Entities Hierarchy に表示されない	SubScene がまだ開いている（アイコンが黄色ではなく白）。Hierarchy のチェックボックスで閉じる。
キューブは表示されるが RotationSpeed コンポーネントがない	<code>RotationSpeedAuthoring</code> をアタッチし忘れたか、追加後に SubScene が再ベイクされていない。SubScene のチェックボックスをオフ・オンしてみる。
キューブが回転しない	Systems ウィンドウに <code>RotationSystem</code> がない。よくある原因は、 <code>struct</code> に <code>partial</code> キーワードを付け忘れたか、スクリプトにコンパイルエラーがあることだ。
コンパイルエラー： <code>TransformUsageFlags does not exist</code>	オーサリングファイルに <code>using Unity.Entities;</code> が足りていない。
コンパイルエラー： <code>math.radians</code> <code>does not exist</code>	<code>using Unity.Mathematics;</code> が足りていない。

症状	原因 / 対処法
SystemAPI.Query<...>が見つからない	<code>using Unity.Entities;</code> が足りていないか、struct に <code>partial</code> が付いていない。
Inspector で Entity に LocalTransform がない	baker が <code>GetEntity(TransformUsageFlags.Dynamic)</code> ではなく <code>None</code> を渡している。フラグを変更する。
ScheduleParallel() で「LocalTransform への競合状態」例外が発生する	別のシステムが、順序付けなしに同じフレームで <code>LocalTransform</code> へ書き込んでいる。 <code>[UpdateBefore]</code> / <code>[UpdateAfter]</code> を付けるか、共有の <code>EntityCommandBuffer</code> を使う。

Part II. DOTS ワークフロー

Chapter 4. Baker パターン & SubScene

1. 概要

Entity は Hierarchy で直接オーサリングできない。そのかわり、**SubScene** の中で **GameObject** をオーサリングし、SubScene を閉じるかビルドするタイミングで **Baker** がそれらを Entity へ変換する。

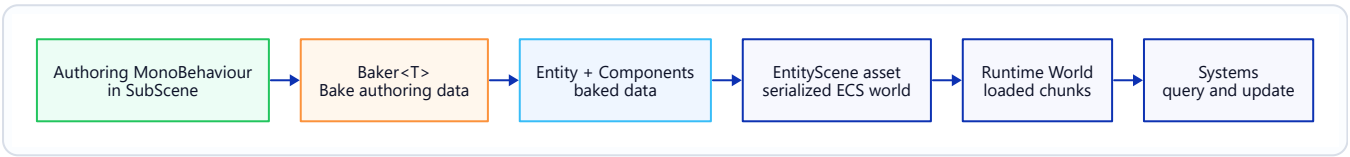


Figure: Baker と SubScene のパイプライン。

2. SubScene のライフサイクル

SubScene には、見た目で分かる 3 つの状態がある。

Hierarchy のアイコン	状態	意味
 (白いチェックボックス)	Open (開)	GameObject を編集できる。まだ GameObject のままで、ベイクは行われない。
 (黄色のチェックボックス)	Closed (閉)	Baker が実行済み。中身は Entity としてのみ存在する。
	Baking (ベイク中)	インクリメンタルベイクが進行中 (一時的)。

SubScene の作成

- **Hierarchy** → **右クリック** → **New Sub Scene** → **Empty Scene...** を選ぶと、.unity アセットが保存され、親 GameObject に SubScene コンポーネントが追加される。
- SubScene 名の横のチェックボックスで開閉する。閉じるとベイクが走る。

インクリメンタルベイク

SubScene を開いた状態で GameObject を編集すると、変更したオーサリングに関する Baker だけが再実行される。だからこそ、**Bake()** の中で重い処理は避けたい。

3. Baker<T> API

Baker は、1 つのオーサリング MonoBehaviour を 1 つ以上の Entity とコンポーネントへ対応づける。

```
public class EnemyAuthoring : MonoBehaviour
{
    public float MaxHealth = 100f;
    public GameObject ProjectilePrefab;

    class Baker : Baker<EnemyAuthoring>
    {
        public override void Bake(EnemyAuthoring authoring)
        {
            // Primary entity for this GameObject.
            var self = GetEntity(TransformUsageFlags.Dynamic);

            AddComponent(self, new Health { Value = authoring.MaxHealth });

            // Bake a GameObject reference into an Entity reference.
            AddComponent(self, new ProjectilePrefabRef
            {
                Value = GetEntity(authoring.ProjectilePrefab, TransformUsageFlags.Dynamic)
            });
        }
    }
}
```

主な API は次のとおり。

API	用途
GetEntity(TransformUsageFlags)	現在のオーサリング GameObject の Entity。
GetEntity(Object, TransformUsageFlags)	別の GameObject や Prefab に対応する Entity (Prefab Entity として登録される)。
CreateAdditionalEntity(TransformUsageFlags)	同じオーサリングに紐づく 2 つ目の Entity (サブパーツなどに使う)。
AddComponent<T>(entity, value)	コンポーネントデータを付与する。
AddBuffer<T>(entity) / SetBuffer<T>	DynamicBuffer<T> を付与する。
AddSharedComponent<T>	ISharedComponentData を付与する。
DependsOn(asset)	アセットへの依存を宣言し、そのアセットが変わったら Baker を再実行させる。

4. TransformUsageFlags

ベイクが LocalTransform / LocalToWorld / Parent を追加するのは、TransformUsageFlags で明示的に要求したときだけだ。使わないトランスフォームコンポーネントはチャンクを無駄に肥大させるので、要件を満たす範囲でもっとも狭いフラグを選ぶ。

フラグ	追加されるもの	典型的な用途
None	なし	純粋なデータ Entity（設定、Singleton、ロジックのみ）。
ManualOverride	なし；トランスフォームは自分で管理する	カスタムトランスフォームレイアウト。
Renderable	LocalToWorld	決して動かない静的メッシュ。
Dynamic	LocalTransform、LocalToWorld （親子関係がある場合は Parent も追加）	System が移動・回転させるもの全般。
WorldSpace	LocalTransform、LocalToWorld。 親子関係は付かない	階層を無視する Entity。
NonUniformScale	PostTransformMatrix を追加	非均一スケール（DOTS では稀）。

System で移動・回転させるものには、Dynamic を選んでおけばまず間違いない。

5. Baking System（上級）

Baker は 1 つのオーサリングからコンポーネントを生成する。一方 **Baking System** は、すべての Baker が走った後に動き、ベイク済みの World 全体を見渡せる。Entity をまたいだ後処理（参照の解決や派生データの計算）に向いている。

```
[WorldSystemFilter(WorldSystemFilterFlags.BakingSystem)]
public partial struct EnemyWaveLinkSystem : ISystem
{
    public void OnUpdate(ref SystemState state)
    {
        // Runs during bake only. Query entities tagged earlier by Bakers
        // and wire them together.
    }
}
```

Baker だけでは必要な情報がそろわないとき（個数の集計や Entity 間の相互参照など）に使う。処理は決定論的に保つ。ベイクは毎回同じ結果にならないといけない。

6. 例 — 弾スポナーをオーサリングする

オーサリング：

```
using Unity.Entities;
using UnityEngine;

public class ProjectileSpawnerAuthoring : MonoBehaviour
{
    public GameObject Prefab;
    public float Interval = 0.5f;

    class Baker : Baker<ProjectileSpawnerAuthoring>
    {
        public override void Bake(ProjectileSpawnerAuthoring authoring)
        {
            var self = GetEntity(TransformUsageFlags.Dynamic);

            AddComponent(self, new ProjectileSpawner
            {
                Prefab = GetEntity(authoring.Prefab, TransformUsageFlags.Dynamic),
                Interval = authoring.Interval,
                TimeLeft = authoring.Interval
            });
        }
    }
}
```

コンポーネント：

```
using Unity.Entities;

public struct ProjectileSpawner : IComponentData
{
    public Entity Prefab;
    public float Interval;
    public float TimeLeft;
}
```

あとはランタイムシステムが、タイマーに合わせて **Prefab** をインスタンス化すればよい。

`02_Spawner Example.md` を参照。

7. トラブルシューティング

症状	原因 / 対処法
Entity が表示されるがコンポーネン トが欠けている	Baker が実行されていない（SubScene を閉じた後にオーサリングを追加した）。 SubScene を開閉して再ベイクする。

症状	原因 / 対処法
GetEntity(prefab, ...) が Entity.Null を返す	参照先の GameObject が Prefab アセットでないか、Baker から到達できないシーンオブジェクトになっている。Prefab であること、または同じ SubScene 内にあることを確認する。
エディター内で Baker が毎フレーム実行される	Bake() に非決定論的なロジック (Random や Time.time など) が混ざっている。Baker はオーサリングを入力とする純粋関数でなければならない。
Entity に LocalToWorld はあるが LocalTransform がない	TransformUsageFlags.Renderable を使っている。Entity が移動するなら Dynamic に切り替える。
オーサリングフィールドの変更が再ベイクされない	Baker がそのフィールドを読んでいないか、外部アセットに対して DependsOn を呼び忘れている。
ベイクが遅い	Bake() の中で重い処理をしている。Entity をまたぐロジックは、Baking System か、ランタイムの一回限りの初期化処理に移す。

Chapter 5. Spawner サンプル

1. 概要

Spawner とは、ほかの Entity を生み出す Entity 一つのことだ。起動時、タイマー、あるいはイベントをきっかけに生成する。本章ではこれを最初から最後まで作り上げ、次の点を解説する。

- SubScene 内での Spawner のオーサリング。
- **Prefab Entity** 参照の保持（GameObject Prefab からベイクしたもの）。
- 構造変更を安全に行うため、**EntityCommandBuffer** を介してランタイムにインスタンス化する方法。

先に 01_Baker Pattern & SubScene.md に目を通しておくと、話がつきやすい。

2. データモデル

```
using Unity.Entities;
using Unity.Mathematics;

public struct Spawner : IComponentData
{
    public Entity    Prefab;        // baked prefab entity
    public float3    SpawnPoint;
    public float     Interval;
    public float     TimeLeft;
    public int       RemainingCount; // stop when 0
}
```

ここで触れておきたい設計判断が 2 つある。

- Prefab は GameObject ではなく **Entity** 型だ。Baker が GameObject Prefab を Prefab Entity へ変換するので、ランタイムが目にするのは Entity だけになる。
 - TimeLeft と RemainingCount を Spawner Entity 側に持たせることで、System 自体はステートレスに保てる。
-

3. オーサリング + Baker

```
using Unity.Entities;
using Unity.Mathematics;
using UnityEngine;

public class SpawnerAuthoring : MonoBehaviour
{
    public GameObject Prefab;
    public float Interval = 0.5f;
    public int Count = 100;

    class Baker : Baker<SpawnerAuthoring>
    {
        public override void Bake(SpawnerAuthoring authoring)
        {
            var self = GetEntity(TransformUsageFlags.Dynamic);

            AddComponent(self, new Spawner
            {
                Prefab = GetEntity(authoring.Prefab, TransformUsageFlags.Dynamic),
                SpawnPoint = authoring.transform.position,
                Interval = authoring.Interval,
                TimeLeft = 0f, // fires immediately on first update
                RemainingCount = authoring.Count
            });
        }
    }
}
```

SubScene 内の空の GameObject に SpawnerAuthoring をドロップし、Prefab を割り当て、生成数を設定する。SubScene を閉じれば、Spawner は Entity になる。

4. ランタイムシステム

```
using Unity.Burst;
using Unity.Entities;
using Unity.Mathematics;
using Unity.Transforms;

[BurstCompile]
public partial struct SpawnerSystem : ISystem
{
    [BurstCompile]
    public void OnCreate(ref SystemState state)
    {
        state.RequireForUpdate<Spawner>();
    }

    [BurstCompile]
    public void OnUpdate(ref SystemState state)
    {
        float dt = SystemAPI.Time.DeltaTime;
```

```

// Use the EndSimulation ECB so structural changes flush at a safe point.
var ecb = SystemAPI
    .GetSingleton<EndSimulationEntityCommandBufferSystem.Singleton>()
    .CreateCommandBuffer(state.WorldUnmanaged);

foreach (var (spawnerRW, spawnerEntity) in
    SystemAPI.Query<RefRW<Spawner>>().WithEntityAccess())
{
    ref var spawner = ref spawnerRW.ValueRW;

    if (spawner.RemainingCount <= 0)
    {
        ecb.DestroyEntity(spawnerEntity);    // done – remove the spawner
        continue;
    }

    spawner.TimeLeft -= dt;
    if (spawner.TimeLeft > 0f) continue;

    spawner.TimeLeft = spawner.Interval;
    spawner.RemainingCount--;

    var instance = ecb.Instantiate(spawner.Prefab);
    ecb.SetComponent(instance, LocalTransform.FromPosition(spawner.SpawnPoint));
}
}
}

```

重要なポイント：

- `RequireForUpdate<Spawner>()` を入れておくと、`Spawner` が一つもなければシステムごとスキップされる。
- `EndSimulationEntityCommandBufferSystem` の `Singleton` は、フレーム終了時にフラッシュされる共有 ECB を提供する。ループの途中で `EntityManager` を直接いじるより安全だ。
- `ecb.Instantiate(prefab)` は **遅延 Entity** を生成し、`ecb.SetComponent(instance, ...)` がそれへのコンポーネント書き込みを予約する。`instance` ハンドルが有効なのは、この ECB のプレイバック中だけだ。14_EntityCommandBuffer・Deferred Entity.md を参照。

5. スケールアップ — 並列 Spawner

多数の `Spawner` から 1 フレームに数千体を生成したいなら、並列 ECB ライターを使う `IJobEntity` に作り替える。

```

[BurstCompile]
public partial struct SpawnJob : IJobEntity
{
    public float                                DeltaTime;
    public EntityCommandBuffer.ParallelWriter ECB;
}

```

```

void Execute([ChunkIndexInQuery] int chunkIndex,
            Entity entity,
            ref Spawner spawner)
{
    if (spawner.RemainingCount <= 0)
    {
        ECB.DestroyEntity(chunkIndex, entity);
        return;
    }

    spawner.TimeLeft -= DeltaTime;
    if (spawner.TimeLeft > 0f) return;

    spawner.TimeLeft = spawner.Interval;
    spawner.RemainingCount--;

    var inst = ECB.Instantiate(chunkIndex, spawner.Prefab);
    ECB.SetComponent(chunkIndex, inst, LocalTransform.FromPosition(spawner.SpawnPoint));
}
}

```

System 側はこうなる。

```

var ecb = SystemAPI
    .GetSingleton<EndSimulationEntityCommandBufferSystem.Singleton>()
    .CreateCommandBuffer(state.WorldUnmanaged)
    .AsParallelWriter();

new SpawnJob { DeltaTime = SystemAPI.Time.DeltaTime, ECB = ecb }
    .ScheduleParallel();

```

並列プレイバックを決定論的に保つ鍵が、ソートキーとしての `chunkIndex` だ。

15_ParallelWriter ・ Deterministic Playback.md を参照。

6. トラブルシューティング

症状	原因 / 対処法
何もスポーンされない	Spawner Entity が存在しない。SubScene を開いて SpawnerAuthoring がアタッチ済みかを確認し、SubScene を閉じる（アイコンが黄色になっているか確認する）。
Prefab がワールド原点にスポーンされる	ECB が LocalTransform を設定していない。ecb.SetComponent(instance, LocalTransform.FromPosition(...)) を追加するか、Prefab 側にあらかじめ目的の位置のトランスフォームを持たせる。
Prefab が表示されるが動かない	Prefab が Dynamic ではなく TransformUsageFlags.Renderable でベイクされている。

症状	原因 / 対処法
SystemAPI.GetSingleton<...ECBS.Singleton>() が例外を投げる	ECB System が登録されていない。極端に切り詰めた World で起こる。 state.EntityManager と手動 ECB に切り替えるか、Bootstrap で ECB System を登録する。
Spawner が停止しない	RemainingCount を spawnerRW.ValueRW ではなく、コピーした Spawner に対してデクリメントしている。書き込みは必ず RefRW.ValueRW を通す。
遅延インスタンスに対する ecb.SetComponent が無視される	Instantiate と SetComponent で別々の ECB を使っている。どちらの呼び出しも同じ ECB インスタンスでなければならない。

Chapter 6. ECS コアコンセプト

1. 5 つの名詞

Entities ECS は、つまるところ 5 つの名詞に集約できる。

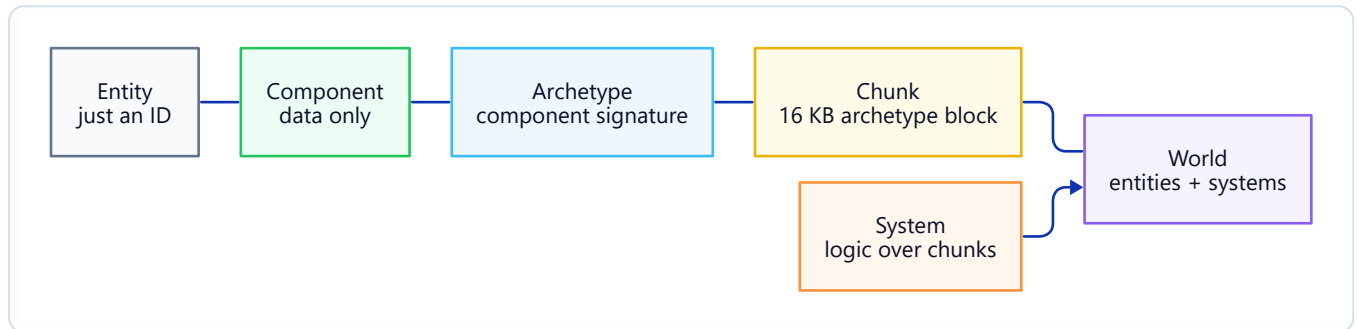


Figure: ECS コアの名詞。

この図が頭に入っていれば、既存の DOTS コードを読む準備はほぼ整ったといってよい。

2. Entity

Entity はただの 64 ビットのハンドルにすぎない。実体は `struct Entity { int Index; int Version; }` だ。データもメソッドも持たず、果たすのは識別という役割だけで、データはすべてコンポーネントが持つ。

```
Entity e = state.EntityManager.CreateEntity(); // fresh, no components
state.EntityManager.AddComponent<Health>(e); // now it has data
state.EntityManager.DestroyEntity(e); // handle becomes stale
```

古い参照を無効にするのが **Version** フィールドの役目だ。同じ **Index** で破棄と再生成が起きると、古い **Entity** 値はもう一致しなくなる。

3. Component

Component は純粋なデータの入れ物だ。振る舞いは持たない。コードは **System** が受け持つ。

```
public struct Health : IComponentData
{
    public float Value;
```

```
public float Max;  
}
```

コンポーネントには種類がいくつかある（アンマネージド、マネージド、バッファ、共有、クリーンアップ、タグ、チャンク、Enableable）。05_Component Types.md を参照。

4. Archetype

Archetype とは、複数の Entity が共有するコンポーネント型の組み合わせを一意に表したものだ。

- { LocalTransform, Health, Enemy } を持つ Entity は Archetype A に属する。
- { LocalTransform, Health, Enemy, Burning } を持つ Entity は Archetype B に属する。
Burning が加わってコンポーネントの構成が変わったため、別の Archetype になる。

Archetype 間の移動、つまりコンポーネントの追加や削除は **構造変更** にあたり、Entity を物理的に別のチャンクへ移す。13_Structural Change & Safety.md を参照。

5. Chunk

同じ Archetype に属する Entity は **16 KB のチャンク** へ詰め込まれる。チャンク内では、コンポーネント型ごとに連続した配列が作られる。これがキャッシュ局所性の高さにつながる。

特性	意味すること
チャンクは 16 KB	各コンポーネントが領域を取り合う。Archetype のコンポーネントが多いほど、また大きいほど、チャンクあたりの Entity 数は減る。
データはコンポーネントごとに列指向（SoA）に配置される	IJobChunk はコンポーネント配列を 1 本ずつたどれるため、Burst でベクトル化しやすい。
コンポーネントの追加/削除で Entity が移動する	構造変更が起きる。クエリのイテレーターは、反復中に Archetype が変わらないことを前提にしている。
チャンクはコンポーネントの変更バージョンを追跡する	System は、前回の実行から変更のないチャンクをスキップできる（EntityQuery.SetChangedVersionFilter）。

実行中のチャンクレイアウトは **Window → Entities → Archetypes** で確認できる。

6. System

System は毎フレーム実行されるロジックのまとまりで、ふつうは **Entity** のクエリに対して処理を行う。

```
[BurstCompile]
public partial struct HealthRegenSystem : ISystem
{
    [BurstCompile]
    public void OnUpdate(ref SystemState state)
    {
        float dt = SystemAPI.Time.DeltaTime;

        foreach (var health in SystemAPI.Query<RefRW<Health>>())
        {
            health.ValueRW.Value = math.min(health.ValueRW.Value + dt, health.ValueRW.Max);
        }
    }
}
```

System は自動的に登録され、いずれかの **SystemGroup** に属する。08_System – ISystem vs SystemBase.md と 09_System Group & Update Order.md を参照。

7. World

World は次のものを保持するコンテナだ。

- **EntityManager** (Entity の作成・破棄・クエリを担う)。
- **System** 群と、その SystemGroup。
- チャンクそのもの (EntityManager 経由で保持する)。

デフォルトのランタイム World は DefaultGameObjectInjectionWorld だ。製品版では多くのゲームが World を 1 つだけ持つ。複数 World の構成が出てくるのは、Netcode (サーバー / クライアント) やエディターツールの場面だ。Netcode の World 分割は 16_Netcode Client-Server World & Bootstrap.md を参照。

```
World world = World.DefaultGameObjectInjectionWorld;
EntityManager em = world.EntityManager;

Entity e = em.CreateEntity(typeof(Health));
em.SetComponentData(e, new Health { Value = 100, Max = 100 });
```

8. EntityManager

EntityManager は、構造変更を行うためのメインスレッド向け API だ。Entity の作成・破棄、コンポーネントの追加・削除、Prefab Entity のインスタンス化、World のコピーなどを扱う。

EntityManager を介した構造変更は即座に反映されるため、実行中のクエリイテレーターやジョブの参照を無効化することがある。

ジョブの中から、あるいはイテレーション中に構造変更を遅延させたいときは

EntityCommandBuffer を使う。14_EntityCommandBuffer ・ Deferred Entity.md を参照。

9. まとめ

```
World
├─ EntityManager
│   └─ Archetype { LocalTransform, Health, Enemy }
│       └─ Chunk (up to ~N entities)
│           └─ Entity { Index, Version }
│               ├── LocalTransform
│               ├── Health
│               └─ Enemy (tag)
```

{ LocalTransform, Health } をクエリする System は、**その両方を含む**すべてのチャンクを処理対象にする。上記の Archetype はもちろん、Enemy やさらに別のコンポーネントを併せ持つ Archetype も含まれる。

10. トラブルシューティング

症状	原因 / 対処法
Entity の検索が古いデータを返す	Entity が破棄・再生成され、Version が一致しなくなっている。Entity を取得し直す。
クエリが何もマッチしないが、Entity に明らかにコンポーネントがある	コンポーネントが別の World で追加されたか、Enableable コンポーネントが現在無効になっている。
Archetype の数が爆発する	コンポーネントの組み合わせが多すぎる（値の種類が多い Entity 別タグでよく起きる）。共有コンポーネントや Enableable フラグでまとめる。
チャンクに 1~2 体の Entity しかない	Archetype が大きすぎる。Entity に不要なコンポーネントを削るか、複数の Entity や Archetype に分ける。

症状	原因 / 対処法
イテレーション中に <code>InvalidOperationException: ... invalidated ...</code>	クエリの途中で構造変更が起きた。ECB を使うか、 ループの後にまとめて変更する。

Chapter 7. Entity 参照 — Entity・EntityPrefabReference・UnityObjectRef

1. 概要

DOTS のコードでは、数種類の参照型を使い分ける。どれも「何かを指す」点では同じなので混同しやすいが、オーサリングからランタイムへ至るパイプラインの中で、それぞれ立ち位置が違う。

型	名前空間	参照先	典型的な用途
Entity	Unity.Entities	1つの ECS World 内の Entity。	ランタイムの関係性とコンポーネントフィールド。
EntityPrefabReference	Unity.Entities.Serialization	あとからロードできる、ベイク済みの Entity Prefab アセット。	ストリーミングやコンテンツ管理のワークフロー。
UnityObjectRef<T>	Unity.Entities	アンマネージドドラッパー経由の UnityEngine.Object アセット/オブジェクト。	ECS データからメッシュ、マテリアル、テクスチャ、オーディオクリップ、オーサリングしたアセットを参照したいとき。

本章で扱うのは DOTS / Entities の参照型だけだ。エンジン全体にまたがる UnityEngine.Object の識別子 API は、本書の対象外とする。

2. Entity

Entity はランタイム ECS のハンドルだ。1つの World の中で、コンポーネントデータの 1 行を指し示すアンマネージドな値である。

```
using Unity.Entities;

public struct Target : IComponentData
{
    public Entity Value;
}
```

押さえておきたいルールは次のとおり。

- Entity 値が意味を持つのは、それを生成した World の中だけだ。

- Entity を破棄すると、古いハンドルは無効になる。
- Entity のインデックスは再利用される。直前にクエリで得たわけではない保存済みハンドルは、参照する前に `EntityManager.Exists(entity)` で確認する。
- 生の Entity 値を、安定した ID としてディスクに保存したり、独立した World 間でやり取りしたりしてはいけない。

```
if (state.EntityManager.Exists(target.Value))
{
    var transform = state.EntityManager.GetComponentData<LocalTransform>(target.Value);
}
```

3. ベイク時の Entity 参照

Baker がオーサリングオブジェクトを ECS データへ変換するとき、参照先のオーサリングオブジェクトや Prefab を Entity 参照に変えるには `GetEntity` を使う。

```
using Unity.Entities;
using UnityEngine;

public class SpawnerAuthoring : MonoBehaviour
{
    public GameObject Prefab;
}

public struct Spawner : IComponentData
{
    public Entity Prefab;
}

public class SpawnerBaker : Baker<SpawnerAuthoring>
{
    public override void Bake(SpawnerAuthoring authoring)
    {
        var entity = GetEntity(TransformUsageFlags.None);
        AddComponent(entity, new Spawner
        {
            Prefab = GetEntity(authoring.Prefab, TransformUsageFlags.Dynamic)
        });
    }
}
```

得られるのはランタイム ECS の参照であって、マネージドな `GameObject` 参照ではない。

4. EntityPrefabReference

EntityPrefabReference は、現在のシーンに Entity 参照として直接埋め込むのではなく、Entities のコンテンツパイプライン経由でロードしたい Prefab コンテンツに使う。

```
using Unity.Entities;
using Unity.Entities.Serialization;
using UnityEngine;

public class ProjectileLibraryAuthoring : MonoBehaviour
{
    public GameObject ProjectilePrefab;
}

public struct ProjectileLibrary : IComponentData
{
    public EntityPrefabReference ProjectilePrefab;
}

public class ProjectileLibraryBaker : Baker<ProjectileLibraryAuthoring>
{
    public override void Bake(ProjectileLibraryAuthoring authoring)
    {
        var entity = GetEntity(TransformUsageFlags.None);
        AddComponent(entity, new ProjectileLibrary
        {
            ProjectilePrefab = new EntityPrefabReference(authoring.ProjectilePrefab)
        });
    }
}
```

Prefab がコンテンツ管理やストリーミングのワークフローに属するなら EntityPrefabReference を使う。Prefab が同じ SubScene にベイクされ、シーンと一緒に使えるなら、通常の Entity Prefab 参照でよい。

5. UnityObjectRef<T>

UnityObjectRef<T> を使えば、コンポーネントをマネージドコンポーネントにしなくても、アンマネージドな ECS データから UnityEngine.Object を参照できる。

```
using Unity.Entities;
using UnityEngine;

public struct ProjectileVisual : IComponentData
{
    public UnityObjectRef<Mesh> Mesh;
    public UnityObjectRef<Material> Material;
}
```

オーサリングデータからベイクするコードは次のようになる。

```

using Unity.Entities;
using UnityEngine;

public class ProjectileVisualAuthoring : MonoBehaviour
{
    public Mesh Mesh;
    public Material Material;
}

public class ProjectileVisualBaker : Baker<ProjectileVisualAuthoring>
{
    public override void Bake(ProjectileVisualAuthoring authoring)
    {
        var entity = GetEntity(TransformUsageFlags.Dynamic);
        AddComponent(entity, new ProjectileVisual
        {
            Mesh = new UnityObjectRef<Mesh>(authoring.Mesh),
            Material = new UnityObjectRef<Material>(authoring.Material)
        });
    }
}

```

.Value による参照解決は、メインスレッドかマネージドなプレゼンテーションコードでのみ行う。

```

Mesh mesh = visual.Mesh.Value;
Material material = visual.Material.Value;

```

ルールは次のとおり。

- 格納されるラッパーはアンマネージドなので、IComponentData の中に置ける。
- 参照先のオブジェクト自体はマネージドな Unity オブジェクトのままで、.Value は Burst ジョブ内では使えない。
- 参照を解決する時点で、参照先オブジェクトがロードされている必要がある。

6. 適切な参照の選択

必要なもの	使用する型
同じ World 内の別の Entity を指す	Entity
同じ SubScene にベイクされた Prefab を保存する	GetEntity(authoring.Prefab, ...) からの Entity Prefab 参照
コンテンツストリーミング/遅延ロード向けの Prefab を保存する	EntityPrefabReference
アンマネージド ECS データにメッシュ、マテリアル、テクスチャ、オーディオクリップ、ScriptableObject 参照を保存する	UnityObjectRef<T>

必要なもの	使用する型
セッションをまたいで Entity を永続化する	生の Entity ではなく、ゲーム固有の安定した ID

7. トラブルシューティング

症状	原因 / 対処法
保存した Entity がもはや存在しない	Entity が破棄されたか、別の World に属している。取得し直すか <code>EntityManager.Exists</code> で確認する。
ベイク後に Prefab 参照が <code>Entity.Null</code> になる	Baker が Prefab に対して <code>GetEntity</code> を呼んでいないか、Prefab がベイク済みシーンやコンテンツワークフローに含まれていない。
<code>UnityObjectRef<T>.Value</code> が <code>null</code> を返す	参照先の Unity オブジェクトがロードされていないか、破棄されている。参照解決の前にチェックを入れ、アセットを再ロードまたは取得し直す。
Burst ジョブでメッシュ/マテリアル参照を使えない	<code>UnityObjectRef<T></code> の格納部分はアンマネージドだが、 <code>.Value</code> はマネージドな Unity オブジェクトにアクセスする。参照解決はメインスレッドのプレゼンテーションコードに移す。
セーブファイルに生の Entity 値が保存されている	生の Entity ハンドルは World 内でのみ有効で、永続的な安定性はない。代わりにゲーム固有の ID かコンテンツキーを保存する。

Chapter 8. Component 型

1. 概要

ECS でいう「コンポーネント」とは、Entity にアタッチされるデータの入れ物だ。Entities には、ストレージ・ライフタイム・コストの性質が異なる種類がいくつか用意されている。適切なものを選ぶかどうかで、ECS System 設計の半分か決まる。

一覧で示すと次のとおり。

インターフェース / 形式	ストレージ	変更時に構造変更が発生するか	典型的な用途
IComponentData (アンマネージド struct)	チャンク内、SoA	なし（値の変更時）	デフォルトのコンポーネント
IComponentData (マネージド class)	サイドテーブル	なし	参照型（メッシュ、GameObject 参照）
IBufferElementData	チャンク内 + あふれた分はヒープ	なし	Entity ごとの動的配列
ISharedComponentData	チャンクごと	あり（Entity が別のチャンクに移る）	設定を共有する Entity のグループ化
ICleanupComponentData	チャンク内	通常	Entity 破棄時のクリーンアップ手続き
ICleanupSharedComponentData	チャンクごと	通常	共有リソースのクリーンアップ手続き
タグ（空の IComponentData）	チャンク内でサイズゼロ	なし	データを持たないフィルター 用マーカー
チャンクコンポーネント	チャンクごと	あり	チャンクごとに一度だけ計算 するメタデータ
Enableable (IEnableableComponent)	チャンク内 + ビットマスク	なし	Entity ごとに振る舞いを ON/OFF する

この章の残りで、表の各行を順に掘り下げていく。

2. アンマネージド IComponentData

基本となる形式だ。隙間なく詰めた SoA データとして、チャンク内に直接置かれる。Burst と相性がよい。

```
public struct Velocity : IComponentData
{
    public float3 Value;
}
```

ルールは次のとおり。

- **struct** でなければならない。参照フィールドは使えない (**class**、**string**、**List<T>** は不可)。
- 使えるフィールド型は、プリミティブ型、**Unity.Mathematics** の型、ほかのアンマネージド構造体、**Entity**、**FixedString***、**BlobAssetReference<T>**、**UnityObjectRef<T>** など。
- **SystemAPI.Query** からは、読み取りに **RefRO<T>**、書き込みに **RefRW<T>** を使う。

3. マネージド IComponentData

Entity をキーにしたサイドテーブルに格納される。参照型を扱える反面、GC アロケーションのコストがかかり、そのコンポーネントは **Burst** 非対応になる。

```
public class EnemyAI : IComponentData
{
    public AIModel Brain;        // a managed class
    public List<Entity> Allies;
}
```

マネージド参照がどうしても必要なとき（メッシュ、テクスチャ、オーサリングした **ScriptableObject** など）に使う。**UnityEngine.Object** への弱参照がほしだけなら、アンマネージドコンポーネントの中で **UnityObjectRef<T>** を使うほうがよい。

4. IBufferElementData — 動的バッファ

DynamicBuffer<T> は、**Entity** ごとに持てるサイズ可変の配列だ。

```
public struct Waypoint : IBufferElementData
{
    public float3 Position;
}
```

```
var buffer = state.EntityManager.AddBuffer<Waypoint>(entity);
buffer.Add(new Waypoint { Position = new float3(1, 0, 0) });
```

格納先は、`InternalBufferCapacity` の要素数までがチャンク内、それを超えた分はヒープにあふれる。容量は `[InternalBufferCapacity(N)]` で調整する。

5. ISharedComponentData — グループ化

共有コンポーネントは **チャンクをグループ分けする値** を持つ。`ISharedComponentData` の値が同じ Entity 同士は同じチャンクに入り、値が違えば別のチャンクになる。

```
public struct RenderMeshGroup : ISharedComponentData
{
    public int MaterialID;
}
```

その結果、次のような性質を持つ。

- 値を変えると Entity が別のチャンクへ移る。つまり **構造変更** となり、コストが高い。
- 頻繁には変わらないが、グループごとに値が異なるデータに向く（マテリアルセット、チーム ID、ベイク領域など）。
- 型が参照を含む場合、値はマネージドテーブルに保存される（`ISharedComponentData` はマネージドにもできる）。

6. クリーンアップコンポーネント — ICleanupComponentData

Entity に印を付けておくと、その Entity が「破棄」されても、専用システムが処理を終えるまではクリーンアップコンポーネントだけを残して生き続ける。

```
public struct ConnectionCleanup : ICleanupComponentData
{
    public int ConnectionHandle;
}

// DestroyEntity(e) removes all normal components but leaves ConnectionCleanup
// behind so you can close the socket in a dedicated system, then destroy the
// cleanup component explicitly.
```

対になる `ICleanupSharedComponentData` は、同じ仕組みを共有リソースに適用したものだ。

使いどころは、ネイティブリソースの解放、外部システムからの登録解除、フレームをまたいだ段階的な破棄など。

7. タグコンポーネント

タグは、中身が空の `IComponentData` だ。

```
public struct Enemy : IComponentData {}
```

チャンク内ではゼロバイトで、ほかのコンポーネントと同じようにクエリできる。「X の一種である」という印に最適だ。ただしタグを使いすぎる（数百種類も用意する）と `Archetype` が断片化する。値の種類が多い状態には、`Enableable` コンポーネントや共有コンポーネントを使う。

8. チャンクコンポーネント

チャンクコンポーネントは、Entity ごとではなく **チャンクごとに 1 つの値** を持つ。

```
state.EntityManager.AddChunkComponentData<RegionBounds>(query, new RegionBounds { ... });
```

典型的な用途は、チャンク内のすべての Entity に共通する事前計算データだ（バウンディング、リージョン ID、パーティションキーなど）。チャンクコンポーネントを変更すると構造変更が起きる。

9. Enableable コンポーネント (`IEnableableComponent`)

詳しくは専用の章 `06_Enableable Component.md` を参照。要点だけ述べると、構造変更なしで Entity ごとに ON/OFF できるコンポーネントで、チャンクごとのビットマスクで切り替える。

```
public struct Stunned : IComponentData, IEnableableComponent
{
    public float Duration;
}

state.EntityManager.SetComponentEnabled<Stunned>(entity, false); // no structural change
```

クエリは、`Enableable` コンポーネントが無効になっている Entity を自動でスキップする。システムから Entity を一時的に外す手段として、低コストでよく使われる。

10. 判断ガイド

やりたいこと	使用するもの
Entity ごとの数値データを保存する	アンマネージド IComponentData
Entity ごとにマネージド参照を保持する	マネージド IComponentData または UnityObjectRef<T>
Entity ごとに可変長リストを保存する	IBufferElementData
バッチ処理のために Entity をグループ化する	ISharedComponentData
フレームごとにコンポーネントを ON/OFF する	Enableable コンポーネント
型をマークするだけでよい	タグ（空の IComponentData）
チャンク全体のデータを事前計算する	チャンクコンポーネント
Entity が破棄されたときにコードを実行する	クリーンアップコンポーネント

11. トラブルシューティング

症状	原因 / 対処法
InvalidOperationException: The type T must be unmanaged	アンマネージドが求められる箇所でマネージド型を使っている。コンポーネントか API を切り替える（ComponentLookup<T> と ManagedAPI.GetComponent<T> の使い分けなど）。
共有コンポーネント値の設定でカクつく	変更のたびに構造変更が起きる。ECB でまとめて処理するか、この値はめったに変えないと割り切る。
DynamicBuffer<T> がヒープにあふれる	[InternalBufferCapacity(N)] が実際の使用量に対して小さすぎる。値を増やすか、サイズを一定範囲に抑える。
DestroyEntity 後に Entity が再出現する	クリーンアップコンポーネントがまだ付いたままになっている。クリーンアップを終えたら明示的に削除する。
Archetype が多すぎる	タグが増えすぎている。Entity ごとに固有な「状態」は Enableable コンポーネントに置き換える。
タグコンポーネントがクエリをフィルタリングしない	タグコンポーネントはデータを持たないため、SystemAPI.Query<...> のタプルで RefRW<Tag> や RefRO<Tag> を要求できない。代わりに .WithAll<Tag>() や .WithNone<Tag>() を使う。

Chapter 9. Enableable Component

1. Enableable コンポーネントが存在する理由

コンポーネントの追加・削除は **構造変更** にあたる。Entity は別のチャンクへ移り、クエリは解決し直しが必要になり、参照が無効になることもある。これを毎フレーム Entity ごとに行うと（「Stunned」の ON/OFF など）、そのコストがフレーム全体を食い潰してしまう。

Enableable コンポーネント なら、構造変更なしで振る舞いを ON/OFF できる。コンポーネント自体は常にチャンク上にあり、チャンクごとのビットマスクの 1 ビットが、クエリから見て各 Entity がそれを「持っているか」を決める。

2. インターフェース

IComponentData と IEnableableComponent の両方を実装する。

```
using Unity.Entities;

public struct Stunned : IComponentData, IEnableableComponent
{
    public float Duration;
}
```

IBufferElementData も Enableable にできる。

3. 切り替え

System から切り替えるには次のようにする。

```
// Turn Stunned off without removing the component
state.EntityManager.SetComponentEnabled<Stunned>(entity, false);

// From an ECB (deferred, safe mid-iteration)
ecb.SetComponentEnabled<Stunned>(entity, true);

// From a job with a ComponentLookup
var stunnedLookup = SystemAPI.GetComponentLookup<Stunned>();
stunnedLookup.SetComponentEnabled(entity, false);
```

現在の状態を読み取るには次のとおり。

```
bool isStunned = state.EntityManager.IsComponentEnabled<Stunned>(entity);
```

どの方法もビットマスクを操作するだけで、構造変更は起こさない。

4. クエリの動作

`SystemAPI.Query<...>` や `EntityQuery` ベースのイテレーションは、`Enableable` コンポーネントが無効になっている `Entity` をスキップする。デフォルトでは、指定したコンポーネントがすべて有効な `Entity` だけが対象になる。

```
foreach (var stunned in SystemAPI.Query<RefRW<Stunned>>())
{
    // only fires for entities where Stunned is enabled
}
```

無効な `Entity` も対象にしたいなら、この挙動を上書きする。

```
foreach (var (stunnedRW, entity) in SystemAPI
    .Query<RefRW<Stunned>>()
    .WithEntityAccess()
    .WithOptions(EntityQueryOptions.IgnoreComponentEnabledState))
{
    // fires for ALL entities with the component, enabled or not
}
```

5. 例 — 時間経過で解除されるスタン

1 つの `IJobEntity` が、同じコンポーネントについて `ref Stunned (RefRW<T>)` と `EnabledRefRW<Stunned>` の両方を受け取ることはできない。Entities は、1 つのジョブで同じコンポーネントを両方の形式でラップすることを明確に禁じている。使える 2 つのパターンを示す。

5.1 値を読み書きし、有効ビットを ECB 経由で切り替える

```
[BurstCompile]
public partial struct StunTickJob : IJobEntity
{
    public float DeltaTime;
    public EntityCommandBuffer.ParallelWriter ECB;

    void Execute([ChunkIndexInQuery] int chunkIndex,
        Entity entity,
        ref Stunned stunned)
    {
        stunned.Duration -= DeltaTime;
        if (stunned.Duration <= 0f)
    }
```

```
        ECB.SetComponentEnabled<Stunned>(chunkIndex, entity, false);
    }
}
```

プレイバックは次の ECB の境界（EndSimulationECBS など）で行われる。

14_EntityCommandBuffer ・ Deferred Entity.md を参照。構造変更はなく、決まったタイミングでビットマスクが反転するだけだ。

5.2 コンポーネントの値が不要な場合に有効ビットを直接切り替える

```
[BurstCompile]
public partial struct StunDisableJob : IJobEntity
{
    void Execute(EnabledRefRW<Stunned> stunnedEnabled)
    {
        stunnedEnabled.ValueRW = false;    // no structural change
    }
}
```

このジョブで Stunned を参照しているのは EnabledRefRW<T> だけなので、コンパイルが通る。System の役割が単なる切り替えだけのときは、この形を使う。

同じ Entity に対して、1 回の処理で値の読み取りと有効ビットの切り替えを両方やりたいなら、メインスレッドで EntityManager.SetComponentEnabled<T>(entity, false) を使うか、state.Dependency でつないだ 2 つのジョブに分ける。

6. Enableable vs 追加/削除の使い分け

状況	選択
高頻度の切り替え（フレームごと、イベントごと）	Enableable
常に存在するとは限らない状態を持たせる（フィールドを持つ Entity 別の修飾子など）	Enableable
低頻度の Archetype 変更（例：恒久的な能力の追加）	通常どおりコンポーネントを追加/削除する
多数のシステムで一律にフィルタリングする	Enableable（クエリが自動的に拾い上げる）

目安として、「このコンポーネントを持っているか」を表すビットが Entity の一生のうちに何度も反転するなら、Enableable にする。

7. 他の機能との相互作用

機能	動作
<code>WithChangeFilter<T></code>	有効ビットを切り替えると、コンポーネントの変更バージョンも更新される。
<code>ICleanupComponentData</code>	クリーンアップコンポーネントは、同時に <code>Enableable</code> にはできない。
Netcode for Entities	<code>Enableable</code> コンポーネントは <code>Ghost</code> 同期できるが、有効・無効の切り替えはオーソリティ側で行う必要がある。
シリアライゼーション / World エクスポート	有効ビットは保持される。

8. トラブルシューティング

症状	原因 / 対処法
クエリに「OFF」にすべき Entity が含まれる	コンポーネントが <code>IEnableableComponent</code> を実装していないか、 <code>WithOptions(EntityQueryOptions.IgnoreComponentEnabledState)</code> が指定されている。
<code>SetComponentEnabled<T></code> が「type T does not support being enableable」と例外を投げる	struct が <code>IComponentData</code> しか実装していない。 <code>IEnableableComponent</code> を追加する。
<code>EnabledRefRW<T></code> パラメーターがソースジェネレーターに拒否される	ジョブかシステムに <code>partial</code> が付いていないか、コンポーネントが <code>Enableable</code> でない。
ソースジェネレーターエラー「cannot have both RefRW and EnabledRefRW for the same component」	ロジックを分ける。ジョブごとに <code>ref T</code> か <code>EnabledRefRW<T></code> のどちらか一方を使い、同じコンポーネントで両方は使わない。§5 の 2 つのパターンを参照。
変更フィルターが誤って発火する	<code>Enabled</code> の切り替えは、 <code>WithChangeFilter</code> から見ると変更として数えられる。これは正常な動作だ。切り替えではなく <code>EnabledRefRO</code> での読み取りと組み合わせる。
ECB の <code>SetComponentEnabled</code> が無視される	遅延 Entity を生成した ECB がフラッシュされる前に、その Entity に対して呼び出している。どちらも同じ ECB サイクル内で行う必要がある。

Chapter 10. Singleton Component

1. Singleton とは何か、そして何でないか

Entities でいう **Singleton** とは、World の中に**ちょうど 1 つの Entity** だけ存在すると想定されるコンポーネントにすぎない。専用の **ISingleton** インターフェースがあるわけではない。慣例として、任意の **IDataComponent** が Singleton になり、「ちょうど 1 つ」というルールをチェックする使いやすいアクセサーを **SystemAPI** が用意している。

典型的な用途は、ゲーム状態、グローバル設定、時刻 / ティックの供給源、ネットワーク状態、ECB の集中登録など。

```
public struct GameState : IDataComponent
{
    public int    Score;
    public float  TimeScale;
    public bool   Paused;
}
```

2. アクセサー

ISystem でも **SystemBase** でも、次のように書ける。

```
// Reads
GameState gs      = SystemAPI.GetSingleton<GameState>();
RefRW<GameState> gsRW = SystemAPI.GetSingletonRW<GameState>();
bool has         = SystemAPI.HasSingleton<GameState>();
bool got         = SystemAPI.TryGetSingleton<GameState>(out var gs2);

// Writes
SystemAPI.SetSingleton(new GameState { Score = 10 });

// Entity handle, if you need it
Entity e         = SystemAPI.GetSingletonEntity<GameState>();

// Buffers work too
DynamicBuffer<Waypoint> buf = SystemAPI.GetSingletonBuffer<Waypoint>();
```

Get* 系は、Singleton が 0 個または 2 個以上のとき例外を投げる。**TryGet*** 系は、その場合に **false** を返す。RW 系はコンポーネントを変更済みとしてマークするため、変更追跡が働く。

3. Singleton の作成

よく使うパターンが2つある。

3.1 SubScene からベイクする

```
public class GameStateAuthoring : MonoBehaviour
{
    public float InitialTimeScale = 1f;

    class Baker : Baker<GameStateAuthoring>
    {
        public override void Bake(GameStateAuthoring authoring)
        {
            var e = GetEntity(TransformUsageFlags.None);
            AddComponent(e, new GameState
            {
                Score      = 0,
                TimeScale = authoring.InitialTimeScale,
                Paused     = false
            });
        }
    }
}
```

SubScene 内の GameObject 1 つに GameStateAuthoring をアタッチすると、Baker がちょうど1つの Entity を Singleton として生成する。

3.2 System 起動時に作成する

```
public void OnCreate(ref SystemState state)
{
    var e = state.EntityManager.CreateEntity(typeof(GameState));
    state.EntityManager.SetComponentData(e, new GameState { TimeScale = 1f });
}
```

オーサリングに置くべきでないデータ（サーバーから注入する設定、手続き的に生成するものなど）に使う。

4. Singleton の有無で System を制御する

Singleton が用意できて初めて意味を持つ System もある。その場合は次のようにする。

```
public void OnCreate(ref SystemState state)
{
    state.RequireForUpdate<GameState>();
}
```

`RequireForUpdate<T>()` を呼ぶと、コンポーネントが存在しない間は `OnUpdate` が丸ごとスキップされる。`OnUpdate` の中に防御的な `null` チェックを書かずに済む。

5. 読み取り専用 vs 読み書きのパフォーマンス

- `GetSingleton<T>()` — コピーを読み取る。変更追跡はしない。
- `GetSingletonRW<T>()` — チャンク内を指す `RefRW<T>` を返す。`.ValueRW` に代入するとコンポーネントが変更済みとしてマークされる。ほかの `System` が `WithChangeFilter<T>` を使う場合に必要になる。
- `SetSingleton` — まるごと書き込み、必ずバージョンを更新する。

読み取り中心の `System` では `GetSingleton` を使い、実際に書き換えるときだけ `GetSingletonRW` に切り替える。

6. 候補が複数 — 「Singleton が 2 つ以上」という落とし穴

2 つの `Entity` が同じ `Singleton` コンポーネントを持ってしまうと、`GetSingleton<T>` は `InvalidOperationException: expected exactly one entity with component T, found N` を投げる。よくある原因は次のとおり。

- `SubScene` 内の 2 つの `GameObject` のどちらにもオーサリングスクリプトが付いている。
- テストフィクスチャがランタイムで `Singleton` を作り、さらに `SubScene` もそれをベイクしている。
- `System` が `HasSingleton` でチェックせずに、`OnCreate` で `Singleton` を作っている。

安全側に倒した書き方は次のとおり。

```
public void OnCreate(ref SystemState state)
{
    if (!SystemAPI.HasSingleton<GameState>())
    {
        var e = state.EntityManager.CreateEntity(typeof(GameState));
        state.EntityManager.SetComponentData(e, new GameState { TimeScale = 1f });
    }
}
```

7. バッファ Singleton

DynamicBuffer<T> も Singleton にできる。

```
public struct QueuedEvent : IBufferElementData
{
    public int Type;
    public int Payload;
}

// Access:
DynamicBuffer<QueuedEvent> events = SystemAPI.GetSingletonBuffer<QueuedEvent>();
events.Add(new QueuedEvent { Type = 1 });
```

グローバルなイベントキューやコマンドストリームに便利だ。

8. トラブルシューティング

症状	原因 / 対処法
InvalidOperationException: expected exactly one entity with component T, found 0	Singleton Entity が作られていない。baker を追加するか、HasSingleton でチェックしたうえで OnCreate で作る。
... found 2	2 つの Entity がそのコンポーネントを持っている。 オーサリング側で重複をなくすか、起動時に整理する。
GetSingletonRW がコンポーネントを変更済みとしてマー クしていないようだ	.ValueRO に書き込んだか、RefRW を使っていない。変更追跡が働くのは .ValueRW への書き込みだけだ。
Singleton が作成される前に System が実行される	RequireForUpdate<T> を使うか、作成側が先に走るよう System を順序付ける ([UpdateBefore])。
順序付けなしで2つの System から Singleton が変更される	[UpdateAfter] / [UpdateBefore] を付けるか、 ロジックを一本化する。Singleton の競合は、 とりわけデバッグが難しい。
Netcode：クライアントとサーバーで Singleton の値が乖離する	Singleton を ghost 同期するか、 同期済みの入力から決定論的に組み立て直す。

Chapter 11. System — ISystem vs SystemBase

1. 概要

Entities には System を宣言する方法が 2 つある。**ISystem**（アンマネージドな struct）と **SystemBase**（マネージドな class）だ。SystemBase でなければならない明確な理由がなければ、ISystem を選ぶ。

	ISystem	SystemBase
宣言形式	partial struct : ISystem	partial class : SystemBase
メモリ	アンマネージド、スタックアロケート可	マネージド、GC 管理
Burst コンパイル可	可（[BurstCompile] を追加）	不可
マネージドフィールド	不可	可
マネージド API アクセス	EntityManager + ManagedAPI 経由	直接
デフォルトの選択	☒	必要な場合のみ

2. ISystem — デフォルト

```
using Unity.Burst;
using Unity.Entities;

[BurstCompile]
public partial struct EnemyAISystem : ISystem
{
    [BurstCompile]
    public void OnCreate(ref SystemState state)
    {
        // Called once when the system is created.
        state.RequireForUpdate<EnemySpawnConfig>();
    }

    [BurstCompile]
    public void OnUpdate(ref SystemState state)
    {
        var config = SystemAPI.GetSingleton<EnemySpawnConfig>();
        // ... logic
    }

    [BurstCompile]
    public void OnDestroy(ref SystemState state)
    {
    }
}
```

```
        // Called once when the system is destroyed.
    }
}
```

外せないポイントは次のとおり。

- `struct` は `partial` でなければならない。ソースジェネレーターが隠れたラッパーを追加するためだ。
- `OnCreate`、`OnUpdate`、`OnDestroy` の 3 つのメソッドはすべて任意で、不要なものは省ける。
- `struct` と、Burst コンパイルしたい各メソッドの両方に `[BurstCompile]` を付ける。

3. SystemBase — マネージド System が避けられない場合

```
using Unity.Entities;

public partial class PlayerInputSystem : SystemBase
{
    private InputActions _input;    // managed field – the reason SystemBase exists

    protected override void OnCreate()
    {
        _input = new InputActions();
        _input.Enable();
    }

    protected override void OnUpdate()
    {
        var move = _input.Gameplay.Move.ReadValue<Vector2>();
        // ... dispatch to components
    }

    protected override void OnDestroy() => _input.Dispose();
}
```

`SystemBase` の出番は次のような場面だ。

- `System` と寿命をともにするマネージドフィールドを持ちたいとき（Input System のオブジェクト、UI Toolkit のハンドル、マネージドなサービスなど）。
- `EntityManager` を介さず、`OnUpdate` の中で直接マネージド API を呼びたいとき。

`SystemBase` の処理はすべて、Burst なしでメインスレッド上を走る。

4. System のライフサイクル

どちらの形式も、フック名は共通だ。

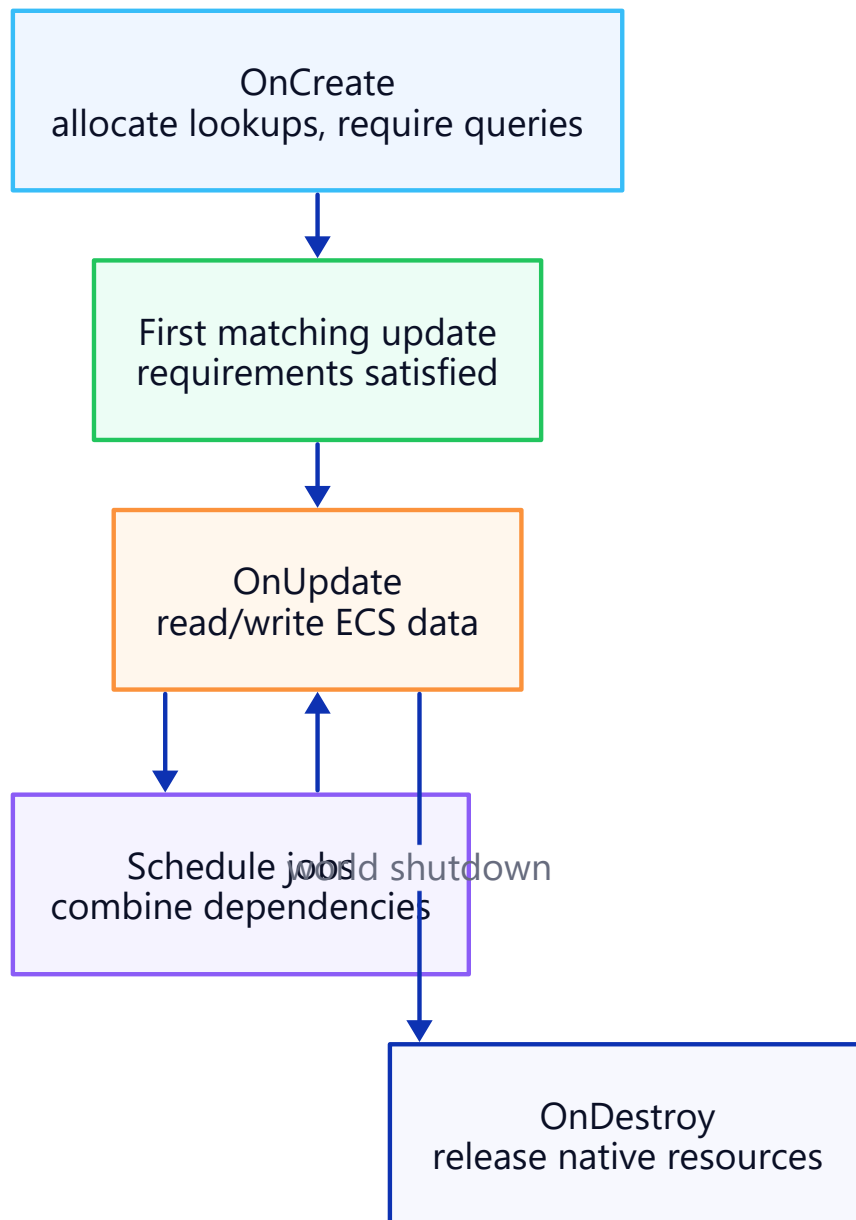


Figure: System ライフサイクル。

フック	発火タイミング	典型的な用途
OnCreate	System の作成時に 1 回	クエリのキャッシュ、RequireForUpdate<T>、ネイティブコンテナの確保
OnStartRunning	クエリが空 → 非空に変わったとき	セッションごとの状態のリセット
OnUpdate	親 SystemGroup のなかで毎フレーム	メインの処理
OnStopRunning	クエリが非空 → 空に変わったとき	バッファされた状態のフラッシュ
OnDestroy	World の破棄時に 1 回	ネイティブ/マネージドリソースの解放

RequireForUpdate<T>()

必要なコンポーネントや Singleton がない間は、OnUpdate をまるごとスキップする。

```
public void OnCreate(ref SystemState state)
{
    state.RequireForUpdate<EnemySpawnConfig>();
    state.RequireForUpdate<GameRunning>();
}
```

「設定がそろっている」「ゲームが始まっている」といった条件で System の実行を制御する、定番の手法だ。

5.2 つの使い分け

判断の目安は次のとおり。

1. **まず ISystem で書く。** Burst と相性がよく、GC も発生しない。
2. **マネージドフィールドがどうしても必要なときだけ SystemBase に切り替える。** その場合も分割を検討したい。マネージドな処理は、結果をコンポーネントへ書き出すだけの薄い SystemBase に閉じ込め、重いロジックはそのコンポーネントを読む ISystem で動かす。
3. ISystem の struct にマネージドフィールドを足して**混在させてはいけない**。コンパイルが通らない。上記の分割が定石の回避策だ。

分割の例を示す。

```
// Managed – bridges Input System into components.
public partial class InputBridgeSystem : SystemBase
{
    private InputActions _input;
    // ... fills a PlayerInput singleton each frame
}

// Unmanaged + Burst – reads the PlayerInput singleton and drives gameplay.
[BurstCompile]
public partial struct PlayerMoveSystem : ISystem
{
    [BurstCompile]
    public void OnUpdate(ref SystemState state) { /* ... */ }
}
```

6. SystemState クイックリファレンス

ISystem の中では、ref SystemState state が World への窓口になる。

メンバー	用途
<code>state.EntityManager</code>	メインスレッドでの構造変更。
<code>state.World</code>	この System が属する World。
<code>state.Dependency</code>	現在のジョブ依存ハンドル。 スケジュールしたジョブと組み合わせる。
<code>state.GetEntityQuery(...)</code>	キャッシュされたクエリの構築または取得。
<code>state.RequireForUpdate<T>()</code>	コンポーネントの有無で System の実行を制御する。
<code>state.Enabled</code>	<code>false</code> にすると、System を破棄せずに実行だけ止められる。

SystemBase では、これらはクラス自身のメンバーになる (`this.EntityManager` など)。

7. トラブルシューティング

症状	原因 / 対処法
<code>error CS0246: The type or namespace name 'Entities' could not be found</code>	<code>using Unity.Entities;</code> が抜けている。
OnUpdate が一度も発火しない	<code>RequireForUpdate<T></code> が、まだ存在しないコンポーネントを条件にしている。 そのコンポーネントがどこかで作られているかを確認する。
コンパイルエラー: 「partial struct must be declared partial」	ISystem では、ソースジェネレーターが struct を拡張できるように <code>partial</code> が必要だ。
Burst 警告: 「method is not burst compiled」	メソッドに <code>[BurstCompile]</code> が付いていない。struct に付けるだけでは足りない。
System が間違った World で実行される (Netcode のクライアント vs サーバー)	<code>[WorldSystemFilter(...)]</code> を付けて、 <code>ServerSimulation / ClientSimulation / LocalSimulation</code> のいずれかを指定する。
マネージドフィールドを ISystem に追加できない	正常な動作だ。上記のように、マネージドな部分を SystemBase へ切り出す。
System が Window → Entities → Systems に表示されない	<code>partial</code> が付いていない、デフォルト World の外にある、または <code>[DisableAutoCreation]</code> で除外されている。

Chapter 12. System Group & 更新順序

1. 概要

System はすべて、いずれかの **SystemGroup** に属する。グループは子の System が毎フレームいつ走るかを決め、順序付けの属性がグループ**内での**並び順を決める。グループ分けがうまくできれば、1フレーム遅れの書き込みや競合状態、「なぜこの Entity がガタつくのか」といったバグを避けられる。

2. 組み込みグループ

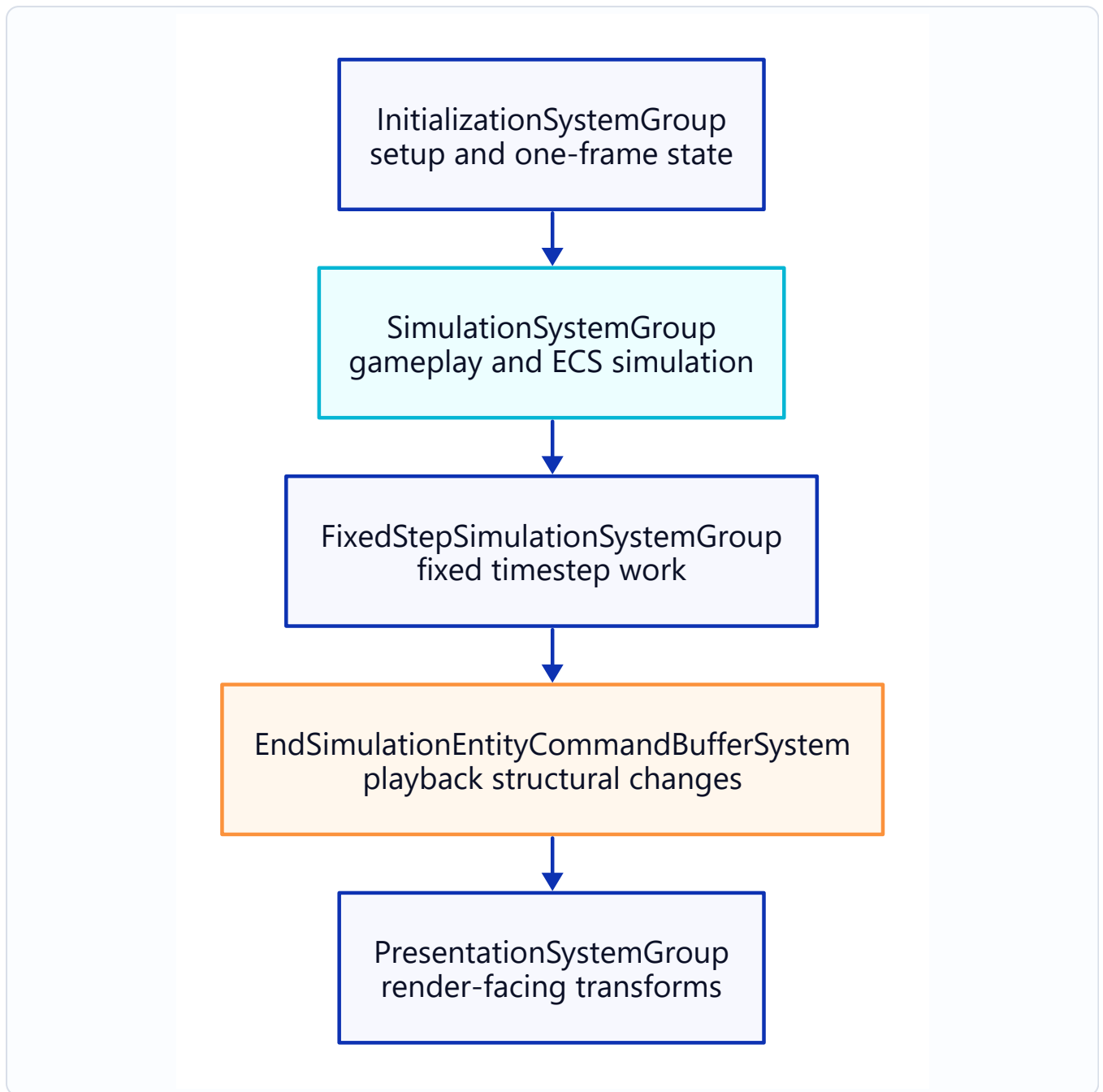


Figure: System Group と更新順序。

グループ	実行タイミング	典型的な内容
InitializationSystemGroup	毎フレーム 1 回、シミュレーションの前	一回限りの初期化处理、ブートストラップ処理
SimulationSystemGroup	毎フレーム 1 回	ほとんどのゲームプレイロジック
FixedStepSimulationSystemGroup	固定タイムステップ（デフォルト 60 Hz）。1 フレームに 0 回、1 回、または N 回実行される	物理演算、決定論的ステップロジック

グループ	実行タイミング	典型的な内容
PresentationSystemGroup	毎フレーム 1 回、シミュレーションの後	レンダリング準備、補間
BeginSimulationEntityCommandBufferSystem	Simulation の先頭	順序付きの ECB プレイバック (前段)
EndSimulationEntityCommandBufferSystem	Simulation の末尾	順序付きの ECB プレイバック (後段)

Netcode for Entities は、さらに多くのグループを追加する (**PredictedSimulationSystemGroup**、**GhostSendSystem** など)。いずれも上記のグループの中にネストされる。**16_Netcode Client-Server World & Bootstrap.md** と **19_Netcode Prediction & Rollback.md** を参照。

3. グループへの System の配置

```
using Unity.Entities;

[UpdateInGroup(typeof(SimulationSystemGroup))]
public partial struct EnemyAISystem : ISystem { /* ... */ }
```

[UpdateInGroup] を省略すると、System は既定で **SimulationSystemGroup** に入る。

ネスト

グループはほかのグループの中に入れられる。

```
[UpdateInGroup(typeof(SimulationSystemGroup), OrderFirst = true)]
public partial class GameplaySystemGroup : ComponentSystemGroup { }

[UpdateInGroup(typeof(GameplaySystemGroup))]
public partial struct MovementSystem : ISystem { /* ... */ }
```

カスタムグループの基底クラスが **ComponentSystemGroup** だ。System であると同時にコンテナとしても働く。

4. グループ内の順序付け

```
[UpdateInGroup(typeof(SimulationSystemGroup))]
[UpdateAfter(typeof(EnemyAISystem))]
[UpdateBefore(typeof(MovementSystem))]
public partial struct EnemyPathfindingSystem : ISystem { /* ... */ }
```


属性	意味
<code>[UpdateBefore(typeof(X))]</code>	X より前に実行する (両方が同じグループ内にある必要がある)。
<code>[UpdateAfter(typeof(X))]</code>	X より後に実行する。
<code>OrderFirst = true</code> (<code>[UpdateInGroup]</code> で)	グループ内でできるだけ早く実行する。
<code>OrderLast = true</code>	できるだけ遅く実行する。

ヒント：OrderFirst / OrderLast よりも `[UpdateAfter]` のチェーンを使うとよい。コードレビューで読みやすく、リファクタリングにも強い。

5. `[RequireMatchingQueriesForUpdate]`

Entities 1.x で追加された比較的新しい属性で、System の各クエリに最低 1 体の Entity がマッチしない限り、OnUpdate をスキップさせる。

```
[RequireMatchingQueriesForUpdate]
public partial struct EnemyAISystem : ISystem
{
    public void OnUpdate(ref SystemState state)
    {
        // Only runs when there is at least one entity matching every query the system uses.
    }
}
```

特定のコンポーネントや Singleton を要求する `state.RequireForUpdate<T>()` と比べると、`[RequireMatchingQueriesForUpdate]` はもっと暗黙的だ。System が実際に持つクエリ次第で条件が決まる。

6. カスタムグループ

カスタムグループを作る理由としては、次のようなものがある。

- まとまりのあるゲームプレイフェーズを表したい（「アニメーションの前にダメージを適用する」など）。
- 異なるティックレートで動かしたい（30 Hz の独自固定ステップなど）。
- 機能のオン・オフ。グループ全体をまとめて有効・無効にする。

```
using Unity.Entities;

[UpdateInGroup(typeof(SimulationSystemGroup))]
public partial class DamagePipelineGroup : ComponentSystemGroup
{
    // You can override OnUpdate to customise how the group ticks.
}

[UpdateInGroup(typeof(DamagePipelineGroup), OrderFirst = true)]
public partial struct ApplyDamageSystem : ISystem { /* ... */ }

[UpdateInGroup(typeof(DamagePipelineGroup))]
[UpdateAfter(typeof(ApplyDamageSystem))]
public partial struct DeathSystem : ISystem { /* ... */ }
```

カスタムティックレート

```
public partial class SlowTickGroup : ComponentSystemGroup
{
    private float _accum;
    protected override void OnUpdate()
    {
        _accum += (float)SystemAPI.Time.DeltaTime;
        if (_accum < 0.1f) return;    // 10 Hz
        _accum = 0f;
        base.OnUpdate();              // tick children
    }
}
```

7. 固定 vs 可変タイムステップ

決定論が重要なとき（物理演算、ネットワーク予測、決定論的な再生）は、System を `FixedStepSimulationSystemGroup` に置く。

```
[UpdateInGroup(typeof(FixedStepSimulationSystemGroup))]
public partial struct PhysicsStepSystem : ISystem { /* ... */ }
```

注意したい点は次のとおり。

- `FixedStepSimulationSystemGroup` は、実時間に追いつくために 1 フレームで**複数回**、あるいは 0 回実行されることがある。
- `SystemAPI.Time.DeltaTime` を使う。このグループ内では、これが固定ステップの dt になる。
- 入力を読む処理やプレゼンテーションを生成する処理は、`SimulationSystemGroup` や `PresentationSystemGroup` に置いておく。

8. ランタイムで順序を確認する

Window → Entities → Systems には次のものが表示される。

- すべてのグループとその子が、実際の実行順序で並ぶ。
- System ごとの処理時間。
- 有効・無効の状態（ランタイムで切り替える System の場合）。

属性で宣言した順序と実際の実行順序が食い違うときは、ほぼ間違いなく循環依存か [UpdateInGroup] の付け忘れが原因だ。

9. トラブルシューティング

症状	原因 / 対処法
System が間違ったフレームフェーズで実行される	[UpdateInGroup] がないか、指定が誤っている。既定は SimulationSystemGroup だ。
[UpdateAfter] が無視される	対象の System が別のグループにある。 両方が同じグループに属していなければならない。
起動時に循環依存エラーが発生する	[UpdateBefore] / [UpdateAfter] のチェーンが循環している。Systems ウィンドウにそのサイクルが出力される。
System が予期せず1フレームに複数回実行される	FixedStepSimulationSystemGroup に置いている。 このグループは実時間に追いつこうとする。その挙動が不要なら SimulationSystemGroup に移す。
カスタムグループが永遠に実行される	オーバーライドした OnUpdate で base.OnUpdate() を呼んでいない。
System は実行されるが何もしない	RequireForUpdate<T> の条件が満たされていないか、 すべてのクエリが空になっている。 Window → Entities → Query で確認する。

Chapter 13. JobSystem & Burst

1. 概要

DOTS のパフォーマンスは、Entities より前から存在し、今も土台となっている 2 つのエンジン機能に支えられている。

- **Job System** — 依存グラフを使い、ワークスレッドへ処理をスケジュールする。
- **Burst Compiler** — C# のジョブを、引き締まったネイティブコード（SIMD、GC なし、仮想ディスパッチなし）にコンパイルする。

どちらも Entities に固有の概念ではない。IJobEntity と IJobChunk は、IJob と Burst の上にかぶせた薄い層にすぎない。この土台を理解しておく、より上位のパターンがすっきり読めるようになる。

2. Job System — 中心となる型

インターフェース	形式	使いどき
IJob	void Execute()	一回限りのシングルスレッド処理。
IJobParallelFor	void Execute(int index)	0 から N まで並列ループ。
IJobParallelForBatch	void Execute(int start, int count)	バッチ単位で並列ループする (インデックス単位よりキャッシュ効率がよい)。
IJobChunk	void Execute(ArchetypeChunk chunk, int chunkIndex, bool useEnabledMask, in v128 enabledMask)	ECS チャンクを並列でイテレートする。
IJobEntity	void Execute([component params])	ECS の Entity を反復する。 ソースジェネレーターがチャンクジョブに変換する。

いずれも **struct** で、すべて Burst に対応する。

最小限の IJobParallelFor の例

```
using Unity.Burst;
using Unity.Collections;
```

```
using Unity.Jobs;

[BurstCompile]
struct ScaleJob : IJobParallelFor
{
    public NativeArray<float> Data;
    public float Factor;

    public void Execute(int index) => Data[index] *= Factor;
}

// Scheduling:
var handle = new ScaleJob { Data = arr, Factor = 2f }.Schedule(arr.Length, 64);
handle.Complete();
```

注記：

- フィールドは **blittable** でなければならない。
- 複数のジョブから同じ **NativeArray** へ書き込むには、依存関係のチェーンが必要だ（後述）。

3. スケジューリングと依存関係

```
JobHandle a = jobA.Schedule();
JobHandle b = jobB.Schedule(a); // b runs after a
JobHandle c = jobC.Schedule(JobHandle.CombineDependencies(a, b));
c.Complete();
```

ルール：

- **Schedule()** は必ず **JobHandle** を返す。新しいジョブが共有データを読み書きするなら、前のハンドルを依存関係として渡す。
- メインスレッドが結果を読む前に、最終ハンドルで **.Complete()** を必ず呼ぶ（Safety System がこれを強制する）。
- **ISystem** では **state.Dependency** がシステムレベルのハンドルになる。**.Schedule(state.Dependency)** を使うか、**IJobEntity.ScheduleParallel(...)** のようなソース生成のヘルパーを使えば、スケジュールしたジョブが自動で結合される。

ISystem の中で使う

```
[BurstCompile]
public void OnUpdate(ref SystemState state)
{
    var job = new MyJob { Dt = SystemAPI.Time.DeltaTime };
    state.Dependency = job.ScheduleParallel(state.Dependency);
}
```

依存関係を明示せずに `IJobEntity.ScheduleParallel()` を呼ぶと、`state.Dependency` と暗黙的に結合される。複数のジョブをチェーンするときは、明示的に書く。

4. Burst — 何であり、なぜ必要か

Burst は、C# のサブセット（`[BurstCompile]` を付けたメソッド、GC アロケーションなし、blittable なデータ）を、LLVM を通じて高度に最適化したネイティブコードへコンパイルする。

Burst がうまくコンパイルするもの

- `[BurstCompile]` を付けた struct の `IJob*` / `ISystem` メソッド。
- `Unity.Mathematics` の型（`float3`、`quaternion`、`int4` など）を使った計算。
- `NativeArray<T>`、`NativeList<T>`、`DynamicBuffer<T>`、`ArchetypeChunk` の列配列を回す密なループ。

Burst がコンパイルできないもの

- マネージド参照（`class`、`string`、`List<T>` など、アンマネージドでないもの）。
- 例外（`catch` はできるが、基本的には避けたい）。
- インターフェース経由の動的ディスパッチ（明示的なジェネリックの実体化なら問題ない）。
- `System.IO` や `Reflection` をはじめ、数学・プリミティブ以外のほとんどの BCL。

Burst の有効化

```
using Unity.Burst;

[BurstCompile]
public partial struct MyJob : IJobEntity
{
    // attribute on the struct marks the type for Burst
    // methods need [BurstCompile] too if you want each compiled
    [BurstCompile]
    void Execute(ref Velocity v) { /* ... */ }
}
```

`System` の場合は次のようになる。

```
[BurstCompile]
public partial struct MySystem : ISystem
{
    [BurstCompile] public void OnCreate(ref SystemState state) { }
    [BurstCompile] public void OnUpdate(ref SystemState state) { }
}
```

型に属性を付けるだけでは**足りない**。コンパイルさせたいメソッドにも、それぞれ [BurstCompile] が要る。

Burst Inspector

Window → Analysis → Burst Inspector は、メソッドごとに LLVM IR、アセンブリ、ベクトル化の状況を表示する。「なぜ速くならないのか」を調べたいときに使う。

5. ネイティブコンテナ

Burst と Safety System は、マネージドなコレクションではなく **ネイティブコンテナ** を前提とする。

マネージド	対応するネイティブ型
T[]	NativeArray<T>
List<T>	NativeList<T>
Dictionary<K,V>	NativeHashMap<K,V>、NativeParallelHashMap<K,V>
HashSet<T>	NativeHashSet<T>、NativeParallelHashSet<T>
Queue<T>	NativeQueue<T>
string	FixedString32Bytes / FixedString64Bytes / など

確保のしかたは次のとおり。

```
var arr = new NativeArray<int>(1024, Allocator.TempJob);  
// ...  
arr.Dispose();
```

ライフタイムが重要だ。Burst と Safety は、コンテナがきちんと Dispose されているか、またジョブの寿命が尽きた後にアクセスされていないかを検証する。

6. アロケーター

アロケーター	ライフタイム	典型的な用途
Allocator.Temp	現在のフレーム内のみ、 メインスレッド専用	OnUpdate 内で使う一時的なコンテナ
Allocator.TempJob	ジョブ 1 つの間、最大 4 フレームまで	ジョブに渡すコンテナ

アロケーター	ライフタイム	典型的な用途
<code>Allocator.Persistent</code>	手動の <code>Dispose()</code> が必要	フレームをまたいで持ち続ける、 <code>System</code> が所有するコンテナ
<code>state.WorldUpdateAllocator</code>	<code>System</code> グループの更新が終わるとクリアされる	<code>System</code> 内の一時データに最適

`System` の中では `state.WorldUpdateAllocator`（あるいはその `Rewindable` アロケーター）を使うとよい。手動での `Dispose()` 管理から解放される。

7. Safety System

「Jobs > Leak Detection」と「Jobs > Safety Checks」を ON にしたエディターでは、Unity が次の点を検証する。

- 2つのジョブが、依存関係なしに同じネイティブコンテナへ書き込んでいないか。
- メインスレッドの読み取りが、実行中の書き込みと競合していないか。
- ネイティブコンテナが、`Dispose()` 後やアロケーターの寿命切れ後にアクセスされていないか。

Safety エラーは、「job A and job B both access X」のような分かりやすいメッセージの例外として現れる。**開発中に Safety チェックを無効にしてはいけない。** 得られる性能はわずかで、デバッグコストのほうがはるかに高くつく。

8. トラブルシューティング

症状	原因 / 対処法
Burst 警告: 「method not burst compiled」	メソッドに <code>[BurstCompile]</code> が付いていないか、Burst が非対応の型を見つけた (Burst Inspector で失敗している行を確認する)。
<code>InvalidOperationException: ... writes to ... without declaring dependency</code>	依存関係が宣言されていない。前の <code>JobHandle</code> を渡すか、 <code>state.Dependency</code> でチェーンする。
<code>NativeArray</code> に <code>ObjectDisposedException</code>	ジョブが終わる前に <code>Dispose</code> した。 <code>arr.Dispose(handle)</code> を使うか、先にハンドルを待つ。
スケジュールされたジョブが完了しない	チェーンのどこかで <code>.Complete()</code> が抜けているか、循環依存がある。

症状	原因 / 対処法
[BurstCompile] を追加した後にパフォーマンスが低下する	ドメインリロードのたびにメソッドが再コンパイルされている。Burst キャッシュを確認する。あるいは Burst 非対応の型が紛れ込み、メソッドが黙って IL 実行に切り替わっている。
NativeHashMap の列挙が例外を投げる	列挙は並行書き込みと相性が悪い。代わりに NativeArray<K> にコピーしてから列挙する。
ジョブを待つメインスレッドがストールする	.Complete() を呼ぶのが早すぎる。完了させる位置を、フレーム内でできるだけ後ろに動かす。

Chapter 14. IJobEntity・SystemAPI.Query

1. 概要

Entities 6.5 では、コンポーネントの反復処理のおよそ 95% を 2 つの API でまかなえる。

API	実行場所	使いどき
SystemAPI.Query<...>()	メインスレッド、OnUpdate 内	処理が軽い（数百 Entity 程度）とき、 またはメインスレッド API が必要なとき。
IJobEntity	ワーカースレッド、Schedule / ScheduleParallel 経由	処理が重いとき、 または並列化の効果が見込めるとき。

これらは、旧来の `Entities.ForEach` に代わる推奨手段だ（`Entities.ForEach` は Entities 1.4 で非推奨となった。6.5 では非推奨の警告を出しつつまだコンパイルできるが、Unity の変更履歴では将来のメジャーリリースで削除予定とされている）。

2. SystemAPI.Query<T>

条件にマッチした Entity を、メインスレッドで反復処理する。コンポーネントへのアクセスには `RefRO<T>` / `RefRW<T>` を使う。

```
[BurstCompile]
public partial struct HealthRegenSystem : ISystem
{
    [BurstCompile]
    public void OnUpdate(ref SystemState state)
    {
        float dt = SystemAPI.Time.DeltaTime;

        foreach (var (health, regen) in
            SystemAPI.Query<RefRW<Health>, RefRO<Regen>>())
        {
            health.ValueRW.Value = math.min(
                health.ValueRO.Value + regen.ValueRO.Rate * dt,
                health.ValueRW.Max);
        }
    }
}
```

ルール：

- `System` の `struct` は `partial` でなければならない。
- 読み取り専用には `RefRO<T>`、読み書きには `RefRW<T>` を使う。`RefRO` 経由で書き込んではいけない。
- ID が必要なら、タプルに `Entity` を加える。`SystemAPI.Query<RefRW<Health>>().WithEntityAccess()` のようにする。

フィルター

```
foreach (var health in SystemAPI.Query<RefRW<Health>>())
    .WithAll<Enemy>()
    .WithNone<Dead>()
    .WithAny<Burning, Poisoned>())
{
    // only entities that are Enemy AND not Dead AND (Burning OR Poisoned)
}
```

- `WithAll<T>()` — 指定したコンポーネントをすべて持つこと。
- `WithNone<T>()` — 指定したものをいずれも持たないこと。
- `WithAny<T...>()` — 指定したもののうち、少なくとも 1 つを持つこと。
- `WithChangeFilter<T>()` — 前回の実行以降にコンポーネント `T` が変化したチャンクだけを対象にする。
- `WithEntityAccess()` — `Entity` の ID も併せて返す。

3. IJobEntity

ソースジェネレーターによる並列反復処理だ。アクセスするコンポーネントは `Execute` のシグネチャで宣言する。

```
using Unity.Burst;
using Unity.Entities;
using Unity.Mathematics;

[BurstCompile]
public partial struct HealthRegenJob : IJobEntity
{
    public float DeltaTime;

    void Execute(ref Health health, in Regen regen)
    {
        health.Value = math.min(health.Value + regen.Rate * DeltaTime, health.Max);
    }
}

[BurstCompile]
public partial struct HealthRegenSystem : ISystem
```

```
{
    [BurstCompile]
    public void OnUpdate(ref SystemState state)
    {
        new HealthRegenJob { DeltaTime = SystemAPI.Time.DeltaTime }
            .ScheduleParallel();
    }
}
```

シグネチャのルールは次のとおり。

- `void Execute(...)` — 各パラメーターがコンポーネントに 1 対 1 で対応する。
- `ref Component` は読み書き、`in Component` は読み取り専用（書き込み不可）。
- `Entity entity` をパラメーターに取れる。ソースジェネレーターが値を埋めてくれる。
- ジョブの `struct` は `partial` でなければならない。

スケジュールの種類

呼び出し	スレッディング
<code>.Run()</code>	メインスレッドで実行し、呼び出しから戻る前に完了する。 デバッグや軽い処理に向く。
<code>.Schedule()</code>	ワーカースレッド 1 本で実行する。
<code>.ScheduleParallel()</code>	チャンクをワーカースレッドに分散させる。

3 つのいずれも、`ISystem.OnUpdate` の中では `state.Dependency` を自動的に更新する。

ジョブでのフィルタリング

```
new HealthRegenJob { DeltaTime = dt }
    .ScheduleParallel(SystemAPI.QueryBuilder()
        .WithAll<Enemy, Regen>()
        .WithNone<Dead>()
        .Build());
```

あるいは、ジョブ `struct` 自体に `[WithAll]` / `[WithNone]` / `[WithAny]` 属性を付けてもよい。

ジョブでの Entity アクセス

```
void Execute(Entity entity, ref Health health) { /* ... */ }
```

チャンクインデックス / `[ChunkIndexInQuery]`

決定論的なプレイバックのために `ParallelWriter` の ECB と組み合わせるときに役立つ。チャンクインデックスを `sortKey` として渡す。15_ParallelWriter・Deterministic Playback.md を参照。

```
void Execute([ChunkIndexInQuery] int chunkIndex, in Health health, Entity entity)
{
    if (health.Value <= 0)
        ecb.DestroyEntity(chunkIndex, entity);
}
```

4. どちらを使うか

状況	選択
数十～数百 Entity、軽い計算	SystemAPI.Query
数千体以上の Entity で、Entity ごとに独立した処理	IJobEntity.ScheduleParallel
チャンク単位の処理（バウンディング、集計、空間分割など）	IJobChunk — 12_IJobEntity vs IJobChunk.md を参照
ループの中でマネージド API を使いたい	SystemBase からの SystemAPI.Query
構造変更を遅延させたい	どちらの形式でもよいが、書き込みは EntityCommandBuffer.ParallelWriter を介する。 14_EntityCommandBuffer・Deferred Entity.md を参照

5. 注意点

- ジョブ内で `this` やマネージドフィールドをキャプチャしてはいけない。やろうとすると、ソースジェネレーターがエラーを出す。
- `RefRW<T>.ValueRW` は変更追跡を発生させる。ValueRW 経由で読み取るだけで書き込まなくても、チャンクはダーティ扱いになる。
- `WithChangeFilter<T>` はチャンク単位で働く。チャンク内のどれか 1 体でも T に書き込めば、チャンク全体が変更済みと判定される。
- **Aspect は廃止された（ただしまだ削除はされていない）**。グループ化用の IAspect インターフェースは 1.4 で非推奨となったが、6.5 の API リファレンスでは今も有効な `Execute` のパラメーター種別として載っている。新しいコードでは素のコンポーネントパラメーターを使う。既存の Aspect は 6.5 でもコンパイルできる。

6. トラブルシューティング

症状	原因 / 対処法
コンパイルエラー： <code>IJobEntity must be partial</code>	ジョブ struct に <code>partial</code> を付ける。
<code>ScheduleParallel</code> で競合状態エラーが発生する	2つのジョブが、順序付けなしに同じコンポーネントへ書き込んでいる。 <code>state.Dependency</code> が正しく結合されているか確認するか、System 間に [UpdateAfter] を付ける。
ジョブが Burst で実行されない	ジョブ struct に [BurstCompile] が付いていないか、Burst 非対応の型 (マネージド参照や、FixedString でない素の string など) をキャプチャしている。
<code>SystemAPI.Query</code> が何も返さない	フィルターの組み合わせが空になっている。 Window → Entities → Query で、 クエリが実際に何にマッチするかを確認する。
RefRO 経由での書き込み (サイレントバグ)	今のコンパイラなら <code>refRO.ValueRW = ...</code> はエラーになる。 古いソースジェネレーターですり抜けてしまう場合は RefRW に切り替える。
<code>SystemAPI.Query</code> と手動で構築した <code>EntityQuery</code> で Entity 数が一致しない	フィルターの構成が違う。 <code>SystemAPI.Query</code> は既定で Enableable コンポーネントによる絞り込みを含むが、 <code>EntityQueryBuilder</code> では明示的な設定が必要だ。

Chapter 15. IJobEntity vs IJobChunk

1. 結論を先に

- **IJobEntity** は「各 Entity に同じ処理をする」場合。
- **IJobChunk** は「Entity ごとの単純な対応づけでは済まない、チャンク単位の処理」をする場合。

IJobEntity は IJobChunk の上に生成され、同じ反復処理の仕組みで動く。IJobChunk は、チャンク全体をそのままコードに見せてくれるだけだ。

2. IJobEntity — Entity 単位の書きやすさ

```
[BurstCompile]
public partial struct DamageTickJob : IJobEntity
{
    public float DeltaTime;

    void Execute(ref Health health, in Burn burn)
    {
        health.Value -= burn.DamagePerSecond * DeltaTime;
    }
}
```

長所は次のとおり。

- メソッド 1 つで、Entity 1 体ぶんのビューを扱える。書きやすく、読みやすい。
- ソースジェネレーターがお膳立てしてくれる。Entity、[ChunkIndexInQuery]、[EntityIndexInQuery] といったパラメーターは自動で埋まる。
- 絞り込みは、struct の [WithAll] / [WithNone] / [WithAny] 属性、または .Schedule* に渡す QueryBuilder で行える。

短所は次のとおり。

- Execute の中からは、ほかの Entity が見えない。状態を持たずに「このチャンクを合計する」といった処理は手軽にはできない。
 - Execute の中から、チャンク単位のメタデータ（変更バージョンや共有コンポーネントの値）にアクセスできない。
-

3. IJobChunk — 完全なチャンク制御

```
[BurstCompile]
public partial struct HealthAggregateJob : IJobChunk
{
    [ReadOnly] public ComponentTypeHandle<Health> HealthHandle;
    public NativeArray<float> PerChunkTotal; // size = chunk count

    public void Execute(in ArchetypeChunk chunk,
                        int unfilteredChunkIndex,
                        bool useEnabledMask,
                        in v128 chunkEnabledMask)
    {
        var healths = chunk.GetNativeArray(ref HealthHandle);
        float total = 0f;
        for (int i = 0; i < chunk.Count; i++)
            total += healths[i].Value;

        PerChunkTotal[unfilteredChunkIndex] = total;
    }
}
```

長所は次のとおり。

- チャンクの配列に直接アクセスできる。`chunk.GetNativeArray(ref handle)` や `chunk.GetBufferAccessor(ref bufferHandle)` を使う。
- 条件分岐に `chunk.Has<T>()` や `chunk.GetSharedComponent<T>(...)` が使える。
- チャンクごとの有効マスクや変更バージョンを参照できる。
- 集約処理、空間分割、ECB のバッチ記録にうってつけた。

代償は次のとおり。

- 定型コードが多い。System 側で `ComponentTypeHandle<T>` を宣言し、毎フレーム `handle.Update(ref state)` で更新する必要がある。
- 内側の `for` ループを自分で書くため、それだけミスが入る余地が増える。

System でハンドルを用意する

```
[BurstCompile]
public partial struct HealthAggregateSystem : ISystem
{
    private EntityQuery _query;
    private ComponentTypeHandle<Health> _healthHandle;

    public void OnCreate(ref SystemState state)
    {
        _query = state.GetEntityQuery(ComponentType.ReadOnly<Health>());
        _healthHandle = state.GetComponentTypeHandle<Health>(true);
    }
}
```



```

public void OnUpdate(ref SystemState state)
{
    _healthHandle.Update(ref state);

    var totals = CollectionHelper.CreateNativeArray<float>(
        _query.CalculateChunkCount(), state.WorldUpdateAllocator);

    state.Dependency = new HealthAggregateJob
    {
        HealthHandle    = _healthHandle,
        PerChunkTotal    = totals
    }.ScheduleParallel(_query, state.Dependency);
}
}

```

`handle.Update(ref state)` は、キャッシュした型ハンドルを毎フレーム更新する。これを忘れると、Burst が古いインデックスを参照してしまう。

4. 有効マスク（`useEnabledMask`）

`IEnableableComponent` を含むクエリでは、`Execute` に `useEnabledMask = true` と、有効ビットを表す `v128` マスクが渡される。このマスクに従わないと、無効な `Entity` まで処理してしまう。

ヘルパーは次のとおり。

```

var enumerator = new ChunkEntityEnumerator(useEnabledMask, chunkEnabledMask, chunk.Count);
while (enumerator.NextEntityIndex(out int i))
{
    // i is the index of the next enabled entity in the chunk
}

```

`IJobEntity` はこのマスクを自動で適用してくれる。チャンク単位での見通しが必要でなければ `IJobEntity` を選ぶ、もう一つの理由だ。

5. 判断ガイド

状況	選択
体力の更新、位置の移動、入力の反映 — <code>Entity</code> 単位の単純な計算	<code>IJobEntity</code>
チャンク内すべての <code>Entity</code> の値を合計する	<code>IJobChunk</code>
<code>Entity</code> 単位ではなく、チャンク単位で ECB コマンドを 1 つ記録する	<code>IJobChunk</code>
ループの中で共有コンポーネントの値を参照する	<code>IJobChunk</code>

状況	選択
チャンクの変更バージョンが必要	IJobChunk
各 Entity の DynamicBuffer<T> を反復処理する	どちらでもよいが、IJobEntity のほうがすっきり書ける
Entity 単位のロジックだが、その Entity のチャンクインデックスが必要	[ChunkIndexInQuery] int chunkIndex を使った IJobEntity
スケジューリングコストが重要で処理が軽い	ベンチマークを取って判断する。反復回数は Entity 数よりチャンク数のほうが少ない。 チャンクが数千ある状況では、たいてい IJobChunk が有利だ。

6. 相互運用 — 両方を組み合わせる

よくあるのは次のパターンだ。IJobChunk でチャンク単位のデータを集め、続いて IJobEntity で Entity 単位の書き込みを行い、両者を依存関係でつなぐ。

```
var gather = new HealthAggregateJob { ... }
    .ScheduleParallel(query, state.Dependency);

var act = new HealJob { PerChunkTotal = totals }
    .ScheduleParallel(gather);

state.Dependency = act;
```

7. トラブルシューティング

症状	原因 / 対処法
IJobChunk.Execute がゴミ値を読む	スケジュール前に handle.Update(ref state) を忘れたか、誤ったクエリを使っている。
ComponentTypeHandle<T> が見つからない	using Unity.Entities; が抜けているか、ハンドルを誤ったアクセスモードで宣言している。読み取り専用なら GetComponentTypeHandle<T>(true) を使う。
有効な Entity がスキップされる	マスクに従っていない。上で示したように ChunkEntityEnumerator を使う。IJobEntity ならこの問題は起きない。
ジョブが、2 つのチャンクから同じ NativeArray へ書き込む	チャンクごとにスロットを用意する（HealthAggregateJob のように）か、チャンクインデックスをキーにした NativeParallelHashMap を使う。

症状	原因 / 対処法
query 引数なしで IJobChunk をスケジュールしてしまう	ScheduleParallel(query, ...) はクエリの明示が必要だ。 IJobEntity.ScheduleParallel() は、 属性やソース生成からクエリを拾ってくれる。
v128 型が見つからない	using Unity.Burst.Intrinsics; が抜けている。

Chapter 16. 構造変更 & 安全性

1. 構造変更とは何か

構造変更とは、Entity をチャンク間で移動させたり、チャンクの構成を変えたりする可能性のある操作のことだ。次に挙げるものはすべて構造変更にあたる。

操作	構造変更にあたる理由
CreateEntity / DestroyEntity	チャンクスロットを確保または解放する。
AddComponent<T> / RemoveComponent<T>	Entity の Archetype が変わるため、物理的に別のチャンクへ移る。
SetSharedComponent<T>	共有コンポーネントの値が変わると別のチャンクになる（値ごとにグループ分けされるため）。
AddChunkComponent<T> / RemoveChunkComponent<T>	チャンク自体のメタデータを変更する。
SetName(entity, ...)	EntityManager 側の Entity メタデータを変更する。
MoveEntitiesFrom	World 間で Entity をコピーする。
Instantiate(prefab)	新しい Entity を確保する（Archetype をあらかじめ埋めることもある）。

SetComponentData や RefRW<T>.ValueRW でコンポーネントの値を書き込むのは、構造変更ではない。コンポーネントはすでにチャンク内にあり、バイト列を書き換えているだけだからだ。

2. 構造変更が危険な理由

Archetype が変わると、チャンク内の位置を頼りにした参照は、軒並み無効になる。

- 実行中のクエリテレーター（SystemAPI.Query<...>）が、Entity を飛ばしたり二重に処理したりしかねない。
- 影響を受けた Entity について、ComponentLookup<T> のエントリが無効になる。
- ジョブ内の ArchetypeChunk ハンドルが、移動前のチャンクを指したままになる。
- 並行して動いているほかのシステムが、矛盾した状態を目にする。

エディターでは、Safety System がこれらの大半を InvalidOperationException として検出する。ビルドでは、クラッシュや Entity の破損につながる。

3. 経験則

同じ **System** の更新の中で、クエリを反復処理しながら構造変更を行ってはいけない。

このルールを守る方法が 2 つある。おすすめ順に挙げる。

1. **EntityCommandBuffer** を使う。反復中に操作を記録しておき、後でまとめてプレイバックする。14_EntityCommandBuffer・Deferred Entity.md を参照。
2. **NativeList<Entity>** に集めてから、ループの後で変更する。小規模なら手軽で、ECB のオーバーヘッドもない。

パターン 2 の例を示す。

```
var toKill = new NativeList<Entity>(state.WorldUpdateAllocator);

foreach (var (health, e) in SystemAPI
    .Query<RefRO<Health>>()
    .WithEntityAccess())
{
    if (health.ValueRO.Value <= 0f)
        toKill.Add(e);
}

state.EntityManager.DestroyEntity(toKill.AsArray());
```

4. Enableable コンポーネント — 逃げ道

あるコンポーネントを毎フレーム追加・削除しているなら、それはまず間違いなく **Enableable コンポーネント** にすべきだ。有効ビットの切り替えは構造変更ではない。06_Enableable Component.md を参照。

5. ジョブと構造変更

構造変更は **メインスレッドで行う操作**であり、ジョブの中では実行できない。

ジョブから変更を加える定石は次のとおり。

アプローチ	使いどき
ジョブ内で <code>EntityCommandBuffer.ParallelWriter</code> に記録し、メインスレッドでプレイバックする	並列コードから行う、Entity 単位の構造変更。
<code>IJob</code> や <code>IJobChunk</code> で <code>EntityCommandBuffer</code> （シングルスレッド）に記録し、メインスレッドでプレイバックする	バッチ単位、チャンク単位の変更。
ジョブ内で <code>EnabledRefRW<T></code> を使い、 <code>Enableable</code> コンポーネントを切り替える	構造変更を伴わない、Entity 単位の「有効化・無効化」。

ECB は操作をコマンドとして記録しておく。プレイバックはメインスレッド上の決まったタイミング、ふつうは `EntityCommandBufferSystem` の境界で行われる。

6. 構造変更と読み取りの組み合わせ

2 つのシステムが続けて走り、システム A の構造変更によりシステム B が依存する場合は、実行順序を明示しておく。

```
[UpdateInGroup(typeof(SimulationSystemGroup))]
[UpdateBefore(typeof(MovementSystem))]
public partial struct SpawnSystem : ISystem { /* uses ECB */ }
```

ECB System のプレイバックが A と B の間に挟まれば（適切な `[UpdateAfter]` チェーンを持つ `EndSimulationECBS` など）、B が読み取る前に構造変更が反映される。

7. コストモデル

変更 1 回あたりのおおよその目安は次のとおり。

<code>SetComponentData</code> (value only)	~ 1x
<code>SetComponentEnabled</code> (enableable bit)	~ 1x (per-chunk bitmask write)
<code>AddComponent</code> (structural)	~ 10x (entity moves chunk)
<code>SetSharedComponent</code> (structural)	~ 10x
<code>DestroyEntity</code> (structural)	~ 10x
Per-frame add + remove pattern	pathological

あくまで桁感であり、正確な値は `Archetype` のサイズ、チャンクの占有率、アロケーターへの負荷で変わる。

8. 構造変更の多発を見つける

次のような症状が出る。

- Window → Entities → Systems に、処理内容のわりに不自然に重い System が見える。
- Profiler のマーカー `EntityManager.SetArchetype` や `Move Entity` がフレームの大半を占める。
- Archetype の数が時間とともに増え続ける。

対策は次のとおり。

- `AddComponent / RemoveComponent` の呼び出しを洗い出す。毎フレーム走っているものはないか。あれば `Enableable` コンポーネントに置き換える。
- 共有コンポーネントの値の集合を確認する。うっかり Entity 単位のキーになっているものはないか。
- Archetypes ウィンドウで、Entity が 1 体しかないチャンクを探す。これはコンポーネント構成を細かく分けすぎたことによる断片化だ。

9. トラブルシューティング

症状	原因 / 対処法
InvalidOperationException: EntityCommandBuffer has already been played back	プレイバック後に ECB へ記録している。ECB は使い切りだ。 毎フレーム、ECBS から新しいものを取得する。
foreach 中に InvalidOperationException: ... invalidated ...	ループ内で構造変更が起きている。ECB に移すか、 いったん集めてからまとめて適用する方式にする。
Entity が知らないうちに消える	<code>DestroyEntity</code> が呼ばれたのに、ほかのシステムが古い Entity ハンドルを持ち続けている。 <code>EntityManager.Exists</code> でハンドルを確認する。
追加・ 削除の繰り返しがフレーム時間を圧迫する	<code>Enableable</code> コンポーネントに置き換える。
<code>RemoveComponent</code> で「cleanup component present」エラーが発生する	Entity にクリーンアップコンポーネントが付いている。 先にクリーンアップを済ませる（05_Component Types.md を参照）。
共有コンポーネントの値の種類が膨れ上がる （異なる値が数百種類）	値が細かすぎる。区分にまとめる（N 単位に丸めるなど）か、 変化する部分は通常のコンポーネントとして持たせる。

Chapter 17. EntityCommandBuffer・遅延 Entity

1. ECB が存在する理由

EntityCommandBuffer (ECB) は、構造変更（作成・破棄・追加・削除・設定）を記録しておき、後でメインスレッド上の安全なタイミングで再生する仕組みだ。これにより、次のような場所から World を変更できる。

- クエリの foreach の中（EntityManager を直接呼ぶとイテレーターが無効になってしまう場所）。
- ジョブの中（メインスレッドでの構造変更が許されない場所）。
- 多数のチャンクにまたがる並列ジョブの中（ParallelWriter を介する）。

「なぜ構造変更が危険なのか」という背景は 13_Structural Change & Safety.md を参照。

2. ECB のライフサイクル



Figure: EntityCommandBuffer のライフサイクル。

ECB は **使い切り** で、プレイバックが終わると破棄される。毎フレーム、適切な EntityCommandBufferSystem に新しいものを要求する。

3. 組み込み ECB システム

システム	プレイバックタイミング	典型的な用途
BeginInitializationEntityCommandBufferSystem	Initialization グループの開始	前フレームから持ち越した World の早期セットアップ
EndInitializationEntityCommandBufferSystem	Initialization グループの終了	—

システム	プレイバックタイミング	典型的な用途
BeginSimulationEntityCommandBufferSystem	Simulation の開始	ゲームプレイが走る前に、保留中の変更を反映する
EndSimulationEntityCommandBufferSystem	Simulation の終了	ゲームプレイ起因のスポンや破壊の 第一候補
BeginPresentationEntityCommandBufferSystem	Presentation の開始	まれ。 表示まわりだけのセットアップ

どれを選ぶかで、**ほかのシステムが変更を目にするタイミング**が決まる。SimulationSystemGroup で弾をスポンし、**同じフレームのうちに動かしたいなら**、次フレームの BeginSimulation に記録する。次フレームでよければ EndSimulation に記録する。

4. ECB の取得

ISystem.OnUpdate の中なら、次のように書ける。

```
var ecb = SystemAPI
    .GetSingleton<EndSimulationEntityCommandBufferSystem.Singleton>()
    .CreateCommandBuffer(state.WorldUnmanaged);
```

ネストされた .Singleton 型は、組み込み ECB システムごとにソースジェネレーターが公開する、System 単位のトークンだ。CreateCommandBuffer は、そのシステムが動いたタイミングでプレイバックされる新しい ECB を返す。

並列ジョブの場合は次のようにする。

```
var ecb = SystemAPI
    .GetSingleton<EndSimulationEntityCommandBufferSystem.Singleton>()
    .CreateCommandBuffer(state.WorldUnmanaged)
    .AsParallelWriter();
```

並列プレイバックの取り決めは [15_ParallelWriter](#) ・ [Deterministic Playback.md](#) を参照。

5. 記録 — Entity の作成

```
Entity e = ecb.CreateEntity();
ecb.AddComponent(e, new Health { Value = 100 });
ecb.AddComponent(e, LocalTransform.FromPosition(0, 0, 0));
```

あるいは Prefab Entity からインスタンス化する。

```
Entity inst = ecb.Instantiate(prefab);
ecb.SetComponent(inst, LocalTransform.FromPosition(spawnPos));
```

CreateEntity も Instantiate も、**遅延 Entity** を返す (§6 を参照)。

6. 遅延 Entity

ecb.CreateEntity() や ecb.Instantiate() が返す Entity は、**まだ存在していない**。負のインデックスを持つプレースホルダーだ。できることは次のとおり。

- **同じ ECB** に対する後続の呼び出しに渡す。AddComponent、SetComponent、AppendToBuffer など。
- ecb.SetComponent(other, new ReferenceTo { Entity = inst }) のように、別のコンポーネントへ保存する。

できないことは次のとおり。

- ECB がプレイバックされるまで、EntityManager 経由で読み書きすること。
- 別の ECB へ受け渡すこと（プレースホルダーが解決されるのは、それを生成した ECB の中だけだ）。

プレイバック時に ECB システムが、プレースホルダーを使ったすべての箇所を本物の Entity ハンドルに置き換える。

7. 記録 — 構造変更

```
ecb.AddComponent(entity, new Stunned { Duration = 2f });
ecb.RemoveComponent<Stunned>(entity);
ecb.SetComponent(entity, new Health { Value = 50 });

ecb.SetComponentEnabled<Stunned>(entity, false);

var buf = ecb.AddBuffer<Waypoint>(entity);
buf.Add(new Waypoint { Position = new float3(1, 0, 0) });
ecb.AppendToBuffer(entity, new Waypoint { Position = new float3(2, 0, 0) });

ecb.DestroyEntity(entity);
```

これらはすべてキューに積まれたコマンドにすぎず、プレイバックまでは何も反映されない。

8. IJobEntity からの ECB

```
[BurstCompile]
public partial struct KillOnHealthZeroJob : IJobEntity
{
    public EntityCommandBuffer ECB;

    void Execute(Entity entity, in Health health)
    {
        if (health.Value <= 0)
            ECB.DestroyEntity(entity);
    }
}

// In OnUpdate:
var ecb = SystemAPI
    .GetSingleton<EndSimulationEntityCommandBufferSystem.Singleton>()
    .CreateCommandBuffer(state.WorldUnmanaged);

new KillOnHealthZeroJob { ECB = ecb }.Schedule();
```

`.ScheduleParallel()` を使う場合は `.AsParallelWriter()` に切り替え、`sortKey` を渡す。これは次章で解説する。

9. スタンドアロン（手動）の ECB

プレイバックを自分で制御したいこともある。たとえば 1 つの `OnUpdate` の中で完結させたい場合だ。

```
var ecb = new EntityCommandBuffer(Allocator.Temp);
// ... record ...
ecb.Playback(state.EntityManager);
ecb.Dispose();
```

自己完結した一回限りの変更にはこれを使う。システム境界をまたぐものやジョブで実行するものには、前述の組み込み ECBS を使う。

10. 決定論性

シングライターの ECB では、コマンドは**記録した順**でプレイバックされる。ParallelWriter では、プレイバック順序は渡した sortKey 次第だ。15_ParallelWriter・Deterministic Playback.md を参照。

Netcode やリプレイシステムでは、決定論的なプレイバックは欠かせない。必ず安定したソートキー（チャンクインデックスや、クエリ内の Entity インデックス）を渡し、実行順が不定なメインスレッドのコードパスで記録することは避ける。

11. トラブルシューティング

症状	原因 / 対処法
InvalidOperationException: ECB has already been played back	フレームをまたいで使い回している。OnUpdate ごとに CreateCommandBuffer で新しい ECB を取得する。
プレイバック後に遅延 Entity ハンドルが Entity.Null になる	2 つの異なる ECB にまたがって参照している。生成とその後の設定を、同じ ECB にまとめる。
スポンされた Entity が今フレームに表示されない	EndSimulation の ECBS に記録している。プレイバックはフレーム終了時だ。多くのシステムより前に反映させたいなら、次フレームの BeginSimulation に切り替える。
ecb.SetComponent で「component does not exist」が発生する	対象の Entity がまだそのコンポーネントを持っていない（先に AddComponent する）か、プレイバックがまだ行われていない。
ECBS が管理する ECB に Dispose() を呼んだ	CreateCommandBuffer で得た ECB に Dispose() を呼んではいけない。寿命は ECBS が管理する。Dispose するのはスタンドアロン（new EntityCommandBuffer(Allocator.Temp)）のものだけだ。
並列 ECB のプレイバックがコマンドを並べ替える	sortKey が決定論的でない。[ChunkIndexInQuery] か、クエリが提供する安定したインデックスを使う。

Chapter 18. ParallelWriter ・ 決定論的なプレイバック

1. なぜ専用の ParallelWriter があるのか

EntityCommandBuffer はシングルライターだ。2 つのスレッドが同じ ECB へ記録すると、内部チェーンで競合が起きてしまう。EntityCommandBuffer.ParallelWriter は、呼び出し元ごとに **スレッドローカルなチェーン** を割り当て、コマンドごとに **ソートキー** を要求することで、この問題を解決する。

プレイバック時、ECB システムは次の手順を踏む。

1. すべてのスレッドローカルチェーンを 1 本にまとめる。
2. コマンドを sortKey でソートする（安定ソート）。
3. メインスレッドで順に実行する。

ソートキーさえ決定論的なら、結果も決定論的になる。

2. ParallelWriter の取得

```
var ecb = SystemAPI
    .GetSingleton<EndSimulationEntityCommandBufferSystem.Singleton>()
    .CreateCommandBuffer(state.WorldUnmanaged)
    .AsParallelWriter();
```

AsParallelWriter() は struct EntityCommandBuffer.ParallelWriter を返す。これは IJobEntity、IJobChunk、IJobParallelFor にコピーして渡せる。

3. sortKey の契約

並列 ECB の呼び出しは、いずれも第 1 引数に sortKey を取る。

```
ecb.Instantiate(sortKey, prefab);
ecb.SetComponent(sortKey, entity, value);
ecb.DestroyEntity(sortKey, entity);
ecb.AddComponent(sortKey, entity, component);
```

良いソートキーの条件は次のとおり。

- **決定論的であること。** スケジューラーが処理をどう分配しても、同じ入力からは同じキーが得られる。
- **十分に一意であること。** 同じ Entity に対する 2 つのコマンドは、プレイバック順序を曖昧にしてよい場合を除き、衝突させない。
- **安価であること。** すでに手元にある `int` を使う。ハッシュは使わない。

実際には、定番の選択肢は次のとおりだ。

ソース	取得方法	使いどき
IJobEntity の [ChunkIndexInQuery]	ソースジェネレーターが埋める	ほとんどのケース
IJobChunk.Execute の unfilteredChunkIndex	API が提供する	IJobChunk ジョブ
IJobEntity の [EntityIndexInQuery]	ソースジェネレーター	Entity ごとのコマンドを個別に順序付けたいとき
手動で計算した決定論的インデックス	自分で計算する	まれ。独自のジョブ構成のとき

4. 完全な例 — 並列スポーン

```
using Unity.Burst;
using Unity.Entities;
using Unity.Mathematics;
using Unity.Transforms;

[BurstCompile]
public partial struct SpawnParallelJob : IJobEntity
{
    public float DeltaTime;
    public EntityCommandBuffer.ParallelWriter ECB;

    void Execute([ChunkIndexInQuery] int chunkIndex,
                Entity spawnerEntity,
                ref Spawner spawner)
    {
        if (spawner.RemainingCount <= 0)
        {
            ECB.DestroyEntity(chunkIndex, spawnerEntity);
            return;
        }

        spawner.TimeLeft -= DeltaTime;
        if (spawner.TimeLeft > 0f) return;

        spawner.TimeLeft = spawner.Interval;
        spawner.RemainingCount--;
    }
}
```

```

        var inst = ECB.Instantiate(chunkIndex, spawner.Prefab);
        ECB.SetComponent(chunkIndex, inst, LocalTransform.FromPosition(spawner.SpawnPoint));
    }
}

[BurstCompile]
public partial struct SpawnerParallelSystem : ISystem
{
    [BurstCompile]
    public void OnUpdate(ref SystemState state)
    {
        var ecb = SystemAPI
            .GetSingleton<EndSimulationEntityCommandBufferSystem.Singleton>()
            .CreateCommandBuffer(state.WorldUnmanaged)
            .AsParallelWriter();

        new SpawnParallelJob
        {
            DeltaTime = SystemAPI.Time.DeltaTime,
            ECB = ecb
        }.ScheduleParallel();
    }
}

```

`Instantiate` とその後の `SetComponent` に同じ `chunkIndex` を渡すと、2 つのコマンドが同じ位置にソートされる。これにより、`Instantiate` が返した遅延 `Entity` 参照が、正しい対象に解決される。

5. 決定論性 — 保証されるもの、されないもの

保証されること。 ECB の内容が同じなら、どのワーカーレッドがどのコマンドを記録したかに関係なく、プレイバックは同一の結果状態を生む。

自動では**保証されない**ことは次のとおり。

- **Entity の ID。** 前段のコマンドが違えば、2 回の実行で `Index` 値の異なる `Entity` が生成されることがある。
- **システム間でのチャンク順序。** システム A が並列で `Entity` を生成し、システム B が `ChunkIndexInQuery` 順でそれを読む場合、反復順序は `Archetype` の生成履歴に左右されることがある。

実行をまたいでビット単位の決定論性が必要ななら（リプレイや `Netcode` のロールバックなど）、次を組み合わせる。

1. 決定論的なソートキーを使った並列 ECB。
2. `FixedStepSimulationSystemGroup` 内で動かす固定ステップのシステム。
3. `Entity.Index` に依存しないこと。キーには、自分で管理するゲーム側の ID を使う。

6. ソートキーのパターン

6.1 チャンク単位（既定）

[ChunkIndexInQuery] int chunkIndex を使う。チャンク内のすべての Entity が同一のコマンドを生成する場合（チャンク全体をスタン状態にするなど）は、キーが衝突しても問題ない。

6.2 Entity 単位

Entity 単位で厳密に順序付けたいときは [EntityIndexInQuery] int entityIndex を使う（Entity 順でプレイバックしなければならない、Entity ごとのダメージイベントを記録する場合など）。

6.3 独自の決定論的インデックス

ごくまれに、入力からフラットなインデックスを自分で組み立てることもある（int sortKey = baseOffset + localIndex; など）。前の 2 つで足りないときだけにする。独自のインデックスは、決定論性のバグを生みやすい。

7. 避けるべきパターン

- sortKey に JobsUtility.ThreadIndex を使う。決定論的でない。
- キーの生成に Random を使う。決定論的でない。
- ECB を混在させる。2 つの異なる ECB に記録すると、全体の順序が失われる。
- 複数のシステムから、System 単位の 1 つの ECB に記録する。1 つの ECB につき記録者は 1 つにする（あるいは、1 つの ECB をラップした ParallelWriter を 1 つ使う）。

8. トラブルシューティング

症状	原因 / 対処法
遅延 Entity が間違ったターゲットに解決される	Instantiate とその後の SetComponent で別々のソートキーを使っている。同じキーにする。
コマンドが実行ごとに異なる順序で発火する	ソートキーが決定論的でない。[ChunkIndexInQuery] に切り替える。
「JobEntity must have partial」コンパイルエラー	ジョブ struct に partial を付ける。

症状	原因 / 対処法
並列ジョブ内の <code>ECB.SetComponent</code> で「wrong thread」が発生する	並列ジョブの中で、非並列の <code>EntityCommandBuffer</code> を使っている。先に <code>.AsParallelWriter()</code> を呼び、並列 API を使う。
Netcode のリプレイがデシンクする	どこかの並列 ECB が決定論的なソートキーを使っていないか、 <code>IJobEntity</code> が非決定論的な値をキャプチャしている。 チェーン全体を見直す。
Dispose 済みの ECB に <code>AsParallelWriter()</code> を呼ぶ	ECB はすでにプレイバック済みだ。 このフレームで新しいものを取得する。

Chapter 19. Netcode クライアント・サーバー World & Bootstrap

Unity 6000.5 · Entities 6.5.0 · Netcode for Entities 6.5.0

1. 概要

Netcode for Entities は **クライアント・サーバーモデル** を採用し、クライアントとサーバーのロジックを別々の ECS World に分ける。

World	役割
Server World	権威を持つシミュレーション。
Client World	ローカルプレイヤーのシミュレーション、予測、補間、プレゼンテーション。
Thin Client World	開発や負荷テストに使うダミーのクライアント。

クライアント端末がサーバーも兼ねる場合、そのプロジェクトは **クライアントホスト型サーバー** 構成で動いていることになる。

2. World の種類とシステムフィルタリング

2.1 WorldSystemFilter

System がどの Netcode World で実行できるかを宣言するには、WorldSystemFilter を使う。

```
using Unity.Entities;

[WorldSystemFilter(WorldSystemFilterFlags.ServerSimulation)]
public partial struct ServerOnlySystem : ISystem
{
    public void OnUpdate(ref SystemState state)
    {
        // Runs only in the server world.
    }
}
```

フラグ	意味
LocalSimulation	Netcode システムを持たないローカル World。
ServerSimulation	サーバーシミュレーション World。
ClientSimulation	クライアントシミュレーション World。
ThinClientSimulation	Thin Client シミュレーション World。

2.2 System グループフィルタリング

一部の Netcode グループは特定の World にしか存在しない。そのため、System をそのグループに置くこと自体が、World の絞り込みにもなる。

```
using Unity.Entities;
using Unity.NetCode;

[UpdateInGroup(typeof(GhostInputSystemGroup))]
public partial struct MyInputSystem : ISystem
{
    // GhostInputSystemGroup exists in Client and Local worlds.
    // The system is excluded from Server worlds.
}
```

System グループ	存在する World
GhostInputSystemGroup	Client、Local
PredictedSimulationSystemGroup	Client、Server
PresentationSystemGroup	Client のみ。Server や Thin Client には存在しない

3. Bootstrap プロセス

ClientServerBootstrap は、実行時に Server World と Client World を生成する。

3.1 既定の動作

既定の Bootstrap は、起動時に Server World と Client World の両方を生成する。ただし自動では接続しないので、いつリッスンし、いつ接続するかはゲーム側で制御する。

3.2 カスタム Bootstrap

```
using Unity.NetCode;

public class GameBootstrap : ClientServerBootstrap
{

```

```

public override bool Initialize(string defaultWorldName)
{
    // Create only the default local world.
    CreateLocalWorld(defaultWorldName);
    return true;
}
}

```

あとは、プレイボタン、ロビー画面、テストハーネス、コマンドラインなどから **World** を生成すればよい。

```

using Unity.NetCode;

void OnPlayButtonClicked()
{
    var serverWorld = ClientServerBootstrap.CreateServerWorld("ServerWorld");
    var clientWorld = ClientServerBootstrap.CreateClientWorld("ClientWorld");

    AutomaticThinClientWorldsUtility.NumThinClientsRequested = 4;
    AutomaticThinClientWorldsUtility.BootstrapThinClientWorlds();
}

```

AutomaticThinClientWorldsUtility は 1.5.0 ラインで追加されたもので、ランタイムに **Thin Client World** を生成・整理するために使う。

4. ティックレート設定

サーバーのシミュレーション速度は、**ClientServerTickRate Singleton** で設定する。

プロパティ	既定値	意味
SimulationTickRate	60	サーバーシミュレーションの 1 秒あたりのティック数。
NetworkTickRate	SimulationTickRate と同じ	スナップショットの送信レート (1 秒あたりのティック数)。
MaxSimulationStepsPerFrame	4	1 フレームあたりのシミュレーションステップ数の上限。
MaxSimulationStepBatchSize	4	スケールした deltaTime でまとめて処理できるティック数。
TargetFrameRateMode	BusyWait	BusyWait、Sleep、または Auto。

```

using Unity.Entities;
using Unity.NetCode;
using UnityEngine;

public class TickRateAuthoring : MonoBehaviour

```

```

{
    public int SimulationTickRate = 60;
    public int NetworkTickRate = 30;

    class Baker : Baker<TickRateAuthoring>
    {
        public override void Bake(TickRateAuthoring authoring)
        {
            var entity = GetEntity(TransformUsageFlags.None);
            AddComponent(entity, new ClientServerTickRate
            {
                SimulationTickRate = authoring.SimulationTickRate,
                NetworkTickRate = authoring.NetworkTickRate,
            });
        }
    }
}

```

`NetworkTickRate` を `SimulationTickRate` より低くすると、帯域を節約できる。この設定では、`GhostSendSystem` の負荷がラウンドロビン方式で複数のシミュレーションティックに分散される。

5. サーバーとクライアントの更新フロー

```

Server World
├─ Fixed timestep, 60 Hz default
│   └─ SimulationSystemGroup
│       ├── GhostSendSystem
│       └─ CommandReceiveSystem

Client World
├─ Dynamic timestep
│   └─ SimulationSystemGroup
│       ├── GhostReceiveSystem
│       └─ CommandSendSystem
├─ Fixed timestep, same as the server
│   └─ PredictedSimulationSystemGroup
│       └─ Prediction code and rollback loop
└─ PresentationSystemGroup
    └─ Rendering and interpolation

```

サーバーは、決定論的で権威を持つシミュレーションのために固定タイムステップを使う。クライアントは通常のシミュレーションと表示には可変タイムステップを使い、予測コードは固定ステップの予測ループで動かす。

6. World マイグレーション

World を切り替えるときに接続状態を引き継ぎたい場合は、`DriverMigrationSystem` を使う。

```
using Unity.Entities;
using Unity.NetCode;

public World MigrateServerWorld(World sourceWorld)
{
    DriverMigrationSystem migrationSystem = null;
    foreach (var world in World.All)
    {
        if ((migrationSystem = world.GetExistingSystemManaged<DriverMigrationSystem>()) != null)
            break;
    }

    var ticket = migrationSystem.StoreWorld(sourceWorld);
    sourceWorld.Dispose();

    var newWorld = ClientServerBootstrap.CreateServerWorld("NewServerWorld");
    var newMigrationSystem = newWorld.GetExistingSystemManaged<DriverMigrationSystem>();
    newMigrationSystem.LoadWorld(ticket);
    return newWorld;
}
```

7. 関連ドキュメント

- 03_ECS Core Concepts.md — World、EntityManager、Archetype、チャンクの基礎。
- 09_System Group & Update Order.md — Netcode が拡張する、基本となる ECS System グループ。
- 17_Netcode Network Connection & Approval.md — リッスン、接続、承認、ゲーム内状態への移行。

8. トラブルシューティング

症状	原因 / 対処法
サーバーで動かすべきでない System が動いてしまう	WorldSystemFilter を付けるか、適切な Netcode System グループへ移す。
ティックレートの設定が反映されない	ClientServerTickRate が、ロード済みの SubScene にバイクされているか確認する。
Thin Client が生成されない	NumThinClientsRequested を設定した後に AutomaticThinClientWorldsUtility.BootstrapThinClientWorlds() を呼んでいるか確認する。
クライアントとサーバーが勝手に接続してしまう	Bootstrap で AutoConnectPort が設定されていないか確認する。

症状	原因 / 対処法
Play モードで表示まわりのコード がエラーを出す	レンダリングシステムがサーバー World で動いている。ClientSimulation フィルターを付けるか、PresentationSystemGroup へ移す。

Chapter 20. Netcode ネットワーク接続 & 承認

Unity 6000.5 · Entities 6.5.0 · Netcode for Entities 6.5.0

1. 概要

Netcode for Entities は、すべてのネットワーク接続を **Entity** として表す。接続 Entity は `NetworkStreamConnection` を持ち、接続が切れるとその Entity は破棄される。

両側が `NetworkStreamInGame` を追加するまで、接続は「ゲーム内」とは見なされない。このコンポーネントがない間は、サーバーはスナップショットを送らず、クライアントもコマンドを送らない。

2. 接続まわりの主要コンポーネント

コンポーネント	意味
<code>NetworkStreamConnection</code>	トランスポートのハンドルを保持する。
<code>NetworkId</code>	承認後にサーバーが割り当てる一意の ID。
<code>NetworkStreamInGame</code>	スナップショットとコマンドデータをやり取りするのに必要。
<code>CommandTarget</code>	プレイヤーコマンドを受け取る Entity。AutoCommandTarget を使わない限り、ゲームコードが管理する。
<code>NetworkStreamRequestDisconnect</code>	切断を要求する。ドライバーの切断を直接呼んではいけない。
<code>NetworkStreamIsReconnected</code>	1.6.1 ラインで追加された、再接続を追跡するためのコンポーネント。

3. 3 通りの接続方法

3.1 NetworkStreamDriver による手動接続

```
using Unity.Entities;
using Unity.Networking.Transport;
using Unity.NetCode;

// Server.
var serverDriver = SystemAPI.GetSingletonRW<NetworkStreamDriver>();
```



```
serverDriver.ValueRW.Listen(NetworkEndpoint.AnyIpv4.WithPort(7979));

// Client.
var clientDriver = SystemAPI.GetSingletonRW<NetworkStreamDriver>();
clientDriver.ValueRW.Connect(
    state.EntityManager,
    NetworkEndpoint.Parse("127.0.0.1", 7979));
```

3.2 Bootstrap の AutoConnectPort

```
using Unity.NetCode;

public class AutoConnectBootstrap : ClientServerBootstrap
{
    public override bool Initialize(string defaultWorldName)
    {
        AutoConnectPort = 7979;
        CreateDefaultClientServerWorlds();
        return true;
    }
}
```

サーバーはワイルドカードアドレスでリッスンし、クライアントはループバックアドレスへ接続する。

3.3 リクエスト用コンポーネント

```
using Unity.Entities;
using Unity.NetCode;

var listenReq = serverWorld.EntityManager.CreateEntity(
    typeof(NetworkStreamRequestListen));
serverWorld.EntityManager.SetComponentData(listenReq,
    new NetworkStreamRequestListen { Endpoint = serverEndpoint });

var connectReq = clientWorld.EntityManager.CreateEntity(
    typeof(NetworkStreamRequestConnect));
clientWorld.EntityManager.SetComponentData(connectReq,
    new NetworkStreamRequestConnect { Endpoint = serverEndpoint });
```

4. 接続ステートマシン

4.1 クライアントの状態

Connecting → Handshake → [Approval] → Connected → Disconnected

状態	意味
Connecting	Connect の直後に一度だけ発生する。
Handshake	トランスポート接続は確立済みで、プロトコルのハンドシェイク待ち。
Approval	接続承認が有効なときだけ現れる。
Connected	サーバーが NetworkId を割り当てた状態。
Disconnected	切断またはタイムアウト。DisconnectReason が設定される。

4.2 サーバーの状態

Handshake → [Approval] → Connected → Disconnected

状態	意味
Handshake	クライアントを受け入れ、NetworkProtocolVersion をやり取りしている。
Approval	接続承認が有効なときだけ現れる。
Connected	接続が承認され、NetworkId が割り当てられた状態。
Disconnected	クライアントが切断した。

4.3 接続イベントを受け取る

```
using Unity.Burst;
using Unity.Entities;
using Unity.NetCode;
using UnityEngine;

[UpdateAfter(typeof(NetworkReceiveSystemGroup))]
[BurstCompile]
public partial struct ConnectionEventListener : ISystem
{
    [BurstCompile]
    public void OnUpdate(ref SystemState state)
    {
        var events = SystemAPI.GetSingleton<NetworkStreamDriver>()
            .ConnectionEventsForTick;

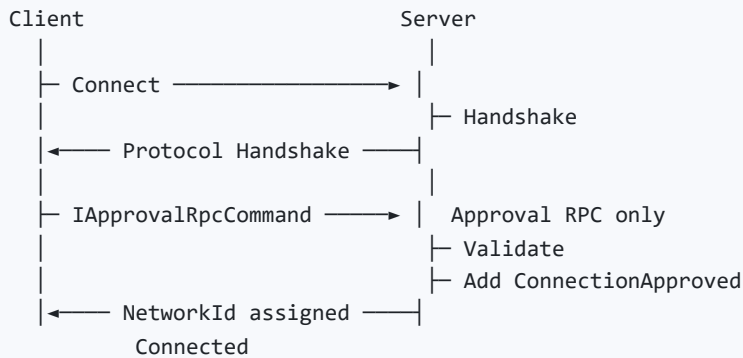
        foreach (var evt in events)
        {
            Debug.Log($"[{state.WorldUnmanaged.Name}] {evt.ToFixedString()}!");
        }
    }
}
```

ConnectionEventsForTick が有効なのは SimulationSystemGroup の中だけだ。ほかの場所で読むと、イベントを取りこぼしたり重複して受け取ったりする。

5. Connection approval

Connection approval を使うと、サーバーは **NetworkId** を割り当てる前に、ホワイトリストやパスワード、トークン、マッチメイキングチケットといった条件で接続を検証できる。

5.1 Approval flow



5.2 Enable approval

```
using Unity.Entities;
using Unity.NetCode;

using var drvQuery = serverWorld.EntityManager
    .CreateEntityQuery(ComponentType.ReadWrite<NetworkStreamDriver>());

drvQuery.GetSingletonRW<NetworkStreamDriver>().ValueRW
    .RequireConnectionApproval = true;
```

クライアント側にも、同じ承認要件を設定する。

5.3 Approval RPC

```
using Unity.Collections;
using Unity.Entities;
using Unity.NetCode;

public struct ApprovalRpc : IApprovalRpcCommand
{
    public FixedString512Bytes Token;
}

[WorldSystemFilter(WorldSystemFilterFlags.ClientSimulation |
    WorldSystemFilterFlags.ThinClientSimulation)]
public partial struct ClientApprovalSystem : ISystem
{
    public void OnUpdate(ref SystemState state)
    {
        var ecb = new EntityCommandBuffer(Allocator.Temp);

        foreach (var (connection, entity) in
            SystemAPI.Query<RefRW<NetworkStreamConnection>>())
```

```

        .WithNone<NetworkId>()
        .WithNone<ApprovalSent>()
        .WithEntityAccess())
    {
        var rpcEntity = ecb.CreateEntity();
        ecb.AddComponent(rpcEntity, new ApprovalRpc { Token = "MY_TOKEN" });
        ecb.AddComponent<SendRpcCommandRequest>(rpcEntity);
        ecb.AddComponent<ApprovalSent>(entity);
    }

    ecb.Playback(state.EntityManager);
}

[WorldSystemFilter(WorldSystemFilterFlags.ServerSimulation)]
public partial struct ServerApprovalSystem : ISystem
{
    public void OnUpdate(ref SystemState state)
    {
        var ecb = new EntityCommandBuffer(Allocator.Temp);

        foreach (var (req, rpc, entity) in
            SystemAPI.Query<RefRO<ReceiveRpcCommandRequest>, RefRO<ApprovalRpc>>()
                .WithEntityAccess())
        {
            var connEntity = req.ValueRO.SourceConnection;

            if (rpc.ValueRO.Token.Equals("MY_TOKEN"))
                ecb.AddComponent<ConnectionApproved>(connEntity);
            else
                ecb.AddComponent<NetworkStreamRequestDisconnect>(connEntity);

            ecb.DestroyEntity(entity);
        }

        ecb.Playback(state.EntityManager);
    }
}

```

Approval のタイムアウトは `ClientServerTickRate.HandshakeApprovalTimeoutMS` で制御する。
既定値は 5000 ms だ。

6. Entering in-game state

承認が済んだら、ゲームプレイデータをやり取りする前に、クライアントとサーバーの両方が `NetworkStreamInGame` を追加する必要がある。

```

using Unity.Collections;
using Unity.Entities;
using Unity.NetCode;

[WorldSystemFilter(WorldSystemFilterFlags.ServerSimulation)]
public partial struct ServerGoInGameSystem : ISystem
{

```

```

public void OnUpdate(ref SystemState state)
{
    var ecb = new EntityCommandBuffer(Allocator.Temp);

    foreach (var (id, entity) in
        SystemAPI.Query<RefRO<NetworkId>>()
            .WithNone<NetworkStreamInGame>()
            .WithEntityAccess())
    {
        ecb.AddComponent<NetworkStreamInGame>(entity);
        // Spawn the player entity and set CommandTarget as needed.
    }

    ecb.Playback(state.EntityManager);
}

[WorldSystemFilter(WorldSystemFilterFlags.ClientSimulation |
    WorldSystemFilterFlags.ThinClientSimulation)]
public partial struct ClientGoInGameSystem : ISystem
{
    public void OnUpdate(ref SystemState state)
    {
        var ecb = new EntityCommandBuffer(Allocator.Temp);

        foreach (var (id, entity) in
            SystemAPI.Query<RefRO<NetworkId>>()
                .WithNone<NetworkStreamInGame>()
                .WithEntityAccess())
        {
            ecb.AddComponent<NetworkStreamInGame>(entity);
        }

        ecb.Playback(state.EntityManager);
    }
}

```

7. Data-stream buffers

Buffer	方向	データ
IncomingRpcDataStreamBuffer	受信	RPC データ。
IncomingCommandDataStreamBuffer	受信	Command データ。
IncomingSnapshotDataStreamBuffer	クライアントで受信	Snapshot データ。
OutgoingRpcDataStreamBuffer	送信	RPC データ。
OutgoingCommandDataStreamBuffer	クライアントで送信	Command データ。

8. Host migration

Host Migration は、サーバーが切り替わってもクライアントの接続状態を保持する仕組みだ。

項目	意味
有効化	1.7.0 系から既定で有効。それ以前のバージョンでは <code>ENABLE_HOST_MIGRATION</code> が必要だった。
再接続追跡	再接続後、両方の接続 Entity に <code>NetworkStreamIsReconnected</code> が追加される。
Prespawn Ghosts	Prespawn Ghost の ID は、Host Migration をまたいでも保持される。

1.7.0 系で Host Migration の API 名が大きく見直されたため、古いサンプルは使う前に移行が必要だ。

9. Related docs

- 16_Netcode Client-Server World & Bootstrap.md — Netcode World の生成方法。
- 18_Netcode Ghost Snapshot & Synchronization.md — `NetworkStreamInGame` が Snapshot のやり取りを制御する理由。
- 21_Netcode RPC.md — 通常の RPC と Approval RPC。

10. Troubleshooting

症状	原因 / 対処
接続は成功するのに Ghost が表示されない	クライアントとサーバーの両方に <code>NetworkStreamInGame</code> を追加する。
承認後もデータがやり取りされない	接続 Entity に <code>ConnectionApproved</code> が追加されているか確認する。
アプリがフォーカスを失うと接続が切れる	<code>Application.runInBackground = true</code> を設定する。
プロトコルバージョンの不一致	クライアントとサーバーで、同一のビルドと Ghost 設定を使う。
Approval がタイムアウトする	<code>HandshakeApprovalTimeoutMS</code> を調整するか、Approval RPC の経路をデバッグする。

Chapter 21. Netcode Ghost Snapshot & Synchronization

Unity 6000.5 · Entities 6.5.0 · Netcode for Entities 6.5.0

1. Overview

Ghost は、サーバーでシミュレートされ、クライアントへレプリケートされるネットワーク Entity だ。サーバーは Ghost の状態を定期的に Snapshot として送り、各クライアントは補間または予測を使ってその Snapshot を適用する。

メカニズム	用途
Ghost	信頼性のないパケットの上で結果整合性を実現する。 レプリケーションの最適化も組み込みで備える。
RPC	一回限りで送る、信頼性のあるイベント。

2. Ghost vs RPC

Ghost を使う場合	RPC を使う場合
クライアントの近くにある、空間的な Entity の状態。	ゲーム進行レベルのイベント。
クライアント予測が必要なデータ。	チャットや設定変更など、一回限りのコマンド。
切断・再接続をはさんでも残しておきたい状態。	レプリケートして残す状態を持たないイベント。

3. Ghost authoring

レプリケートしたい Prefab に GhostAuthoringComponent を追加する。

プロパティ	意味
Name	Ghost の識別子。
Importance	帯域が逼迫したときの Snapshot の優先度。
Supported Ghost Mode	All、Interpolated、または Predicted。

プロパティ	意味
Default Ghost Mode	Interpolated、Predicted、または OwnerPredicted。
Optimization Mode	既定は Dynamic。ほとんど変化しないオブジェクトには Static を使う。
MaxSendRate	レプリケーションレートの上限 (Hz)。NetworkTickRate によって頭打ちになる。
UseSingleBaseline	1.5.0 系で追加された、シングルベースライン方式の差分圧縮オプション。

3.1 Ghost modes

モード	意味
Interpolated	クライアントがサーバーの Snapshot 間を補間する。見た目は滑らかだが、その分遅れる。
Predicted	クライアントが先回りしてシミュレートし、入力遅延を抑える。その分 CPU コストは高い。
OwnerPredicted	所有者のクライアントは予測し、ほかのクライアントは補間する。

4. Serialization attributes

4.1 GhostField

シリアライズしたい Component のフィールドに **GhostField** を付ける。

```
using Unity.Entities;
using Unity.Mathematics;
using Unity.NetCode;

public struct MyComponent : IComponentData
{
    [GhostField(Quantization = 1000, Smoothing = SmoothingAction.Interpolate)]
    public float3 Position;

    [GhostField(Quantization = 100)]
    public float Health;

    [GhostField]
    public int Score;

    [GhostField(SendData = false)]
    public float LocalOnly;
}
```


GhostField プロパティ	意味
Quantization	float を整数に変換するための乗数。 1000 なら小数点以下 3 桁を保持する。
Smoothing	Clamp、Interpolate、または InterpolateAndExtrapolate。
MaxSmoothingDistance	この距離を超えると補間を切る、テレポート判定用のしきい値。
Composite	構造体をまとめて 1 つの変更ビットとして扱う。
SendData	false にすると、そのフィールドをシリアライズの対象から外す。
SubType	専用のシリアライズルールを選ぶ。

4.2 GhostEnabledBit

Enableable Component の有効状態をレプリケートするには **GhostEnabledBit** を使う。

```
using Unity.Entities;
using Unity.NetCode;

[GhostEnabledBit]
public struct MyEnableableComp : IComponentData, IEnableableComponent
{
    [GhostField] public int Value;
}
```

4.3 GhostComponent

Prefab バリエーションや、子 Entity のシリアライズ、レプリケーションの範囲を制御するには **GhostComponent** を使う。

```
using Unity.Entities;
using Unity.NetCode;

[GhostComponent(
    PrefabType = GhostPrefabType.All,
    SendTypeOptimization = GhostSendType.AllClients,
    SendToOwner = SendToOwnerType.SendToNonOwner)]
public struct EnemyData : IComponentData
{
    [GhostField] public float Health;
}
```

プロパティ	オプション
PrefabType	InterpolatedClient、PredictedClient、Client、Server、AllPredicted、All。
SendTypeOptimization	None、Interpolated、Predicted、All。
SendToOwner	SendToOwner、SendToNonOwner、All。

プロパティ	オプション
SendDataForChildEntity	子 Entity の Component データをレプリケートする。ルート Entity のデータより遅い。

5. Buffer serialization

`IBufferElementData` では、すべての `public` フィールドに `[GhostField]` を付ける必要がある。

```
using Unity.Entities;
using Unity.NetCode;

[InternalBufferCapacity(8)]
public struct MyBuffer : IBufferElementData
{
    [GhostField] public int Value;
    [GhostField] public float Score;
}
```

バッファは `Component` よりも条件が厳しい。バッファ要素では、シリアライズするフィールドを 1 つ選ぶのではなく、すべての `public` フィールドに注釈を付ける必要がある。

6. Ghost component variants

元の `Component` 型に手を加えずに別のシリアライズスキーマを定義したいときは、バリエントを使う。

```
using Unity.Mathematics;
using Unity.NetCode;
using Unity.Transforms;

[GhostComponentVariation(typeof(LocalTransform), "Transform - 2D")]
[GhostComponent(PrefabType = GhostPrefabType.All)]
public struct Transform2dVariant
{
    [GhostField(Quantization = 1000, Smoothing = SmoothingAction.InterpolateAndExtrapolate)]
    public float3 Position;
}
```

バリエント	用途
ClientOnlyVariant	Component をクライアントにだけ存在させる。
ServerOnlyVariant	Component をサーバーにだけ存在させる。
DontSerializeVariant	シリアライズをまったく行わない。

7. Snapshot system

7.1 Partial snapshots

Snapshot が MTU を超えそうなとき、Netcode は重要度の高い Ghost から先に送る。残りの Ghost は後続のティックで送られるため、World の状態は少しずつストリーミングされていく。

7.2 NetworkTickRate

NetworkTickRate が SimulationTickRate より低いと、GhostSendSystem の処理は複数のシミュレーションティックに分散される。

```
SimulationTickRate = 60, NetworkTickRate = 30
Tick 1: GhostSendSystem sends a snapshot
Tick 2: GhostSendSystem skips
Tick 3: GhostSendSystem sends a snapshot
...
```

8. Ghost prefab registration workflow

1. Create a Ghost prefab
 - └ Add GhostAuthoringComponent
 - └ Add the required authoring components
2. Place or reference it from a SubScene
 - └ Put the prefab inside the SubScene, or
 - └ Let GhostCollection register the prefab
3. Bake
 - └ Baker analyzes GhostField attributes
 - └ Source Generator emits serialization code
4. Runtime
 - └ Server instantiates the Ghost entity
 - └ GhostSendSystem sends snapshots
 - └ Client automatically spawns the Ghost

8.1 Prespawned Ghosts

SubScene の中に配置した Ghost は **Prespawned Ghosts** になる。

- SubScene のロード時に、サーバーとクライアントが自動的に同じ Entity を対応づける。
- 個別のスポンメッセージは要らない。Snapshot の同期だけで十分だ。
- 1.6.1 以降の系では、Prespawn Ghost の ID が Host Migration をまたいでも保持される。

建物や地形といった動かない World オブジェクトは、たいてい Static Optimization Mode の Prespawned Ghosts にするのがよい。

9. Checklist

- ☐ Ghost Prefab に GhostAuthoringComponent を追加する。
- ☐ レプリケートするフィールドに [GhostField] を付ける。
- ☐ Ghost Mode を選ぶ。Predicted、Interpolated、OwnerPredicted のいずれか。
- ☐ Snapshot のやり取りを始められるよう NetworkStreamInGame を追加する。
- ☐ 帯域と精度のバランスを見て Quantization を調整する。
- ☐ 子 Entity のレプリケーションが必要なときだけ SendDataForChildEntity を有効にする。

10. Related docs

- 22_Netcode Ghost Optimization ・ Importance ・ Relevancy.md — 帯域、関連性、送信レートの制御。
- 19_Netcode Prediction & Rollback.md — 予測 Ghost のシミュレーション。
- ../Changelog/Netcode for Entities 1.4 → 6.5 Key Changes.md — バージョン間の Ghost シリアライズ変更。

11. Troubleshooting

症状	原因 / 対処
Ghost がクライアントに表示されない	NetworkStreamInGame を追加し、Ghost Prefab が登録されているか確認する。
float 値がガタつく	Quantization が低すぎるか、スムージングを設定していない。
テレポート時に補間によるノイズが出る	MaxSmoothingDistance を設定するか、独自のスムージング処理を使う。
プロトコルバージョンの不一致	クライアントとサーバーの Ghost スキーマが異なる。
バッファがシリアライズされない	バッファ要素のすべての public フィールドに [GhostField] を付ける必要がある。

Chapter 22. Netcode Prediction & Rollback

Unity 6000.5 · Entities 6.5.0 · Netcode for Entities 6.5.0

1. Overview

クライアント予測とは、権威を持つサーバーの結果が届くより前に、クライアント側で同じシミュレーションコードを走らせることだ。これにより入力遅延が減り、プレイヤーが操作するキャラクターの動きが即座に反応しているように感じられる。

予測には相応の CPU コストがかかる。権威ある Snapshot が届くと、クライアントは古いティックまで巻き戻し、現在の予測ティックまで再シミュレーションし直す。

2. Core components and systems

Component / system	意味
PredictedGhost	クライアント側の予測 Ghost と、サーバー側のすべての Ghost に追加される。
Simulate	現在の予測ティックでシミュレートすべき Entity に立てられる、Enableable なタグ。
PredictedSimulationSystemGroup	予測ループを実行する固定タイムステップグループ。
NetworkTime	サーバーティック、予測ティック、関連するタイミングデータを保持する。

3. Client prediction flow

1. Receive a server snapshot
 - └ Apply the latest authoritative state to predicted entities
2. Find the oldest applied tick
 - └ Use the oldest tick across affected predicted entities
3. Roll back and re-simulate
 - └ PredictedSimulationSystemGroup repeats from oldest tick to target tick
 - └ TimeData and NetworkTime update for each tick
4. Render
 - └ Present the predicted result

レイテンシが高いほど、このコストはふくらむ。300 ms の接続では、ティックレートや予測設定によっては、20 フレーム以上の再シミュレーションが必要になることもある。

4. Writing prediction systems

4.1 Use the Simulate tag

予測ループの先頭で、Netcode は予測 Ghost の **Simulate** をいったん無効にし、現在のティックで実行すべき Entity だけを有効にする。

```
using Unity.Burst;
using Unity.Entities;
using Unity.Mathematics;
using Unity.NetCode;
using Unity.Transforms;

[UpdateInGroup(typeof(PredictedSimulationSystemGroup))]
public partial struct MovementSystem : ISystem
{
    [BurstCompile]
    public void OnUpdate(ref SystemState state)
    {
        var speed = 5f;
        var dt = SystemAPI.Time.DeltaTime;

        foreach (var (transform, input) in
            SystemAPI.Query<RefRW<LocalTransform>, RefRO<PlayerInput>>()
                .WithAll<Simulate>())
        {
            var move = new float3(input.ValueRO.Horizontal, 0, input.ValueRO.Vertical);
            transform.ValueRW.Position += move * speed * dt;
        }
    }
}
```

予測クエリには、必ず **.WithAll<Simulate>()** を入れる。これを忘れると、すでに処理済みのティックの Entity まで再シミュレートしてしまう。

4.2 Server-side behavior

サーバーでは、予測ループはサーバーティックごとにちょうど 1 回だけ走る。

- **TimeData** には、通常のサーバーティックのデータが入る。
 - **Simulate** は常に有効になっている。
 - 同じシステムコードが、クライアントとサーバーの両方で動く。
-

5. Remote-player prediction

ほかのプレイヤーのコマンドデータがシリアル化されていれば、そのリモートプレイヤーも予測の対象にできる。

```
using Unity.Burst;
using Unity.Entities;
using Unity.Mathematics;
using Unity.NetCode;
using Unity.Transforms;

[UpdateInGroup(typeof(PredictedSimulationSystemGroup))]
public partial struct RemotePlayerMovement : ISystem
{
    [BurstCompile]
    public void OnUpdate(ref SystemState state)
    {
        foreach (var (transform, input) in
            SystemAPI.Query<RefRW<LocalTransform>, RefRO<PlayerInput>>()
                .WithAll<PredictedGhost, Simulate>())
        {
            transform.ValueRW.Position +=
                new float3(input.ValueRO.Horizontal, 0, input.ValueRO.Vertical)
                    * SystemAPI.Time.DeltaTime;
        }
    }
}
```

`IInputComponentData` を使うと、入力システムが現在のティックぶんの入力を自動的に供給してくれる。

6. Prediction smoothing

Prediction smoothing は、サーバーの結果が予測結果とずれたときに、目につく補正の動きをやわらげる。

```
using Unity.NetCode;
using Unity.Transforms;

GhostPredictionSmoothing.RegisterSmoothingAction<LocalTransform>(
    state.EntityManager,
    CustomSmoothing.Action);
```

```
using Unity.Burst;
using Unity.Collections.LowLevel.Unsafe;
using Unity.Mathematics;
using Unity.NetCode;
using Unity.Transforms;

[BurstCompile]
public static unsafe class CustomSmoothing
```

```

{
    public static readonly PortableFunctionPointer<
        GhostPredictionSmoothing.SmoothingActionDelegate> Action =
        new(SmoothingAction);

    [BurstCompile(DisableDirectCall = true)]
    private static void SmoothingAction(
        void* currentData, void* previousData, void* userData)
    {
        ref var current = ref UnsafeUtility.AsRef<LocalTransform>(currentData);
        ref var backup = ref UnsafeUtility.AsRef<LocalTransform>(previousData);

        var dist = math.distance(current.Position, backup.Position);
        if (dist > 0)
        {
            current.Position = math.lerp(backup.Position, current.Position, 0.5f);
        }
    }
}

```

7. Prediction switching

予測はコストが高いため、Ghost は実行時に **Predicted** と **Interpolated** を切り替えられるようになっている。

7.1 Switch prerequisites

- Supported Ghost Modes が **All** であること。
- 現在のモードが **OwnerPredicted** でないこと。
- その Ghost がすでに切り替え処理中でないこと。

7.2 Queueing switches

```

using Unity.NetCode;

ref var switchQueues = ref SystemAPI
    .GetSingletonRW<GhostPredictionSwitchingQueues>()
    .ValueRW;

switchQueues.ConvertToPredictedQueue.Enqueue(new ConvertPredictionEntry
{
    TargetEntity = entity,
    TransitionDurationSeconds = 1f,
});

switchQueues.ConvertToInterpolatedQueue.Enqueue(new ConvertPredictionEntry
{
    TargetEntity = entity,
    TransitionDurationSeconds = 0f,
});

```


Interpolated の Ghost と Predicted の Ghost は別々のタイムライン上で動き、ふつうは ping 約 2 回分ほど離れている。SwitchPredictionSmoothing は、設定した時間をかけて位置と回転を補間し、目に見えるテレポートを防ぐ。

8. Prediction performance controls

テクニック	効果
Prediction Switching	遠くにある、または重要度の低い Ghost を Interpolated に切り替える。
Restrict GhostField write access	予測ループの中で変化しないフィールドは、読み取り専用でアクセスする。
Physics batching	物理の再シミュレーションを繰り返す場合は MaxPredictionStepBatchSizeRepeatedTick を使う。
Tick-rate tuning	SimulationTickRate を下げると再シミュレーションの回数は減るが、その分だけ応答性が犠牲になる。

9. Related docs

- 20_Netcode Command Stream & Input.md — 予測システムが使う入力データ。
- 22_Netcode Ghost Optimization ・ Importance ・ Relevancy.md — 最適化手法としての Prediction switching。
- 23_Netcode Physics Integration & Lag Compensation.md — 予測下での物理の挙動。

10. Troubleshooting

症状	原因 / 対処
予測が実行されない	Ghost モードが Predicted になっているか、また予測クエリに Simulate が含まれているかを確認する。
動きが激しくガタつく	Prediction smoothing を登録し、Quantization がゲームの要件に合っているか確認する。
サーバー補正で Entity がテレポートする	MaxSmoothingDistance を設定するか、独自のスムージングを追加する。
予測の CPU コストが高すぎる	Prediction switching を使い、ティックレートを下げ、必要な箇所では物理の再シミュレーションをまとめて処理する。

症状	原因 / 対処
1ティックで Entity が複数回シミュレートされる	予測クエリに <code>.WithAll<Simulate>()</code> が抜けている。

Chapter 23. Netcode Command Stream & Input

Unity 6000.5 · Entities 6.5.0 · Netcode for Entities 6.5.0

1. Overview

クライアントはゲームに参加すると、サーバーへ向けて **command stream** を絶え間なく送り続ける。このストリームには入力データと Snapshot ACK データが含まれる。プレイヤーが実際には何も操作していないときでも、接続の同期を保つために送信は止まらない。

2. ICommandData vs IInputComponentData

	ICommandData	IInputComponentData
用途	低レベルでコマンドを制御したいとき。	高レベルな入力ワークフロー。
バッファ管理	手動。AddCommandData で行う。	生成コードによって自動で処理される。
ティックマッピング	手動。GetDataAtTick で行う。	自動。現在のティックの入力が渡される。
InputEvent	未対応。	一回限りのイベントに対応。
最大ペイロード	1024 バイト。	1024 バイト。
適した用途	独自のコマンドパイプライン。	通常のプレイヤー入力。

3. IInputComponentData

3.1 Define input

```
using Unity.NetCode;

public struct PlayerInput : IInputComponentData
{
    public int Horizontal;
    public int Vertical;
    public InputEvent Jump;
}
```

InputEvent は **GetKeyDown** のような一回限りのイベントを保持する。冗長に送られたコマンドパケットの一部が失われても、サーバー側では確実にちょうど 1 回だけ登録される。

3.2 Gather input on the client

```
using Unity.Entities;
using Unity.NetCode;
using UnityEngine;

[UpdateInGroup(typeof(GhostInputSystemGroup))]
public partial struct GatherInputSystem : ISystem
{
    public void OnCreate(ref SystemState state)
    {
        state.RequireForUpdate<PlayerInput>();
    }

    public void OnUpdate(ref SystemState state)
    {
        bool jump = Input.GetKeyDown(KeyCode.Space);
        bool left = Input.GetKey(KeyCode.A);
        bool right = Input.GetKey(KeyCode.D);

        foreach (var input in
            SystemAPI.Query<RefRW<PlayerInput>>()
                .WithAll<GhostOwnerIsLocal>())
        {
            input.ValueRW = default;
            if (jump) input.ValueRW.Jump.Set();
            if (left) input.ValueRW.Horizontal -= 1;
            if (right) input.ValueRW.Horizontal += 1;
        }
    }
}
```

3.3 Process input on server and prediction loop

```
using Unity.Burst;
using Unity.Entities;
using Unity.Mathematics;
using Unity.NetCode;
using Unity.Transforms;

[UpdateInGroup(typeof(PredictedSimulationSystemGroup))]
public partial struct ProcessInputSystem : ISystem
{
    public void OnCreate(ref SystemState state)
    {
        state.RequireForUpdate<PlayerInput>();
    }

    [BurstCompile]
    public void OnUpdate(ref SystemState state)
    {
        var speed = 3f;
        var dt = SystemAPI.Time.DeltaTime;
```

```

    foreach (var (input, transform) in
        SystemAPI.Query<RefR0<PlayerInput>, RefRW<LocalTransform>>()
            .WithAll<Simulate>())
    {
        if (input.ValueR0.Jump.IsSet)
        {
            // Jump logic; runs once on the input tick.
        }

        transform.ValueRW.Position +=
            new float3(input.ValueR0.Horizontal, 0, 0) * speed * dt;
    }
}

```

予測ループでは、再シミュレーションの最中にこの **System** が何度も走ることがある。
InputEvent.IsSet が true になるのは、本来のティックのときだけだ。

4. ICommandData

4.1 Define a command

```

using Unity.Mathematics;
using Unity.NetCode;

public struct MyCommand : ICommandData
{
    public NetworkTick Tick { get; set; }
    public int Action;
    public float2 Direction;
}

```

4.2 Add and read command data

```

using Unity.Entities;
using Unity.NetCode;

DynamicBuffer<MyCommand> buffer = default;
NetworkTick networkTick = default;

buffer.AddCommandData(new MyCommand
{
    Tick = networkTick,
    Action = 1,
    Direction = new float2(1, 0),
});

if (buffer.GetDataAtTick(networkTick, out var cmd))
{

```

```
// Use cmd.Action and cmd.Direction.  
}
```

5. Command-send mechanism

```
Command packet  
├ Tick, Command  
├ Delta(Tick - 1)  
├ Delta(Tick - 2)  
└ Delta(Tick - 3)
```

The last three inputs are redundantly sent with delta compression.
Maximum payload: 1024 bytes.

CommandSendPacketSystem は SimulationTickRate のペースでフラッシュする。

CommandReceiveSystem はサーバー側でコマンドデータを受け取り、対象の Entity へ渡す。

6. AutoCommandTarget

Ghost に AutoCommandTarget が付いていれば、コマンドは自動で送信される。

要件	意味
Ghost has owner enabled	GhostAuthoringComponent で有効にする。
Support Auto Command Target enabled	GhostAuthoringComponent で有効にする。
Client is the owner	サーバーが、GhostOwner.NetworkId にクライアントの NetworkId.Value を設定する。
Ghost is Predicted or OwnerPredicted	コマンドデータが予測の対象になる。
AutoCommandTarget.Enabled is true	実行時に無効にできる。

AutoCommandTarget を使わないなら、接続 Entity の CommandTarget を手動で設定する。

```
using Unity.Entities;  
using Unity.NetCode;  
  
Entity connectionEntity = default;  
Entity playerEntity = default;  
  
state.EntityManager.SetComponentData(connectionEntity,  
    new CommandTarget { targetEntity = playerEntity });
```

7. Identifying locally owned entities

7.1 GhostOwnerIsLocal

```
using Unity.Entities;
using Unity.NetCode;
using Unity.Transforms;

foreach (var (input, transform) in
    SystemAPI.Query<RefRW<PlayerInput>, RefRW<LocalTransform>>>()
        .WithAll<GhostOwnerIsLocal>())
{
    // Local-owned entity only.
}
```

7.2 Manual owner check

```
using Unity.Entities;
using Unity.NetCode;
using Unity.Transforms;

var localId = SystemAPI.GetSingleton<NetworkId>().Value;

foreach (var (owner, transform) in
    SystemAPI.Query<RefRO<GhostOwner>, RefRW<LocalTransform>>>())
{
    if (owner.ValueRO.NetworkId == localId)
    {
        // Local-owned entity.
    }
}
```

8. Forced input latency

`ClientTickRate.ForcedInputLatencyTicks` は、入力に意図的な遅延を加える。

設定	意味
ForcedInputLatencyTicks	上乗せする入力遅延（ティック単位）。既定値は 0。
ユースケース	格闘ゲームのようなジャンルでは、全プレイヤーの入力遅延を同じにそろえる。
効果	<code>MaxPredictAheadTimeMS</code> の計算に自動で組み込まれる。

8.1 InputTargetTick vs ServerTick

Forced input latency が有効なときは、入力システムで `NetworkTime.ServerTick` ではなく `NetworkTime.InputTargetTick` を使う。

```
using Unity.NetCode;

var inputTick = SystemAPI.GetSingleton<NetworkTime>().InputTargetTick;
var serverTick = SystemAPI.GetSingleton<NetworkTime>().ServerTick;
```

`ForcedInputLatencyTicks` が 0 でないとき、`InputTargetTick` と `ServerTick` はずれる。入力は `InputTargetTick` を対象にする。

9. Related docs

- [19_Netcode Prediction & Rollback.md](#) — 入力を受け取って使う予測システム。
- [17_Netcode Network Connection & Approval.md](#) — `NetworkStreamInGame` と接続 Entity。
- [21_Netcode RPC.md](#) — 継続的な入力ではなく、一回限りで送る信頼性のあるメッセージ。

10. Troubleshooting

症状	原因 / 対処
サーバーが入力を受信しない	<code>NetworkStreamInGame</code> を追加し、 <code>CommandTarget</code> を設定するか、 <code>AutoCommandTarget</code> の要件をすべて満たす。
ジャンプが複数回発火する	素の <code>bool</code> ではなく、 <code>InputEvent.Set()</code> / <code>.IsSet</code> を使う。
ペイロードサイズのエラーが出る	コマンド構造体を 1024 バイト以内に収める。
予測中に入力が抜け落ちる	<code>IInputComponentData</code> の自動ティックマッピングを使うか、 <code>GetDataAtTick</code> を確認する。
サーバーのコードが <code>GhostInputSystemGroup</code> で動いてしまう	このグループはクライアント側専用だ。サーバーのロジックは <code>PredictedSimulationSystemGroup</code> に置く。

Chapter 24. Netcode RPC

Unity 6000.5 · Entities 6.5.0 · Netcode for Entities 6.5.0

1. Overview

RPC は、クライアントとサーバーのあいだでやり取りする **信頼性のある、一回限りのメッセージ** だ。RPC のシリアライズ・デシリアライズコードは自動生成され、RPC の処理も ECS / Job System の枠組みにきちんと収まる。

RPC は、Ghost レプリケーションやコマンドストリームの代わりではない。高頻度に変わる状態ではなく、単発のゲーム進行イベントに使う。

2. RPC vs Ghost vs Command

	RPC	Ghost	Command
トランスポートの挙動	信頼性あり、順序保証あり。	非信頼。 レプリケーションの最適化を伴う。	非信頼。冗長送信を伴う。
方向	双方向。	サーバー → クライアント。	クライアント → サーバー。
頻度	一回限り。	必要に応じて、 ネットワークティックごとに。	シミュレーションティックごとに。
用途	ゲーム進行、ロビー、チャット。	レプリケートする Entity の状態。	プレイヤー入力。
送信中パケットの上限	あり。 信頼性パイプラインの制限による。	なし。	なし。

RPC は順序保証と信頼性を備えるため、RPC を頻繁に流すと、送信中のパケットの後ろで詰まってしまうことがある。

3. Define RPCs

3.1 Empty RPC

```
using Unity.NetCode;

public struct PingRpc : IRpcCommand
{
}
```

3.2 RPC with data

```
using Unity.Collections;
using Unity.NetCode;

public struct ChatMessageRpc : IRpcCommand
{
    public FixedString128Bytes Message;
    public int SenderId;
}
```

4. Send RPCs

4.1 Client to server

```
using Unity.Collections;
using Unity.Entities;
using Unity.NetCode;
using UnityEngine;

[WorldSystemFilter(WorldSystemFilterFlags.ClientSimulation)]
public partial struct SendChatSystem : ISystem
{
    public void OnCreate(ref SystemState state)
    {
        state.RequireForUpdate<NetworkId>();
    }

    public void OnUpdate(ref SystemState state)
    {
        if (!Input.GetKeyDown(KeyCode.Space))
            return;

        var ecb = new EntityCommandBuffer(Allocator.Temp);
        var entity = ecb.CreateEntity();

        ecb.AddComponent(entity, new ChatMessageRpc
        {
            Message = "Hello!",
            SenderId = SystemAPI.GetSingleton<NetworkId>().Value,
        });
    }
}
```

```

    });

    ecb.AddComponent<SendRpcCommandRequest>(entity);
    ecb.Playback(state.EntityManager);
}
}

```

ISystem は構造体だ。フレームをまたぐ状態を **System** のインスタンスフィールドに持たせてはいけない。状態は **Component** か **Singleton** に保存する。

4.2 Server to one client

```

using Unity.Entities;
using Unity.NetCode;

Entity targetConnectionEntity = default;

var entity = ecb.CreateEntity();
ecb.AddComponent(entity, new NotifyRpc());
ecb.AddComponent(entity, new SendRpcCommandRequest
{
    TargetConnection = targetConnectionEntity,
});

```

4.3 Server broadcast

```

using Unity.Entities;
using Unity.NetCode;

var entity = ecb.CreateEntity();
ecb.AddComponent(entity, new GameStartRpc());
ecb.AddComponent(entity, new SendRpcCommandRequest
{
    TargetConnection = Entity.Null,
});

```

Entity.Null を指定すると、RPC はブロードキャストされる。

5. Receive RPCs

```

using Unity.Collections;
using Unity.Entities;
using Unity.NetCode;
using UnityEngine;

[WorldSystemFilter(WorldSystemFilterFlags.ServerSimulation)]
public partial struct ReceiveChatSystem : ISystem
{
    public void OnUpdate(ref SystemState state)
    {

```

```

var ecb = new EntityCommandBuffer(Allocator.Temp);

foreach (var (req, chat, entity) in
    SystemAPI.Query<RefRO<ReceiveRpcCommandRequest>, RefRO<ChatMessageRpc>>()
        .WithEntityAccess())
{
    var sourceConnection = req.ValueRO.SourceConnection;
    Debug.Log($"Player {chat.ValueRO.SenderId}: {chat.ValueRO.Message}");

    ecb.DestroyEntity(entity);
}

ecb.Playback(state.EntityManager);
}

```

受信した RPC Entity は、処理し終わったら必ず破棄する。さもないと、同じ RPC が毎フレーム繰り返して処理されてしまう。

6. RpcQueue

RpcQueue を使うのは、RPC のスケジューリングを手動で行いたいときだけにする。

```

using Unity.Entities;
using Unity.NetCode;

var rpcQueue = SystemAPI.GetSingleton<RpcCollection>()
    .GetRpcQueue<MyRpc, MyRpc>();
var bufferLookup = SystemAPI.GetBufferLookup<OutgoingRpcDataStreamBuffer>();

if (bufferLookup.HasBuffer(connectionEntity))
{
    var buffer = bufferLookup[connectionEntity];
    rpcQueue.Schedule(buffer, new MyRpc());
}

```

7. Manual serialization

既定では自動生成が使われる。これを無効にするなら、シリアライズとデシリアライズの処理を対称に保つ必要がある。

```

using Unity.Burst;
using Unity.Entities;
using Unity.NetCode;
using Unity.Networking.Transport;

[BurstCompile]
public struct MyCustomRpc : IComponentData, IRpcCommandSerializer<MyCustomRpc>
{

```

```

public int Data;

public void Serialize(ref DataStreamWriter writer, in RpcSerializerState state,
    in MyCustomRpc data)
{
    writer.WriteInt(data.Data);
}

public void Deserialize(ref DataStreamReader reader, in RpcDeserializerState state,
    ref MyCustomRpc data)
{
    data.Data = reader.ReadInt();
}

// Execute implementation omitted.
}

```

8. IApprovalRpcCommand

IApprovalRpcCommand は、接続承認の間にだけ使う特別な RPC だ。通常の IRpcCommand メッセージは、Handshake / Approval の状態の間はブロックされる。

```

using Unity.Collections;
using Unity.NetCode;

public struct LoginApproval : IApprovalRpcCommand
{
    public FixedString512Bytes AuthToken;
}

```

Approval フローの全体像は 17_Netcode Network Connection & Approval.md を参照。

9. Common patterns

パターン	RPC の例
ロビー / マッチメイキング	JoinLobbyRpc、ReadyRpc。
ゲームフロー	GameStartRpc、RoundEndRpc。
チャット	ChatMessageRpc。
アイテム取引	TradeRequestRpc、TradeAcceptRpc。
接続承認	IApprovalRpcCommand を実装したもの。

10. Related docs

- 18_Netcode Ghost Snapshot & Synchronization.md — 状態を扱う Ghost レプリケーション。
 - 20_Netcode Command Stream & Input.md — 入力を扱うコマンドストリーム。
 - 17_Netcode Network Connection & Approval.md — Approval RPC。
-

11. Troubleshooting

症状	原因 / 対処
RPC が受信されない	SendRpcCommandRequest が追加されているか、また System が意図した World で動いているかを確認する。
RPC が毎フレーム処理されてしまう	受信した RPC Entity を、処理後に破棄する。
RPC の順序が遅れているように見える	信頼性のある順序付きトランスポートは、送信中のパケットの後ろで詰まることもある。頻繁に流すデータには Ghost やコマンドを使う。
承認中に RPC がブロックされる	IRpcCommand ではなく IApprovalRpcCommand を使う。
手動シリアライズが失敗する	Serialize と Deserialize が、同じ順序でフィールドを読み書きしているか確認する。

Chapter 25. Netcode Ghost Optimization ・ Importance ・ Relevancy

Unity 6000.5 ・ Entities 6.5.0 ・ Netcode for Entities 6.5.0

1. Overview

大規模なマルチプレイヤー World では、すべての Ghost を毎ティック全クライアントへ送ることはできない。Netcode for Entities は、**Optimization Mode**、**Importance Scaling**、**Relevancy**、そして Ghost ごとの送信レート制御によって、CPU コストと帯域を抑える。

基本となる考え方は単純だ。サーバーは Snapshot の予算を、その接続にとって今まさに重要な Ghost に振り向けるべきだ。

2. Optimization Mode

Optimization Mode は GhostAuthoringComponent で設定する。

モード	意味	適した用途
Dynamic	既定。データが変化するときもしないときも、Snapshot サイズを最適化する。	動きのある一般的な Ghost。
Static	Ghost が変化していなければ、データの再送を行わない。	建物、地形、動かない小道具。

1.11.0 系で Static Ghost の最適化が加わり、変化のない Static Ghost に対する GhostUpdateSystem の処理時間を大きく減らせるようになった。

3. Snapshot-size limits

Snapshot の予算は、いくつかのレベルで制限できる。

設定	意味
NetworkStreamSnapshotTargetSize	接続ごとの、目安となるバイト数（厳密な上限ではない）。
GhostSendSystemData.MaxSendEntities	Snapshot 1 つあたりの Entity 数の上限。

設定	意味
<code>GhostSendSystemData.MaxSendChunks</code>	Snapshot 1 つあたりのチャンク数の上限。

予算に制限があるときは、重要度の高い Ghost から先に送られる。

4. Importance Scaling

Importance Scaling は、Ghost ごとの送信優先度をサーバー側で計算する。

4.1 Inputs

`GhostImportance` は、3 種類のデータを組み合わせて使う。

データ	意味
接続ごとのデータ	プレイヤーの位置など、接続ごとに固有のデータ。
Singleton データ	タイルサイズなど、グローバルな設定。
チャンクごとの共有データ	チャンク単位のスケーリングパラメータ。

4.2 Distance-based importance

```
using Unity.Entities;
using Unity.Mathematics;
using Unity.NetCode;

var gridSingleton = state.EntityManager.CreateSingleton(
    new GhostDistanceData
    {
        TileSize = new int3(50, 50, 256),
        TileCenter = new int3(0, 0, 128),
        TileBorderWidth = new float3(1f, 1f, 1f),
    });

state.EntityManager.AddComponentData(gridSingleton, new GhostImportance
{
    ScaleImportanceFunction = GhostDistanceImportance.ScaleFunctionPointer,
    GhostConnectionComponentType = ComponentType.ReadOnly<GhostConnectionPosition>(),
    GhostImportanceDataType = ComponentType.ReadOnly<GhostDistanceData>(),
    GhostImportancePerChunkDataType = ComponentType.ReadOnly<GhostDistancePartitionShared>(),
});
```

1.8.0 系での破壊的変更：`GhostDistanceImportance` のスケール関数は、`baseImportance` に 1000 を掛けなくなった。旧来の乗数を前提に書いた独自の `Importance` コードは、見直しが必要だ。

4.3 Automatic GhostDistancePartitionShared


```
using Unity.NetCode;

GhostDistancePartitioningSystem.AutomaticallyAddGhostDistancePartitionSharedComponent = true;
```

4.4 Importance Visualizer

1.8.0 以降の系では、PlayMode Tool に Importance Visualizer が含まれている。コードから当たりを付けるのではなく、これで実行時の Importance の分布を確認するとよい。

5. Ghost Relevancy

Ghost Relevancy を使うと、サーバーは接続ごとに、特定の Ghost を見せたり隠したりできる。

モード	意味
Disabled	既定。絞り込みを行わず、すべての Ghost が送信候補になる。
SetIsRelevant	リストに載せた Ghost だけが、その接続にレプリケートされる。
SetIsIrrelevant	リストに載せた Ghost が、その接続から除外される。

5.1 Use cases

シナリオ	実装のアイデア
距離によるカリング	遠くにある Ghost を Irrelevant にする。
フォグ オブ ウォー	視野外の Entity を非表示にする。
クライアント固有の Ghost	その Ghost を、特定の 1 クライアントにだけ Relevant にする。
チームフィルタリング	敵側だけが持つ内部データを隠す。

5.2 Relevancy example

```
using Unity.Entities;
using Unity.Mathematics;
using Unity.NetCode;
using Unity.Transforms;

[WorldSystemFilter(WorldSystemFilterFlags.ServerSimulation)]
[UpdateInGroup(typeof(GhostSimulationSystemGroup))]
[UpdateBefore(typeof(GhostSendSystem))]
public partial struct RelevancySystem : ISystem
{
    public void OnUpdate(ref SystemState state)
    {
        var relevancy = SystemAPI.GetSingletonRW<GhostRelevancy>();
```

```

relevancy.ValueRW.GhostRelevancyMode = GhostRelevancyMode.SetIsRelevant;
relevancy.ValueRW.GhostRelevancySet.Clear();

foreach (var (ghostInstance, transform) in
    SystemAPI.Query<RefRO<GhostInstance>, RefRO<LocalTransform>>>())
{
    foreach (var (connId, connPos) in
        SystemAPI.Query<RefRO<NetworkId>, RefRO<GhostConnectionPosition>>>())
    {
        float dist = math.distance(
            transform.ValueRO.Position,
            connPos.ValueRO.Position);

        if (dist < 100f)
        {
            relevancy.ValueRW.GhostRelevancySet.TryAdd(
                new RelevantGhostForConnection(
                    connId.ValueRO.Value,
                    ghostInstance.ValueRO.ghostId),
                1);
        }
    }
}
}
}
}

```

この簡略化した例は計算量が $O(\text{Ghost} \times \text{接続})$ になる。大規模な World では、距離ベースの Importance パーティショニングを基本とし、Relevancy はフォグ オブ ウォーやチームごとの可視性など、パーティショニングでは表現できないルールに限って使う。

5.3 DefaultRelevancyQuery

DefaultRelevancyQuery では、すべての接続に対して常に Relevant な Ghost チャンクを定義する。グローバルなゲーム状態の Ghost や、UI 関連のネットワーク状態などが該当する。

5.4 Relevancy plus importance

1.6.1 以降の系では、PrioChunks.isRelevant を介して Ghost Relevancy と Importance Scaling を組み合わせられ、より速い Relevancy 処理が使える。

6. Prediction CPU optimization

6.1 Prediction Switching

遠くにある、または重要度の低い Ghost を、Predicted から Interpolated に切り替える。

```

using Unity.Entities;
using Unity.Mathematics;
using Unity.NetCode;

```

```
float distance = math.distance(playerPos, ghostPos);
if (distance > predictionRange)
{
    switchQueues.ConvertToInterpolatedQueue.Enqueue(new ConvertPredictionEntry
    {
        TargetEntity = ghostEntity,
        TransitionDurationSeconds = 0.5f,
    });
}
```

6.2 Minimize GhostField write access

GhostField の Component に書き込みアクセスを持つ Job は、シリアライズのチェックを発生させることがある。予測ループの中で変化しないフィールドは、読み取り専用でアクセスする。

6.3 Physics re-simulation

予測の再シミュレーションが 20 フレーム以上に及ぶ高レイテンシのケースでは、次の手を打つ。

- メインスレッドにまとめたほうが安く済むなら、PhysicsStep Singleton を介してマルチスレッド物理を無効にする。
- 物理ティックの繰り返し処理をまとめるために、MaxPredictionStepBatchSizeRepeatedTick を使う。

7. MaxIterateChunks

GhostSendSystemData.MaxIterateChunks は、1 ティックで処理するチャンク数に上限を設ける。

設定	意味
MaxIterateChunks	1 ティックあたりのシリアライズ CPU コストを抑える。
MaxSendChunks	Snapshot に含めるチャンク数に上限を設け、帯域を抑える。

この 2 つの設定が解決する問題は別物だ。MaxIterateChunks は CPU の処理を分散させ、MaxSendChunks は Snapshot のサイズを抑える。

8. MaxSendRate

GhostAuthoringComponent.MaxSendRate は、Ghost の送信頻度を Hz 単位で制限する。

```
NetworkTickRate = 60, MaxSendRate = 10
→ This Ghost can be included once every 6 network ticks.
```

更新頻度の低い Ghost や、重要度の低い Ghost にこの設定を使う。

9. UseSingleBaseline

`GhostAuthoringComponent.UseSingleBaseline = true` にすると、差分圧縮にベースラインを 1 つだけ使う。CPU コストは下がるが、帯域は増えることがある。

10. Optimization checklist

- ☐ ほとんど変化しない Ghost には **Static Optimization Mode** を使う。
- ☐ `NetworkStreamSnapshotTargetSize` を設定し、接続ごとの帯域を抑える。
- ☐ 距離ベースの Importance Scaling を組み込む。
- ☐ パーティショニングで表現できないルールに限り `Relevancy` を使う。
- ☐ 遠くにある Ghost は `Interpolated` モードに切り替える。
- ☐ 更新頻度の低い Ghost には `MaxSendRate` を使う。
- ☐ 変化しない GhostField データへの書き込みアクセスは避ける。
- ☐ Importance Visualizer で実行時の優先度を確認する。

11. Related docs

- 18_Netcode Ghost Snapshot & Synchronization.md — Ghost のシリアルイズと Snapshot の流れ。
- 19_Netcode Prediction & Rollback.md — Prediction switching の挙動。
- 24_Netcode Profiler & Debugging.md — 帯域と予測コストのプロファイリング。

12. Troubleshooting

症状	原因 / 対処
Ghost が表示されるまで時間がかかりすぎる	Importance が低すぎるか、Snapshot の予算が厳しすぎる。

症状	原因 / 対処
Importance Scaling 導入後に Ghost が送信されなくなる	GhostDistancePartitionShared があるか確認するか、自動追加のオプションを使う。
Relevancy を有効にすると Ghost がちらつく	境界の条件を確認し、毎フレーム一貫した手順で Relevancy セットを組み直す。
帯域には余裕があるのに Ghost が抜け落ちる	MaxSendEntities と MaxSendChunks の制限を確認する。

Chapter 26. Netcode Physics Integration & Lag Compensation

Unity 6000.5 · Entities 6.5.0 · Netcode for Entities 6.5.0

1. Overview

Netcode for Entities は、ネットワーク物理シミュレーションのために **Unity Physics** と連携する。物理の挙動は、Ghost が Interpolated か Predicted かで変わる。

予測物理を動かすには、シーンに Predicted Ghost が少なくとも 1 体は必要だ。

2. Interpolated vs Predicted physics

2.1 Interpolated Ghosts on clients

- 位置と回転は、サーバーの Snapshot から得る。
- `Simulate` が無効になっているため、物理オブジェクトはキネマティックとして振る舞う。
- `PhysicsVelocity` は無視される。

2.2 Predicted Ghosts on clients

- 物理は予測ループの中で実行され、描画 1 フレームにつき複数回走ることもある。
- `PhysicsSystemGroup` は `PredictedFixedStepSimulationSystemGroup` の中で実行される。
- 予測物理では、クライアントがサーバーと同じシミュレーション結果を出すことが求められる。

Netcode initialization

```
└ PhysicsSystemGroup moves under PredictedFixedStepSimulationSystemGroup
└ FixedStepSimulationSystemGroup moves under PredictedFixedStepSimulationSystemGroup
```

3. Predicted physics performance

予測物理はコストが高い。ロールバックのたびに物理が繰り返し実行されるからだ。

3.1 Server batching

```
using Unity.NetCode;

ClientServerTickRate tickRate = default;
tickRate.MaxSimulationStepBatchSize = 4;
tickRate.MaxSimulationStepsPerFrame = 4;
```

3.2 Client batching

```
using Unity.NetCode;

ClientTickRate clientRate = default;
clientRate.MaxPredictionStepBatchSizeFirstTimeTick = 2;
clientRate.MaxPredictionStepBatchSizeRepeatedTick = 4;
```

バッチ処理は予測ミスリスクを高めることがある。常用するのではなく、性能とのトレードオフとして使う。

3.3 Quantization

物理 Ghost の Transform / Velocity の Quantization は、既定で **1000** だ。

高い Quantization	低い Quantization
シミュレーションがより精密になる。	帯域が少なくて済む。
帯域を多く使う。	精度の低下が大きくなる。

4. Lag compensation

クライアントはサーバーより先回りして予測する。そのため、権威を持つサーバーは、クライアントが行動した時刻に合わせて、過去のコリジョンワールドを参照しなければならないことがある。

4.1 Enable lag compensation

SubScene に NetCodePhysicsConfig を追加し、EnableLagCompensation = true にする。

4.2 Query an old collision world

```
using Unity.NetCode;
using Unity.Physics;

var physicsHistory = SystemAPI.GetSingleton<PhysicsWorldHistorySingleton>();

physicsHistory.GetCollisionWorldFromTick(
    serverTick,
    interpolationDelay,
    ref physicsWorld,
    out var collisionWorld);

// Use collisionWorld for raycasts or other physics queries.
```

4.3 Collider deep cloning

設定	デフォルト	意味
DeepCopyDynamicColliders	true	動的コライダーをディープクローンする。6.5.0の既定値。
DeepCopyStaticColliders	false	静的コライダーをディープクローンする。

6.5.0 系では、Blob アセットのアサーションを避けるため、動的 Ghost コライダーのディープクローンが既定で有効になっている。

4.4 Physics history buffer

バージョン	ヒストリサイズ
1.4.x まで	16 ティック。
1.5.0+	32 ティック。
カスタム	コンパイラ定義で指定する <code>GhostSystemConstants.SnapshotHistorySize</code> 。

5. Multi Physics World

VFX やパーティクルのように、ネットワークシミュレーションに関与しないクライアント専用のエフェクトには、別の物理ワールドを使う。

5.1 Setup

1. SubScene に `NetCodePhysicsConfig` を追加する。
2. **Client Non Ghost World** を、1 など 0 以外の値にする。
3. クライアント専用の物理オブジェクトに、同じ値の `PhysicsWorldIndex` を追加する。

5.2 Custom physics system group

```
using Unity.Entities;
using Unity.NetCode;
using Unity.Physics.Systems;

[WorldSystemFilter(WorldSystemFilterFlags.ClientSimulation)]
[UpdateInGroup(typeof(FixedStepSimulationSystemGroup))]
public partial class VfxPhysicsGroup : CustomPhysicsSystemGroup
{
    public const int WorldIndex = 1;

    public VfxPhysicsGroup() : base(WorldIndex, shareStaticColliders: true)
    {
    }
}
```

`shareStaticColliders: true` を指定すると、カスタム物理ワールドがメインの物理ワールドと静的コライダーを共有できる。

6. Physics Proxy

Predicted Ghost を、クライアント専用の物理 Entity と相互作用させたいときは、Physics Proxy を使う。

- CustomPhysicsProxyAuthoring がプロキシ Entity を生成する。
- CustomPhysicsProxyDriver がプロキシと Predicted Ghost を結びつける。
- SyncCustomPhysicsProxySystem が位置と回転を同期する。

7. PredictedFixedStepSimulationSystemGroup

PredictedFixedStepSimulationSystemGroup が更新されるのは、必要な条件がすべてそろったときだけだ。

要件	意味
NetworkStreamInGame Singleton が存在する	接続がゲーム中の状態にある。
Predicted Ghost が少なくとも 1 体存在する	予測物理にシミュレートする対象がある。
固定ティックレートで動いている	固定タイムステップのシミュレーションが有効になっている。

ネットワーク接続がゲーム中の状態になる前や、Predicted Ghost が流れ込んでくる前には、予測物理は動かない。

7.1 Physics before connection

ネットワーク接続が確立する前に物理を動かしたい場合は、予測物理のセットアップを無効にし、通常の FixedStepSimulationSystemGroup 物理が動くようにする。

```
using Unity.Entities;
using Unity.NetCode;

[UpdateInGroup(typeof(InitializationSystemGroup))]
[CreateAfter(typeof(PredictedPhysicsConfigSystem))]
public partial struct DisablePredictedPhysics : ISystem
{
    public void OnCreate(ref SystemState state)
    {
        {
            state.World.GetExistingSystemManaged<PredictedPhysicsConfigSystem>()
                .Enabled = false;
        }
    }
}
```

8. Limitations

制限	意味
Partial tick 未対応	物理は Partial tick を使わない。Physics Interpolation を使う。
マルチワールドデバッグの制限	Unity Physics のデバッグシステムは、マルチワールド構成に完全には対応していない。
最小 Predicted Ghost	予測物理を動かすには、Predicted Ghost が少なくとも 1 体は必要だ。

9. Related docs

- 19_Netcode Prediction & Rollback.md — 予測ループの挙動。
- 22_Netcode Ghost Optimization ・ Importance ・ Relevancy.md — Ghost を予測から外す方法。
- 24_Netcode Profiler & Debugging.md — 予測物理のコストのプロファイリング。

10. Troubleshooting

症状	原因 / 対処
物理が動かない	Predicted Ghost と NetworkStreamInGame があるか確認する。
物理の動きがガタつく	Quantization と Prediction smoothing を確認する。
Lag compensation のレイキャストが失敗する	EnableLagCompensation を有効にし、 ヒストリバッファが必要な範囲をカバーしているか確認する。
Blob アセットのアサーション	コライダーのディープクローン設定を確認する。6.5.0 では動的コライダーのクローンが既定で有効だ。
クライアント専用の物理が動かない	PhysicsWorldIndex が Client Non Ghost World の値と一致しているか確認する。
予測物理の CPU コストが高すぎる	バッチ処理を有効にし、不要な Ghost は Interpolated に切り替える。

Chapter 27. Netcode Profiler & Debugging

Unity 6000.5 · Entities 6.5.0 · Netcode for Entities 6.5.0

1. Overview

Netcode for Entities には、接続状態、Snapshot、予測、補間、帯域を分析するための **Netcode Profiler** が用意されている。1.12.0 系で、ブラウザベースの Network Debugger は廃止され、組み込みの Unity Profiler モジュールに置き換えられた。

2. Netcode Profiler

2.1 Access

Unity 6.0 以降では、Unity Profiler に Netcode モジュールが 2 つ含まれている。

モジュール	確認する内容
Server Profiler	サーバー側の Ghost 送信、Snapshot の構成、帯域。
Client Profiler	クライアントの受信、予測ティック、補間の状態。

2.2 Key features

機能	意味
Ghost Snapshot View	Ghost ごとの Snapshot サイズと構成を、ツリー形式で表示する。
Tick navigation	Snapshot を送受信したフレームのあいだを行き来する。
Search fields	Ghost Snapshot TreeView と Prediction Error List を検索する。
Client ticks	Prediction Tick と Interpolation Tick を可視化する。
Context menu	6.5.0 系では、TreeView のラベルから Ghost Prefab や Component を確認できる。
Average per-entity column	Entity ごとの平均データサイズを表示する。

2.3 NETCODE_PROFILER_ENABLED

1.12.0 系で、NETCODE_PROFILER_ENABLED のスクリプティング定義は不要になった。この定義がなくても Profiler を使える。

3. Network Debugger

ブラウザベースの Network Debugger は 1.12.0 系で廃止された。代わりに Netcode Profiler を使う。

かつては **Multplayer** → **Open NetDbg** という流れで、Snapshot の構成、Ghost タイプの分布、サイズ分析を見るブラウザベースのビューを開いていた。

4. Multiplayer PlayMode Window

Multiplayer PlayMode Window は、Play モードでクライアント / サーバー構成をテストするための、エディター上の中心的なワークフローだ。

機能	意味
Play Mode Type	Client + Server、Client Only、Server Only から選ぶ。
Thin Client count	ダミーのクライアントの数を設定する。
Simulator	ネットワークレイテンシとパケットロスをシミュレートする。
Importance Visualizer	1.8.0 以降の系で、Importance Scaling を可視化する。

5. Debugging checklists

5.1 Connection

- ☐ Application.runInBackground = true
- ☐ Protocol versions match between client and server
- ☐ NetworkStreamInGame is present
- ☐ Firewall and port are open
- ☐ Unity Transport version is compatible

5.2 Ghost synchronization

- ☐ GhostAuthoringComponent is present
- ☐ GhostField attributes are correct
- ☐ Quantization is appropriate

- Smoothing is configured
- Ghost prefab is registered on both sides

5.3 Prediction

- Systems run in PredictedSimulationSystemGroup
- Queries include Simulate
- Simulation code is deterministic enough for prediction
- Client and server run the same simulation code
- Prediction smoothing is registered where needed

5.4 Input and commands

- Input is gathered in GhostInputSystemGroup
- AutoCommandTarget requirements are met, or CommandTarget is set manually
- Command payload is 1024 bytes or less
- One-shot input uses InputEvent

6. Prefab List menu

1.11.0 以降の系には Prefab List メニューがあり、プロジェクトやシーン Hierarchy に登録された Ghost Prefab を手早く一覧表示して確認できる。

7. Useful debugging components

Component	用途
NetworkStreamConnection	接続状態を確認する。
NetworkId	クライアント ID を確認する。
GhostInstance	Ghost タイプと ID を確認する。
PredictedGhost	予測状態と適用済みティックを確認する。
NetworkSnapshotAck	Snapshot ACK 状態と推定 RTT を確認する。
GhostOwner	Ghost のオーナー ID を確認する。

7.1 Entity Inspector

Entities Hierarchy で Ghost Entity を選ぶと、次の内容を確認できる。

- ライブの GhostField 値、

- 予測ティックデータ、
 - ネットワーク接続状態。
-

8. Packet dumps

低レベルなネットワークパケットの分析が必要なときは、パケットダンプを使う。1.6.2 系では、パケットのタイムスタンプログに `UnityEngine.Time.frameCount` が加わり、フレーム単位での突き合わせがしやすくなった。

9. Common bottlenecks

エリア	確認場所
Snapshot 帯域幅	Profiler Ghost Snapshot View で Entity ごとのサイズを確認する。
予測再シミュレーション	Profiler Client Tick で予測ティック数を確認する。
GhostSendSystem	Profiler Timeline でシステムの実行時間を確認する。
シリアライズコスト	GhostField への書き込みアクセスと変更頻度を確認する。
物理再シミュレーション	PredictedFixedStepSimulationSystemGroup の実行時間を確認する。

10. Editor Analytics

1.12.0 系では、Netcode Profiler の利用パターンに関するエディターアナリティクスが収集される。

11. Related docs

- Optimizations and Debugging/03_Profiler ・ Bottleneck Analysis.md — DOTS の一般的なプロファイリングワークフロー。
 - 22_Netcode Ghost Optimization ・ Importance ・ Relevancy.md — 帯域の制御。
 - 23_Netcode Physics Integration & Lag Compensation.md — 予測物理のコスト。
-

12. Troubleshooting

症状	原因 / 対処
Netcode Profiler のデータがない	テストがクライアント / サーバーの Play Mode で動いているか、また Unity 6.0 以降を使っているかを確認する。
Ghost Snapshot View は大きいのに、 ゲームプレイ自体は問題ない	フィールドが多すぎたり送信レートが高すぎたりする、優先度の低い Ghost を探す。
予測エラーが急増する	Simulate のクエリ、決定論的なコードパス、 入力ティックの扱いを確認する。
Network Debugger のワークフローがない	すでに廃止されている。Unity Profiler の Netcode モジュールを使う。
パケットレベルのログを突き合わせに くい	1.6.2 以降の系で、 フレームカウント付きタイムスタンプのパケットダンプを使う。

Part III. Optimization and Debugging

Chapter 28. Chunk Layout & TypeManager

1. The 16 KB chunk

どの Archetype も、Entity を **16 KB のチャンク** に格納する。チャンクが持つのは次のものだ。

1. 小さなヘッダ（メタデータ。Archetype ID、バージョン番号、Enableable ビットマスク、変更バージョンなど）。
2. 連続した **SoA** 領域。Component 型ごとに 1 カラムで、各カラムは値の配列だ。
3. 16 KB の境界までのパディング。

チャンクの容量、つまり収まる Entity の数は、次の式で決まる。

```
capacity ≈ (16 KB - header) / sum(sizeof(component_i))
```

小さな Component が少しだけの引き締まった Archetype なら、数百の Entity を収められる。大きな Component を多数抱えた肥大した Archetype は、ほんの数体しか入らない。占有率の低いチャンクは、本来もっと多くの Entity を入れられたはずのバイトを無駄にしている。

2. TypeManager — the component registry

TypeManager は World の起動時に、すべての Component、バッファ、System を登録し、それぞれに安定した **型インデックス** を割り当てる。ランタイムは、このレジストリから次の情報を読み取る。

フィールド	用途
TypeIndex	クエリやチャンクのメタデータで使われる、パックされた識別子。
SizeInChunk	チャンク内で、この Component が Entity 1 体あたりに占めるバイト数。
Alignment	チャンク内でのカラムのアライメント。
ElementSize	バッファ要素のサイズ（IBufferElementData 用）。
HasBlobReferences / HasEntityReferences	シリアライズや移動の処理中に行う検証を有効にする。

TypeManager を直接いじることは、まずない。押さえておきたいのは、**Component 型を新しく追加したり、名前を変えたり、アセンブリ間で移したりすると、型レジストリの再ビルドが走る**という点だ。大規模プロジェクトの起動時間は、ここで食われる。

Entities 1.4 では、不要なアセンブリスキャンを省くことで、大規模プロジェクトでの TypeManager.Initialize が約 2 倍速くなったと報告された。6.5 にもこの最適化が引き継がれている。

3. Inspecting chunk layout — the Archetypes window

Window → Entities → Archetypes には、World 内のすべての Archetype が一覧表示される。各行に出る情報は次のとおり。

- **Entities** — 現在の Entity 数。
- **Chunks** — 使用中のチャンク数。
- **Entities / Chunk** — 平均の占有率。
- **Capacity** — チャンク 1 つあたりの理論上の上限。

注目したい読み方は次のとおり。

読み方	解釈
1 チャンクに Entity 1 体しかない Archetype が多数ある	タグを細かく分けすぎたことによる Archetype の急増。まとめる。
1 つの Archetype に Entity が多く、占有率も高い	健全な状態。敵や発射物、弾丸などがこれにあたる。
Entities / Chunk ≪ Capacity	断片化。構造変更が頻発し、Entity が Archetype 間を移動し続けている。占有がまばらになる。
Capacity = 1	Archetype が太りすぎている (Component が大きすぎる)。サイズの大きいマネージド Component や共有 Component を確認する。

4. Designing archetype-friendly components

効果の大きい順に、目安となる指針を挙げる。

1. **アンマネージドな構造体は、アンマネージドのまま保つ。** マネージドな `IData` が 1 つあるだけで、Entity ごとのサイドテーブル格納が強制され、Burst が使えなくなる。
2. **構造体にすき間を作らない。** `float3` は `float` の後ろではなく前に置く。アライメントの穴が `SizeInChunk` をふくらませる。
3. **めったに使わないフィールドは切り出す。** 1 つの System しか読まないフィールドは、別の Component に移す。使わないカラムも Entity あたりのバイト数を消費する。
4. **毎フレームの add / remove より Enableable Component を優先する。** `13_StructuralChange & Safety.md` を参照。ビットの切り替えのほうが、Archetype を組み替えるより有利だ。
5. **共有 Component の値の種類を絞る。** 新しい値が増えるたびに、新しいチャンクの区分が生まれる。
6. **タグ Component はゼロバイトだが、それでも Archetype を 1 つ増やす。** すべての Entity に固有のマーカを付けたりしないこと。

5. Chunk version numbers

各 Component のカラムは **変更バージョン** を持つ。これは、System が `RefRW<T>.ValueRW`、`SetComponentData`、ECB の `SetComponent` を介して書き込むたびに 1 つ増える、32 ビットのカウンタだ。

System は、前回の実行からバージョンが変わっていないチャンクをスキップできる。

```
var query = state.GetEntityQuery(ComponentType.ReadWrite<Health>());
query.SetChangedVersionFilter(typeof(Health));
```

変更フィルターを付けてクエリを反復処理すると、System の前回実行以降に更新されたチャンクだけが対象になる。「Health が変わったら UI を更新する」といったパターンを低コストに実現できるのは、このおかげだ。

注意点は次のとおり。

- フィルターは Entity 単位ではなく **チャンク単位** で働く。1 体の Entity への書き込みが、チャンク全体をダーティにする。
- `RefRW<T>.ValueRW` 経由で読み取ると、実際には書き換えていなくてもバージョンが上がる。
- Enableable Component を切り替えても、バージョンが上がる。

6. BlobAsset storage

`BlobAssetReference<T>` は、大きな読み取り専用データ（メッシュ、LOD テーブル、ナビゲーションフィールドなど）をチャンクの**外**に置く。チャンクに入るのはポインタだけだ。これにより、多数の Entity が同じベイク済みデータを共有しても、チャンクの容量がふくらまずに済む。

実践的なヒント。ある種類の Entity すべてで中身が同じになる `FixedList` や `DynamicBuffer` が Component の大半を占めているなら、その大きなペイロードを Blob アセットに移し、軽量の Component からそれを参照させる。

7. Examples

7.1 A fat archetype to avoid

```
public struct EnemyState : IComponentData
{
    public float3 Position;
    public quaternion Rotation;
    public float Health;
    public float HealthMax;
    public float Stamina;
    public float StaminaMax;
    public int AIState;
    public float3 LastKnownPlayerPos;
    public float TargetDistance;
    public Entity Target;
    public FixedList512Bytes<float3> PathBuffer;    // 512 bytes per entity!
}
```

`PathBuffer` フィールド 1 つだけで、Entity あたり 512 バイトを食う。16 KB チャンクの容量は、二桁台前半まで落ち込む。これを分割する。

```
public struct EnemyStats : IComponentData
{
    public float Health, HealthMax, Stamina, StaminaMax;
    public int AIState;
    public Entity Target;
}

public struct EnemyPathPoint : IBufferElementData
{
    public float3 Value;
}
```

これで `DynamicBuffer` は専用の領域に置かれ、チャンクの密度が戻る。

7.2 Change filter for cheap UI updates

```
var query = state.GetEntityQuery(ComponentType.ReadWrite<Health>(),
                                ComponentType.ReadWrite<HealthBarLink>());
query.SetChangedVersionFilter(typeof(Health));

state.Dependency = new UpdateHealthBarJob { /* ... */ }
    .ScheduleParallel(query, state.Dependency);
```

これで、Health が変わったチャンクだけが処理対象になる。

8. Troubleshooting

症状	原因 / 対処
大規模プロジェクトでの起動が遅い	TypeManager.Initialize がスキャンするアセンブリが多すぎる。Profiler の起動トレースを確認し、新しい Entities のバージョンを使っているか確かめる。
Archetype の容量が 1 になっている	Component が大きすぎる（バッファ容量や、埋め込んだ Blob など）。かさばるデータは BlobAssetReference<T> か IBufferElementData に移す。
「何も変わっていない」のに変更フィルターが毎フレーム発火する	どこかで、書き込まずに RefRW<T>.ValueRW で読み取っている。可能なら RefRO<T> に切り替える。
Archetype の数が増え続ける	値の種類が多い Entity 別タグか、連続値をキーにした共有 Component が原因。まとめる。
Entities / Chunk が Capacity を大きく下回る	構造変更の頻発。add / remove が Entity を Archetype 間にばらまいている。Enableable Component に切り替える。
マネージド Component が CPU プロファイルの大半を占める	マネージドフィールドを UnityObjectRef<T> に移すか、Component を分割する。アンマネージドの経路のほうがずっと速い。

Chapter 29. Systems · Entity Inspector · Query Window

1. Overview

日常的に使う Entities のツールウィンドウは、次の 3 つだ。

ウィンドウ	開く方法	答えてくれる問い
Entities Hierarchy	Window → Entities → Hierarchy	「どんな Entity が存在するか？どんな Component を持つか？」
Systems	Window → Entities → Systems	「どの System が、どんな順番で、どれくらいの時間動いているか？」
Query	Window → Entities → Query	「このコンポーネント構成には、どの Entity がマッチするか？どの System がそれをクエリしているか？」

4 つ目の **Archetypes** (Window → Entities → Archetypes) は 01_Chunk Layout & TypeManager.md で扱う。

2. Entities Hierarchy

通常の Hierarchy の Entity 版にあたるもので、リアルタイムに状態を映す。

World picker (top of window)

DefaultGameObjectInjectionWorld (シングルプレイヤー)、ClientWorld / ServerWorld (Netcode)、テストフィクスチャの World を切り替えられる。World を変えると、その World の Entity に対してツリーが組み直される。

Tree view

- 各行が 1 つの Entity を表す。子の行は、Parent や LinkedEntityGroup でつながった Entity だ。
- 右側のカラムには、Component 数と子 Entity 数が出る。
- 行をクリックすると、その Component が **Inspector** に表示される。

Search

- `c:Health` — Health Component を持つ Entity に絞り込む。
- `n:Enemy` — 名前で絞り込む。
- `w:Server` — World で絞り込む。
- 組み合わせもできる。`c:Health c:Enemy n:boss` のように書く。

Inspector

Entity の Inspector には次が表示される。

- **Components** セクション — 各 `IComponentData` とその現在値。Play モードならその場で編集できる。
- **Relationships** — ほかの Entity へのリンク（Parent、Child、LinkedEntityGroup）。
- **Enabled state** — `IEnableableComponent` ごとの ON/OFF トグル。

その場での編集は **ライトスルー**だ。フィールドを変えると、そのチャンクの変更バージョンが即座に更新される。

3. Systems

Systems ウィンドウには、登録済みの System がすべて、`SystemGroup` ごとにまとめて、スケジューラが実際に動かす順番で並ぶ。

Columns to read first

カラム	意味
Name	System の型。
Namespace	System が属する場所。パッケージか自前のコードかの判別によく使う。
Time (ms)	OnUpdate に費やしたメインスレッド時間。ほぼ毎フレーム更新される。
Queries	System がアクセスしたクエリ数。
Entity Count	プライマリクエリにマッチした Entity 数。

What to look for

読み方	解釈
System が 0 ms だが緑色	<code>RequireForUpdate<T></code> または <code>[RequireMatchingQueriesForUpdate]</code> によりスキップされている。正常な状態だ。

読み方	解釈
System が常に高い ms を示す	実際の処理が行われている。詳細は Profiler で確認する。
System の Entity 数が -- を示す	マッチするクエリがない（またはクエリがまだビルドされていない）。
2つの System が予期した順序と異なる	[UpdateBefore] / [UpdateAfter] のチェーンが間違っているか、異なるグループにある。

Enable / disable

System にはそれぞれチェックボックスが付いている。外せば、再コンパイルなしで Play モード中にその System をスキップできる。A/B テストに便利だ。

System detail pane

System を選ぶとサイドペインが開き、次の内容が表示される。

- **Queries** — 各クエリのコンポーネント構成（All / Any / None / Disabled / Present / Absent）。
- **Dependencies** — この System が [UpdateBefore] / [UpdateAfter] で指定した System。実際のスケジュールに反映された結果が出る。
- **Type Info** — クラスと各メソッドに [BurstCompile] が付いているかどうか。

1.4 以降 / 6.x では、**Queries タブに Disabled / Present / Absent / None のコンポーネント状態が表示される**ようになった。これで Enableable なクエリが曖昧に見えることはなくなった。

4. Query window

「X にマッチするのはどの Entity か」を調べるための専用ツールだ。

使い方は次のとおり。

1. プラスボタンで **All** のコンポーネントを追加する。
2. 必要なら **Any / None** も追加する。
3. ウィンドウの左パネルに **マッチする Entity**、右パネルに **このクエリを使う System** が一覧表示される。

使いどころを挙げる。

- 「どの System が Health を読んでいるか」を知りたい → All に Health を追加する。右側に並ぶ System はすべて Health を扱っている。

- 「この Entity が System のクエリにマッチしないのはなぜか」を調べたい → System のクエリを貼り付け、Hierarchy ウィンドウの Entity と、コンポーネントの有無を見比べる。
- 「Prefab がこのクエリに紛れ込んでいないか」を確かめたい → Prefab は結果リスト上でアイコンによって見分けられる（1.4 以降で追加）。

Query ウィンドウでは、クエリを保存して、そのセッション中に再利用することもできる。

5. Journaling — the last-resort diagnostic

Window → Analysis → Entities Journaling（Project Settings → Entities → Journaling で有効にする）は、すべての構造変更、コンポーネントへの書き込み、System の更新をリングバッファに記録する。再生用のビューは次のとおり。

- **Operations** — 記録された各変更を、タイムスタンプ・発生元の System・影響を受けた Entity とともに表示する。
- **Entities** — Entity ごとに、いつ何がそれに触れたかをタイムラインで表示する。

Journaling にはランタイムコストがかかる。バグを調べているときだけ有効にし、ふだんの開発では切っておく。

6. Field workflow — debugging a missing update

「System は Entity を処理したのに、コンポーネントが変わらない」というのはよくあるバグだ。調査の手順を示す。

1. **Entities Hierarchy** で Entity を探し、目的のコンポーネントを持っているか確認する。
 2. **Inspector** で、Play 前に値を控えておく。Play を押し、その値の変化を見る。
 3. 値が変わらないなら、**Systems ウィンドウ**で該当の System を探し、**Time (ms)** を見る。0 ならスキップされている。
 4. 時間が 0 より大きいのに値が変わらないなら、**Query ウィンドウ**に System のクエリと Entity のコンポーネントを貼り付け、Entity がマッチするか確かめる。
 5. それでも解決しないなら、**Journaling** を 1~2 フレームだけ有効にする。そのコンポーネントに対して、ほかの System が上書きしていないかを探す。
-

7. Troubleshooting

症状	原因 / 対処
Play モードで Entities Hierarchy が空	World ピッカーが間違った World に設定されている（例: Client のみ、Server なし）。
ライブフィールドが更新されない	ドメインリロード後にキャッシュされた Entity を表示している。再選択する。
Systems ウィンドウが 0 ms と表示されるが確実に実行されている	実際の処理が <code>ScheduleParallel</code> にある場合、Profiler ではチャンク Job のマーカー下に表示されることがある。その場合、メインスレッドの <code>OnUpdate</code> は実際には 0 になる。Profiler で Job のタイミングを確認する。
Query ウィンドウが System にマッチしない Entity を見つける	System が Query ウィンドウに伝えていない <code>WithAll<T>()</code> や変更フィルターを持っている。正確に一致させる。
Journaling ウィンドウが空	Project Settings で Journaling が有効になっていないか、バッファがロールオーバーした。
Entity と GameObject が同期していないように見える	SubScene が開いている（白アイコン） — 閉じる（黄アイコン）と Baker が実行され Entity が表示される。

Chapter 30. Profiler · Bottleneck Analysis

1. Overview

DOTS で「なぜこのフレームが遅いのか」を突き止める主役は、Unity Profiler (Window → Analysis → Profiler) だ。Entities は十分な数のマーカーを出力するので、どこを見ればよいか分かっている、特定の System や Job、チャンクに処理時間を割り当てられる。

本章では、プロファイルの読み方、DOTS でありがちなボトルネックのパターン、そしてその対処法を見ていく。

2. Before you profile

1. **Deep Profile は使いどころを絞る。** 特定のマネージドな経路を追うには役立つが、タイミングが歪む。性能の数値を見るときに Deep Profile を当てにしてはいけない。
 2. **「Record in Editor & Standalone」を有効にする。** こうすれば Editor のフレームだけでなく、ビルドも計測できる。
 3. **製品版の性能を測るときは、Profiler の対象を Editor ではなくプレイヤーにする。** Editor のオーバーヘッドは無視できない。
 4. **読み取る前に数秒プレイする。** 最初の数フレームには、ドメインリロードや JIT の処理が含まれる。
-

3. Reading the CPU Usage module

DOTS フレームの直下に並ぶ、トップレベルのマーカーは次のようになる。

```
PlayerLoop
├─ Initialization.*
├─ Update.ScriptRunBehaviourUpdate           (MonoBehaviour.Update)
├─ SimulationSystemGroup                    ← DOTS simulation lives here
│   ├── SystemA.OnUpdate
│   ├── SystemB.OnUpdate
│   └─ EndSimulationEntityCommandBufferSystem.Playback
├─ FixedStepSimulationSystemGroup
├─ PresentationSystemGroup
├─ Render
└─ WaitForTargetFPS
```

`SystemA.OnUpdate` のマーカーを掘り下げると、その下の呼び出し（`ScheduleParallel`、`Job` の完了待ち、`EntityManager` の呼び出しなど）が見える。

4. Where the time actually goes — common shapes

4.1 Main-thread OnUpdate dominant

1 つの `SystemX.OnUpdate` が 10 ms 以上を占めているなら、その処理はメインスレッドで走っている。次の点を確認する。

- 本来 `IJobEntity.ScheduleParallel` にすべき密なループで、`SystemAPI.Query` を使っていないか。
- マネージドフィールドのせいでメインスレッドに縛られた `SystemBase` になっていないか。
- `EntityManager.AddComponent / RemoveComponent` が `Entity` ごとに呼ばれていないか。

4.2 Job timings

`Job` は **Timeline** ビュー、または **Hierarchy** ビューの「Workers」の下に表示される。次の点を確認する。

- あるジョブだけが埋まり、ほかが遊んでいる。これは処理がチャンク間に分散していないということだ。
- メインスレッドが `Job` を待っている（`JobHandle.Complete()` のマーカー）。`.Complete()` を呼ぶのが早すぎる。フレームの後ろへ動かす。
- `Burst` の関数名がやたら長い。これは正常で、シンボルがジェネリックの完全な実体化を表しているだけだ。

4.3 EntityManager hotspots

次の名前のマーカーに注目する。

- `EntityManager.SetArchetype / Move Entity` — 構造変更の多発。13_Structural Change & Safety.md を参照。
- `EntityCommandBufferSystem.OnUpdate (Playback)` — 重い ECB プレイバック。たいていは、1 フレームに記録するコマンドが多すぎることを示す。
- `TypeManager.Initialize` — 起動時にしか出ないはずのもの。毎フレーム出るなら、何かが `World` を再ロードしている。

4.4 GC.Alloc markers

シミュレーション中に `GC.Alloc` が出たら、それは DOTS としては劣化だ。原因としては次のものがある。

- `OnUpdate` の中でアロケーションを行う `SystemBase`。
- `System` の中で、マネージドなコレクションを `foreach` で回している。
- ボクシング。構造体に対する `object.ToString()` など。

Burst の `System` なら、Profiler のメモリモジュールでアロケーションがゼロになっているはずだ。

5. Profiler modules to pin

- **CPU Usage** — マーカーのタイムラインと階層。
- **Memory** — 確保されたヒープ。 `GC.Used Memory` がじわじわ増えていないか見る。
- **Entities** (使える場合) — `System` ごとの処理時間を時系列で表示する、DOTS 専用のモジュール。

Entities Profiler モジュールは 1.8.0 / 6.x で導入され、タイムライン上で `Systems` ウィンドウと対応づけられる。複数フレームにわたるスパイクが見えたら、その原因を突き止めるいちばんの近道がこれだ。

ネットワーク対応のプロジェクトでは、Ghost Snapshot、予測ティック、Netcode 固有の帯域を調べるために、`../DOTS Workflows/24_Netcode Profiler & Debugging.md` と併せて使う。

6. Common bottlenecks and fixes

パターン	Profiler での症状	対処
Entity ごとの <code>Entities.ForEach</code> (レガシー)	小さなサンプルが多い、メインスレッドが重い <code>OnUpdate</code>	<code>IJobEntity.ScheduleParallel</code> に移行する。
フレームごとの <code>AddComponent/RemoveComponent</code>	<code>Move Entity</code> マーカーが支配的	状態を <code>Enableable Component</code> に変換する。
ECB の過負荷	<code>...ECBS.Playback</code> が ms を占める	コマンドをバッチ処理する (Entity ごとではなくチャンクごとに1回) か、バッチサイズを大きくする。

パターン	Profiler での症状	対処
共有 Component の値が爆発する	SetSharedComponent が支配的	値をバケット化する (より少ない個別キーに量子化する)。
.Complete() をフレーム中に呼ぶ	メインスレッドの JobHandle.Complete 待機が長い	完了をフレームの末尾まで延期するか、 メインスレッドがまだ結果を必要としないよう再構成 する。
Burst フォールバック	ホットメソッドがアンマングル された C# 名を示す	キャプチャされた非 blittable 型が IL にフォールバックした。Burst Inspector で確認する。
Archetype のフラグメンテーション	小さなチャンクが多数。 EntityManager.*Query マーカーが高い	Archetype が細かすぎる — タグを統合するか Enableable Component を使用する。
Bake 時のリグレーション	Play 中に Bake マーカーが表示される	ドメインリロードが Baking を再トリガーした。通常 Editor のみ。ビルドで確認する。

7. Frame budget worked example

16.6 ms の予算 (60 Hz) を想定する。Render を除いて見ていく。

- SimulationSystemGroup: 6 ms
 - EnemyAISystem.OnUpdate: 2.1 ms (Burst、4 ワーカーで並列 Job)
 - MovementSystem.OnUpdate: 1.4 ms (Burst、並列)
 - EndSimulationECBS.Playback: 1.8 ms ← 要注意
 - その他合計: 0.7 ms

構造変更はわずかなはずなのに、ECB プレイバックに **1.8 ms** かかっている。たいていは、ここがいちばん効く改善どころだ。コマンド数を確認する。

- Entities Journaling を 1 フレームぶん取り、System ごとの Instantiate / AddComponent / DestroyEntity の回数を見る。
- 毎フレーム数百のコマンドを記録している System があれば、まとめて処理する (Entity ごとではなくチャンクごとに 1 コマンドにするか、チャンクインデックスをソートキーにした並列レコーダーに切り替える)。

8. Standalone Profiling

実際のビルドを計測する手順は次のとおり。

1. **File → Build Profiles → Development Build + Autoconnect Profiler** を選ぶ。
2. プレイヤーが起動し、Profiler がリモートのプロセスから計測する。
3. こうして得た結果は、実際にプレイヤーが体験する性能をより正直に映す。Editor での計測は、コードとは無関係な理由で 2～5 倍遅くなることがある。

9. Troubleshooting

症状	原因 / 対処
プロファイルに DOTS マーカーが表示されない	Profiler モジュールがそれらを除外している。モジュール設定で「Scripts」と「Jobs」を有効にする。
フレームは遅いのに、目立って重いマーカーがない	レンダースレッドの待ちかもしれない。GPU モジュールとタイムラインビューを有効にする。
Deep Profile が異なるホットスポットを示す	Deep Profile はアロケーションや呼び出し回数を歪める。相対的なタイミングは、Deep Profile を使わないプロファイルで判断する。
Burst メソッドのスタックにマネージドな呼び出しが出る	フォールバックだ。Burst がコンパイルできなかった。Burst Inspector の出力を確認する。
GC.Collect のマーカーがスパイクする	System のどこかでマネージドなアロケーションが起きている。メモリモジュールの Snapshot で絞り込む。
System の処理時間がフレームごとに大きく揺れる	処理量が Entity 数に比例して増え、毎フレーム、チャンクがクエリに出入りしている。構造変更の多発を確認する。

Chapter 31. Managed Object Reference Audit

1. Overview

マネージドな Unity オブジェクト参照は、ECS System をうっかり高速パスから外してしまう、もっとも起こりやすい原因の 1 つだ。この監査は、コンポーネントデータや System、レンダリングのつなぎ込みコードが、Burst や Job、きれいな SubScene ベイクを妨げる形で `UnityEngine.Object` 参照を抱えている箇所を見つけ出すのに役立つ。

コアの ECS 移行を終えた後や、データ指向のコードを期待していた場所でプロファイラーがマネージドな処理を示したときに、この監査を行う。

2. What to search for

パターン	重要な理由
<code>class .*: IComponentData</code>	マネージドコンポーネントデータ。有効だが通常ホットパスには適さない。
<code>UnityEngine.Object</code>	マネージドされた Unity オブジェクト参照の広範なマーカー。
<code>GameObject</code> 、 <code>Transform</code> 、 <code>Component</code>	ランタイムのシーンオブジェクト参照は通常シミュレーションデータには属さない。
<code>Mesh</code> 、 <code>Material</code> 、 <code>Texture</code> 、 <code>AudioClip</code>	多くの場合 <code>UnityObjectRef<T></code> またはプレゼンテーション側でキャッシュされたデータにするべきだ。
<code>Resources.Load</code>	ランタイムのアセット検索。Baking / コンテンツシステムで処理する方が多い場合が多い。
<code>ScriptableObject</code>	有用な Authoring ソースだが、ランタイム Job はベイクされた値や Blob を読み取るべきだ。

監査の対象は、ECS に関わるコードに絞る。MonoBehaviour だけで完結する UI やエディターツールを、無理に DOTS のパターンに合わせる必要はない。

3. Triage rules

発見事項	推奨アクション
負荷の高いシミュレーションクエリにマネージド Component がある	アンマネージドな IComponentData、BlobAssetReference<T>、UnityObjectRef<T> のいずれかに変換する。
プレゼンテーションでしか使わないマネージド Component	Burst や Job のシミュレーション経路から切り離されているなら、そのままでよい。
毎フレーム読まれている ScriptableObject	必要な値を、Component か Blob アセットにベイクする。
ECS データにメッシュやマテリアルの参照がある	UnityObjectRef<Mesh> / UnityObjectRef<Material> を使い、参照の解決はプレゼンテーションコードで行う。
コンポーネントデータに GameObject や Transform が格納されている	ECS データに置き換えるか、その参照をハイブリッドなプレゼンテーションブリッジへ移す。

4. Audit example

```
// Suspicious: managed component data used as per-entity visual state.
public class EnemyVisual : IComponentData
{
    public Material Material;
}
```

次のように書くのが望ましい。

```
using Unity.Entities;
using UnityEngine;

public struct EnemyVisual : IComponentData
{
    public UnityObjectRef<Material> Material;
}
```

そのうえで、メインスレッドのプレゼンテーション System で参照を解決するか、解決済みのマテリアルをレンダラーのレジストリにキャッシュする。

5. Profiler signs

Profiler の症状	確認事項
シミュレーション System が毎フレームアロケーションしている	OnUpdate 内でのマネージドオブジェクトの検索や、LINQ ・コレクションのアロケーション。

Profiler の症状	確認事項
データだけで完結するはずの System で Burst が効いていない	クエリの経路にマネージドコンポーネントがあるか、マネージド API にアクセスしている。
メインスレッドに、小さなレンダリング用の検索が多数ある	UnityEngine.ObjectRef<T>.Value の結果を、Entity ごとのループの外でキャッシュする。
SubScene の再ベイクが想定より頻繁に起きる	オーサリングスクリプトかオブジェクト参照が、不要な依存関係を生んでいる。

6. Validation

- §2 の検索パターンをもう一度実行する。
- 負荷の高いシミュレーション System が、アンマネージドな Component だけをクエリしているか確認する。
- Burst でコンパイルされるはずの System や Job について、Burst Inspector を確認する。
- 代表的なシーンをプロファイルし、ホットパスからマネージドなアロケーションが消えたことを確認する。
- Editor をまっさらな状態で起動して Play モードに入り、SubScene のベイクが参照を解決できることを確認する。

7. Troubleshooting

症状	原因 / 対処
変換した Component が、まだ Burst でクエリできない	残っているフィールドがマネージドだ。ネストした構造体も確認する。
UnityEngine.ObjectRef<T>.Value が Play モードで null を返す	参照先のアセットがベイクされていないか、ロードされていない。Baker と、SubScene やコンテンツワークフローを確認する。
変換後も、表示まわりの System が遅い	毎フレーム、Entity ごとに参照を解決している。 共有アセット単位でキャッシュするか、 小さなルックアップキーをベイクしておく。
ScriptableObject を変更してもランタイムデータに反映されない	値はベイク済みだ。SubScene を再インポートまたは再ベイクするか、適切なベイク依存関係を追加する。
Editor 専用の参照が、プレイヤービルドで失敗する	エディター API は #if UNITY_EDITOR の中に入れ、ランタイムでも安全なデータをベイクしておく。

Part IV. Migration

Chapter 32. Entities 1.x → 6.5 Overview

1. What you're signing up for

Entities 1.x から 6.5 へのアップグレードは、パッケージ配布形態の移行を土台に、**3 つの DOTS 移行**が重なったものと考えるとよい。

1. **Editor とパッケージ配布のアップグレード**。Unity 6000.4 以降 (`com.unity.entities` 6.4) か 6000.5 以降 (Entities 6.5) へ移行する。Entities は Editor に組み込まれた Core Package になる。
2. **ECS データの整理**。ECS に関わるコードのマネージドな Unity オブジェクト参照を洗い出し、ホットパスのコンポーネントデータを、アンマネージドコンポーネント、ベイク済みの値、`BlobAssetReference<T>`、`UnityObjectRef<T>` へ移していく。
3. **ECS API の移行**。Entities 1.4 で非推奨になった旧 API (`Entities.ForEach`、`IAAspect`) は、6.5 でも非推奨のまま。あわせて `ComponentLookup.GetRefRWOptional` → `TryGetRefRW` の移行も必要になる。

それぞれの移行は、このフォルダ内の個別の文書で扱う。本章はその **見取り図** にあたり、何を、いつ、どの順番でやるかを示す。

バージョン表記の注意：Entities パッケージのバージョン (1.4 / 6.4 / 6.5) は、Unity Editor のバージョン (6000.3 / 6000.4 / 6000.5) とは別物だ。6.4 以降は慣例的に対応するが、リリースノートは別々に管理される。本書ではこれを 2 つの軸として扱う。Changelog のページ (`../Changelog/Entities 1.4 → 6.5 Key Changes.md`) も同じだ。

2. Recommended order

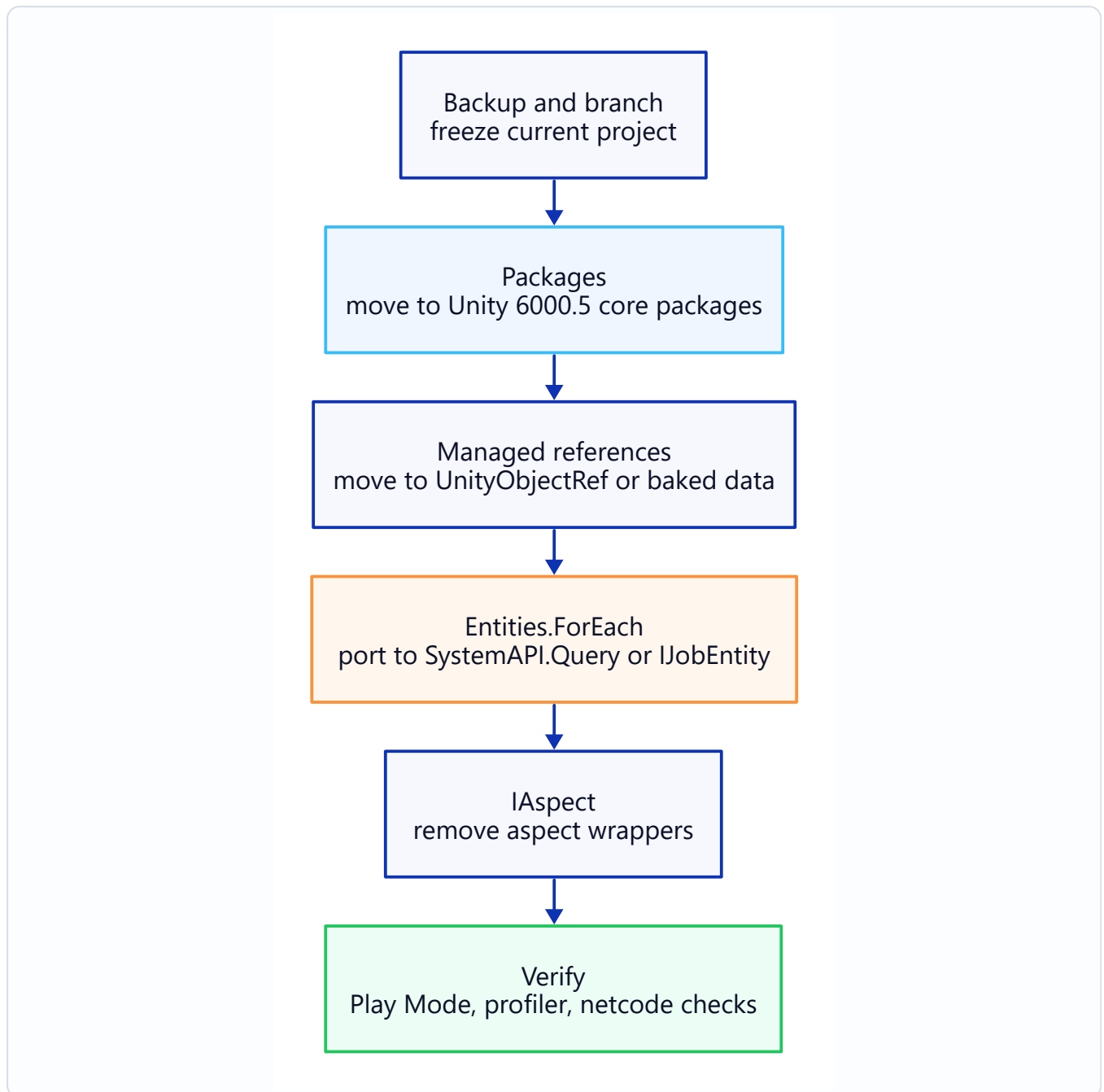


図: Entities 1.x から 6.5 への移行順序。

この順番にする理由は次のとおり。

- **まず Editor。** Editor が 6.x の API を認識するまで、ほかのコードはその API に対してコンパイルできない。
- **次に Package Manager → Core Package。** これを済ませないと、パッケージ解決が重複したり競合したりする。
- **反復処理のリファクタより前に、マネージド参照を整理する。** System や Job を書き直す前に、データモデルをはっきりさせておく。

- **foreach / IAspect は最後。** これらは ECS 内部の話で、コードベースのほかの部分の足を引っ張らない。

3. What stays the same

変わらない部分は意外なほど多い。移行が見た目ほど怖くないのは、このためだ。

変わらないもの	備考
IComponentData、IBufferElementData、ISharedComponentData	インターフェースも意味づけも同じ。
ISystem / SystemBase	ライフサイクルフックも属性も同じ。
EntityManager の API	CreateEntity や AddComponent など、まったく同じ。
SystemAPI.Query	1.x で導入され、引き続き中心的な反復処理 API であり続ける。
IJobEntity、IJobChunk	形は同じ。基本は IJobEntity が推奨される。
EntityCommandBuffer + ECBS	同一。
Burst、Jobs、Collections、Mathematics	同一。
Netcode for Entities のコンセプト（Ghost・RPC・Prediction）	アーキテクチャは同じ。 違うのはパッケージの配布方法だけだ。

1.x のコードがすでに（旧来の Entities.ForEach ではなく）IJobEntity と SystemAPI.Query を使っているなら、移行のステップ 4 と 5 はほとんどやることがなくなる。

4. What changes, at a glance

エリア	1.x	6.5
配布	com.unity.entities UPM パッケージ	Core Package（manifest エントリなし）
バージョン番号	1.4.x	6.5.x（Unity 6000.5 と一致）
ECS データ内のマネージド Unity オブジェクト参照	多くはマネージドコンポーネント に格納	適切な場面では、ベイク済みデータ、Blob、 UnityObjectRef<T> を優先
Component の反復処理（旧来）	Entities.ForEach が 1.4 で非推奨に	引き続き非推奨。 警告付きでコンパイルは通る。 将来のメジャーリリースで削除予定

エリア	1.x	6.5
Aspect	IAspect が 1.4 で非推奨に	引き続き非推奨。コンパイルは通る。 将来のメジャーリリースで削除予定
オプション参照	GetRefRWOptional / GetRefR0Optional が非推奨に	TryGetRefRW / TryGetRefR0

この期間の変更履歴の全体は `../Changelog/Entities 1.4 → 6.5 Key Changes.md` にまとまっている。

5. Backup strategy

作業を始める前に、次のことは必ずやっておく。

1. **git でブランチを切る。** 内容が分かる名前にする (`migrate/entities-6.5` など)。main で移行作業をしてはいけない。
2. **移行前の状態の manifest.json と packages-lock.json をコミットしておく。** パッケージ解決が壊れたときに戻すための、信頼できる基準になる。
3. **移行前のコミットにタグを付ける。** `git tag pre-entities-6.5` としておけば、戻すのが簡単になる。
4. **準備が整うまでは、古い Editor バージョンでプロジェクトを開いたままにしておく。** 6000.5 で一度でも開くと `ProjectVersion.txt` が書き換わり、後戻りできない移行ルートに乗ってしまう。

6. Per-step checklists

6.1 Editor upgrade

- Hub から Unity 6000.5.x をインストールする。
- まだプロジェクトは開かない。先に、未コミットの作業をコミットしておく。
- 新しい Editor でプロジェクトを開く。ProjectVersion.txt のアップグレードを提案されるので、承認する。
- 最初のインポートが終わるまで待つ。非推奨 API やパッケージ不足の警告が出るはずだ。それらは、次以降のステップで直していく対象だ。

6.2 Package cleanup → 02_Package Manager → Core Package.md

- `manifest.json` から `com.unity.entities`、`com.unity.collections`、`com.unity.mathematics`、`com.unity.entities.graphics` を削除する。
- `packages-lock.json` を削除し、Editor に再生成させる。
- Netcode for Entities を使っているプロジェクトなら、それも削除する（6.5 は Core Package 版だ）。

6.3 Managed object references → 03_Managed Object References → `UnityObjectRef.md`

- ECS に関わるコードの、マネージドな Unity オブジェクトフィールドを洗い出す。
- 可能なら、ホットパスのコンポーネントデータをアンマネージドな ECS データに変換する。
- Unity のアセットやオブジェクトを ECS 側から参照するには `UnityObjectRef<T>` を使う。
- 大きくて変化しないゲームプレイデータには `BlobAssetReference<T>` を使う。

6.4 Legacy ForEach → 04_foreach → `IJobEntity.md`

- `Entities.ForEach` や `Job.WithCode` を使っている System は、`IJobEntity` と `SystemAPI.Query` に移行する。

6.5 Aspect removal → 05_IAspect Removal.md

- Aspect のメソッドを、`RefRW<T>` / `RefRO<T>` のパラメータを使う形で呼び出し元にインライン化する。
 - `IAspect` の構造体を削除する。
-

7. Testing after migration

移行完了とみなす前に行うスモークテストは次のとおり。

1. 代表的な **SubScene** で **Play モード** に入る。Entity が存在し、System が動き、起動時に例外が出ないことを確認する。
2. **スタンドアロンプレイヤーをビルド**する。ビルドでしか起きない不具合（多くはシリアライズ関連）は、ここで見つかる。
3. **既存のゲームプレイ回帰テストを走らせる**。性能が重要なら、移行前にベースラインの Profiler トレースを取っておき、突き合わせる。
4. **セーブ / ロードを確認**する。生の Entity ハンドルは World 内でしか有効でなく、安定しない。保存するのは、ゲーム側の ID、コンテンツキー、オーサリングしたデータにする。
5. **対象プラットフォームで実行**する。プラットフォーム固有のバックエンド（IL2CPP の癖、AOT など）の問題は、ここで表面化することがある。

8. When to roll back

きっぱりロールバックすべきなのは、次のような場合だ。

- Editor がプロジェクトをまったく開けない（まれ。たいていは **ProjectVersion.txt** の不一致）。
- **packages-lock.json** の解決が意味の通らないグラフを生成し、削除しても直らない。
- 依存している重要なサードパーティ製パッケージが、まだ 6.5 に対応していない。

手順は、**git checkout pre-entities-6.5** を実行し、古い Unity バージョンを入れ直し、旧バージョンでゲームが動くことを確認する。

途中まで進んで行き詰まった移行は、きれいに中止してタグから再開する。中途半端に移行したコードをリリースしてはいけない。

9. Troubleshooting

症状	原因 / 対処
Editor が開かない — 「Project was saved in a newer version」	過去に誰かが 6000.5 で開いている。ProjectSettings/ProjectVersion.txt を、以前のバージョン文字列に戻す。

症状	原因 / 対処
manifest を整理した後に 「Unity.Entities assembly not found」 のコンパイルエラーが出る	Editor が 6000.4 より古く、Core Package を使えない。Editor をアップグレードするか、UPM パッケージを戻す。
非推奨の警告が何百も出る	想定どおりだ。これを移行の作業リストとして使う。警告 1 つ 1 つが、直すべき箇所を指している。
ビルドは成功するが、 ゲームが起動時にクラッシュする	たいていは、Component の削除によって生じた ECB かシリアライズの問題だ。ベイク済みの EntityScene アセットのサイズを比較する。大きく縮んでいたら、Component がひそかに落とされたサインだ。
移行後に性能が落ちる	たいていは、不要なはずの SystemBase が混入したか（マネージドフィールドのため）、IJobEntity が Burst からフォールバックしている。Profiler と Burst Inspector を確認する。

Chapter 33. Package Manager → Core Package

1. What changes

Unity 6000.4 以降では、Entities（と、その仲間の DOTS パッケージ）は **Core Package** になる。これらはリモートレジストリからダウンロードするのではなく Editor 自体からインストールされ、そのバージョンは Editor によって固定される。それでも **Package Manager から追加することになり**はなく、プロジェクトの依存関係としてリストに載りつづける。Core Package になっても変わるのはパッケージがどこから来るのか、誰がそのバージョンを決めるのかであって、プロジェクトがそれを要求しなければならないかどうかではない。1.x プロジェクトを移行するときに変わるのは次のとおり。

- **Packages/manifest.json** — `com.unity.entities*` のエントリは**残る**が、固定された **1.x** バージョンは Editor の Core バージョンに置き換わる（Package Manager から追加し直す）。
- **Packages/packages-lock.json** — 対象のパッケージは `"source": "builtin"` として解決されるようになる。
- **Package Manager ウィンドウ** — パッケージは引き続き「In Project」に並ぶ。ただしバージョンを選ぶことはできなくなる（Editor に従う）。

C# のコードは変わらない。アセンブリ名（`Unity.Entities`、`Unity.Collections` など）は同じで、`.asmdef` の参照にも手を入れる必要はない。

2. Which packages move

6000.4 以降で **Core Package** に移行したものは次のとおり。

UPM id（移行前）	移行後
<code>com.unity.entities</code>	Core Package。Editor に同梱される
<code>com.unity.collections</code>	Core Package
<code>com.unity.mathematics</code>	Core Package
<code>com.unity.entities.graphics</code>	Core Package
<code>com.unity.netcode</code> （Netcode for Entities）	6.5 で Core Package になった

Package Manager のまま：

- `com.unity.physics` (Unity Physics)
- `com.unity.charactercontroller` (Character Controller)

エンジンに組み込み済み（変更なし）のものは次のとおり。

- Jobs system
- Burst（環境によっては `com.unity.burst` が UPM エントリとして見えることもあるが、6000.4 以降では Editor のバージョンに紐づく。manifest はそのままにしておくのがいちばん安全だ）。

3. Editing manifest.json

`Packages/manifest.json` を開く。次のような内容になっている。

```
{
  "dependencies": {
    "com.unity.entities": "1.4.2",
    "com.unity.collections": "2.4.5",
    "com.unity.mathematics": "1.3.2",
    "com.unity.entities.graphics": "1.4.2",
    "com.unity.netcode": "1.12.0",
    "com.unity.physics": "1.3.14",
    "com.unity.render-pipelines.universal": "17.0.4",
    ...
  }
}
```

6000.4 以降の Editor では、これら固定された **1.x** バージョンはもう解決できない。レジストリがこの Editor 向けには提供しなくなるからだ。行をただ削除しては**いけない**（依存関係が消えてしまい、Core Package は自動では追加し直されない）。そうではなく、各 Core Package を Editor のバージョンに向ける。方法は 2 つあり、どちらもパッケージを依存関係として残す。

推奨 — Package Manager から追加し直す。 各 Core Package について、古くなった 1.x の固定を取り除き、**Window → Package Manager → Unity Registry → Install** から追加し直す（または **+ → Add package by name...** で、たとえば `com.unity.entities` を指定する）。Package Manager が Editor の Core バージョンを `manifest.json` に書き込み、依存する Core Package も自動で取り込んでくれる。

手動で行う。 各 Core Package の 1.x バージョンを、Editor の Core バージョン（たとえば Unity 6000.5 では 6.5.0）に置き換える。

```
{
  "dependencies": {
    - "com.unity.entities": "1.4.2",
    + "com.unity.entities": "6.5.0",
    - "com.unity.entities.graphics": "1.4.2",
```

```
+   "com.unity.entities.graphics": "6.5.0",
-   "com.unity.netcode": "1.12.0",
+   "com.unity.netcode": "6.5.0",
    "com.unity.physics": "1.3.14",
    "com.unity.render-pipelines.universal": "17.0.4",
    ...
  }
}
```

`com.unity.collections` と `com.unity.mathematics` は `Entities` の依存関係として取り込まれるため、通常は自分で記載する必要はない。`com.unity.physics` をはじめ、まだレジストリ (UPM) パッケージであるものはそのままにしておく。

ファイルを保存する。

4. Rebuilding packages-lock.json

方法は 2 つある。

4.1 Editor に再生成させる

Editor を閉じ、`Packages/packages-lock.json` を削除して、もう一度開く。Unity が作り直してくれる。これがいちばん確実だ。

4.2 直接編集する

`packages-lock.json` を開き、上記のパッケージの古くなったエントリを更新する（削除して Editor に再解決させる）。Editor が次の起動時に整合性を取る。ロックファイルが複雑な場合は少し速いが、ミスが入り込みやすい。

どちらの方法でも、最終的なロックファイルには次のようなエントリが入る。

```
"com.unity.entities": {
  "version": "6.5.0",
  "depth": 0,
  "source": "builtin",
  "dependencies": {}
}
```

`"source": "builtin"` こそが、そのパッケージが Core Package（Editor から供給される）になった証だ。一方で `"depth": 0` は、それが `manifest.json` で宣言された **直接の依存関係** であることを示している。エントリがなくなったわけではない。

5. Verifying the result

1. **Package Manager ウィンドウ** (Window → Package Manager → 「In Project」) — Entities、Collections、Mathematics、Entities Graphics (使っていれば Netcode for Entities も) が並んでいて、そのバージョンが Editor によって固定されている (バージョンのドロップダウンがない) はずだ。
2. **Editor のメニュー** — Window → Entities → Hierarchy / Systems / Archetypes が存在し、動作する。
3. **コンパイル** — Play を押す。アセンブリ不足を訴えるコンパイルエラーなしに Play モードへ入れるはずだ。

6. Troubleshooting

症状	原因 / 対処
Editor をアップグレードした後に Unable to find package 'com.unity.entities@1.x.y' が出る	manifest がまだ 1.x のバージョンを固定している。Core バージョンに更新し (Package Manager から追加し直す)、続いて packages-lock.json を削除して Editor に再生成させる。
The type or namespace Unity.Entities could not be found	Entities がプロジェクトに追加されていない (Package Manager から追加する)。あるいは Editor が 6000.4 より古い (アップグレードする)。
Package Manager に、古い、 あるいは競合する Entities のバージョンが表示される	manifest.json に 1.x の固定が残っている。Core バージョンに更新し (Package Manager から追加し直す)、packages-lock.json を再生成する。
サードパーティ製アセットが com.unity.entities@1.x に固定しているため、 コンパイルできない	そのアセット側の更新が必要だ。更新を待つか、フォークするか、このプロジェクトは 1.x 系にとどめる。
Unity Physics が特定の Entities バージョンを要求して壊れる	Entities 6.5 に対応した Unity Physics のバージョンにアップグレードする。パッケージの変更履歴は docs.unity3d.com で確認する。
packages-lock.json の再生成が失敗する	ネットワークの問題か、固定されたプライベートレジストリが原因。Editor のログで、具体的な解決失敗の内容を確認する。
Netcode for Entities がまだ古い 1.x バージョンに解決される	6000.5 以降では Netcode は 6.5 の Core Package だ。com.unity.netcode を 6.5 のバージョンに更新し (Package Manager から追加し直す)、再生成する。

Chapter 34. Managed Object References → UnityObjectRef

1. Overview

古い ECS コードを移行するときによく出くわすのが、マネージドなオブジェクト参照がランタイムのコンポーネントデータに紛れ込んでいることだ。Entities 6.5 では、ゲームプレイのコンポーネントはできる限りアンマネージドのまま保ち、ECS データがメッシュ、マテリアル、テクスチャ、オーディオクリップ、オーサリングした ScriptableObject といった Unity オブジェクトを指す必要があるときは `UnityObjectRef<T>` を使う。

本章で扱うのは DOTS のコンポーネント設計だ。エンジン全体にまたがる Unity オブジェクト識別子 API は扱わない。

2. What to look for

マネージドな Unity オブジェクトを直接格納しているコンポーネントを探す。

パターン	確認すべき理由
<code>IComponentData</code> を実装する class コンポーネント	マネージドコンポーネントは使えるが、遅く、メインスレッドに縛られる。
コンポーネントデータ内の <code>UnityEngine.Object</code> フィールド	たいていは、マネージドコンポーネントか、不正なアンマネージドコンポーネント設計のサインだ。
<code>GameObject</code> ・ <code>Transform</code> ・ <code>Material</code> ・ <code>Mesh</code> ・ <code>Texture</code> ・ <code>AudioClip</code> フィールド	これがオーサリング、プレゼンテーション、 <code>UnityObjectRef<T></code> のどこに属すべきかを見極める。
Entity ごとに <code>Resources.Load</code> を呼ぶランタイム System	アセットの解決を、ベイク処理か、中央のコンテンツシステムに移す。

3. Before and after

Before — managed component

```
using Unity.Entities;  
using UnityEngine;
```

```
public class ProjectileVisual : IComponentData
{
    public Mesh Mesh;
    public Material Material;
}
```

これでも動作するが、Component がマネージドになってしまう。マネージドコンポーネントは Burst や Job の自由度を狭め、Entity ごとのゲームプレイデータには大抵向かない。

After — unmanaged component with UnityObjectRef<T>

```
using Unity.Entities;
using UnityEngine;

public struct ProjectileVisual : IComponentData
{
    public UnityObjectRef<Mesh> Mesh;
    public UnityObjectRef<Material> Material;
}
```

参照は一度だけベイクしておく。

```
using Unity.Entities;
using UnityEngine;

public class ProjectileVisualAuthoring : MonoBehaviour
{
    public Mesh Mesh;
    public Material Material;
}

public class ProjectileVisualBaker : Baker<ProjectileVisualAuthoring>
{
    public override void Bake(ProjectileVisualAuthoring authoring)
    {
        var entity = GetEntity(TransformUsageFlags.Dynamic);
        AddComponent(entity, new ProjectileVisual
        {
            Mesh = new UnityObjectRef<Mesh>(authoring.Mesh),
            Material = new UnityObjectRef<Material>(authoring.Material)
        });
    }
}
```

4. Runtime access pattern

シミュレーションの Job はあくまでデータ指向に徹する。Unity オブジェクトの逆参照は、マネージドな Unity API を呼べる場所だけで行う。

```

using Unity.Entities;
using UnityEngine;

public partial class ProjectileVisualUploadSystem : SystemBase
{
    protected override void OnUpdate()
    {
        foreach (var visual in SystemAPI.Query<RefRO<ProjectileVisual>>())
        {
            Mesh mesh = visual.ValueRO.Mesh.Value;
            Material material = visual.ValueRO.Material.Value;

            if (mesh == null || material == null)
                continue;

            // Upload to a rendering registry, material property block cache, etc.
        }
    }
}

```

描画量が多い場面では、Entity ごと・フレームごとの逆参照は避ける。解決済みのオブジェクトは、コンパクトな ECS 値をキーにしたプレゼンテーション側のレジストリにキャッシュしておく。

5. When not to use UnityObjectRef<T>

ケース	より良い選択
純粋なゲームプレイデータ	プリミティブ・math・Blob データを IComponentData に直接持たせる。
大きな読み取り専用テーブル	BlobAssetReference<T>。
同じ SubScene にベイクされる Entity Prefab	Entity の Prefab 参照。
コンテンツワークフローでストリーミングする Prefab	EntityPrefabReference。
Entity ごとの複雑なマネージド処理	シミュレーションパスの外に置くか、マネージドなプレゼンテーション System に切り出す。

6. Migration checklist

- ☐ Unity オブジェクト参照を持たせるためだけに使っているマネージドコンポーネントを洗い出す。
- ☐ アンマネージドコンポーネントで足りる箇所は、該当フィールドを `UnityObjectRef<T>` に置き換える。
- ☐ オブジェクトの代入処理を `Baker` に移す。
- ☐ `.Value` の逆参照を `Burst Job` の外に出す。
- ☐ 多くの `Entity` が同じアセットを共有するなら、逆参照済みのオブジェクトをキャッシュする。
- ☐ 大きく不変なゲームプレイデータは、ランタイムで `ScriptableObject` を読むのではなく `BlobAssetReference<T>` を使う。

7. Troubleshooting

症状	原因 / 対処
Component を Burst Job で使えない	まだマネージドのまま。Unity オブジェクトのフィールドを <code>UnityObjectRef<T></code> に変えるか、プレゼンテーション用データを切り出す。
.Value が null になる	参照先のアセットが未ロードか、破棄済みだ。Authoring 側の参照とコンテンツのロード経路を確認する。
System が毎フレーム確保を行う	Entity ごとにアセットを解決またはロードしている。参照をベイクし、解決済みのマネージドオブジェクトをキャッシュする。
ゲームプレイの System がランタイムで ScriptableObject に依存している	必要な値を Component データか Blob アセットにベイクしておく。
ビルドでアセットがストリップされる	アセットを Baking やコンテンツワークフローから参照しているか確認する。ランタイムの文字列パスだけで参照してはいけない。

Chapter 35. foreach → IJobEntity Migration

1. Status — obsolete, not yet removed

旧来のコンポーネントイテレーション API は、Entities 1.4 以降（6.5 を含む）では **obsolete** 扱いだが、コンパイル自体はまだ通る。

レガシー	6.5 でのステータス	代替
Entities.ForEach	1.4 以降 廃止 。使うたびに廃止警告が出る。Unity の Changelog によれば、将来のメジャーリリースで削除される予定だ	SystemAPI.Query<...> (メインスレッド) または IJobEntity (Job)
Job.WithCode	ForEach と同じく 廃止	素の IJob 構造体、または OnUpdate へのインライン化
IJobForEach (1.0 以前)	1.4 のはるか以前に削除済み	IJobEntity

この章を開いた理由は、おそらく次のどちらかだろう。1.x のプロジェクトを 6000.5 以降で開いてコンパイラが警告を大量に吐いているか、あるいは将来のメジャーリリースで API が消える前に警告を片付けておきたいか。いずれにせよ、パターンさえ掴めばリファクタリングは機械的に進む。

2. The two replacement APIs

- **SystemAPI.Query<T, U, ...>** — OnUpdate 内でのメインスレッドイテレーション。シンプルで Job のオーバーヘッドもなく、処理量が少ない場合やマネージド API が必要な場合に向く。
- **IJobEntity** — ソースジェネレーターが生成する並列 Job。マルチスレッド化に踏み出す準備ができているなら、そのまま移行できる。DOTS の真価が出るのはここだ。

背景は ../DOTS Workflows/11_IJobEntity ・ SystemAPI.Query.md を参照。

3. Port — Entities.ForEach → SystemAPI.Query

3.1 Before

```

public partial class HealthRegenSystem : SystemBase
{
    protected override void OnUpdate()
    {
        float dt = Time.DeltaTime;

        Entities
            .WithAll<Alive>()
            .WithNone<Dead>()
            .ForEach((ref Health health, in Regen regen) =>
            {
                health.Value = math.min(health.Value + regen.Rate * dt, health.Max);
            })
            .ScheduleParallel();
    }
}

```

3.2 After — SystemAPI.Query (main thread)

```

[BurstCompile]
public partial struct HealthRegenSystem : ISystem
{
    [BurstCompile]
    public void OnUpdate(ref SystemState state)
    {
        float dt = SystemAPI.Time.DeltaTime;

        foreach (var (health, regen) in
            SystemAPI.Query<RefRW<Health>, RefRO<Regen>>()
                .WithAll<Alive>()
                .WithNone<Dead>())
        {
            health.ValueRW.Value = math.min(
                health.ValueRO.Value + regen.ValueRO.Rate * dt,
                health.ValueRO.Max);
        }
    }
}

```

書き換えで変わる点を整理しておく。

- **SystemBase** → **ISystem**（より軽量で Burst 化できる）。ただし旧コードがマネージドフィールドをキャプチャしているなら、**SystemBase** のまま残す必要がある。
- **ref Health** → タプル要素の **RefRW<Health>**。読み取りは **.ValueRO**、書き込みは **.ValueRW** を使う。
- **in Regen** → **RefRO<Regen>**。
- **Time.DeltaTime** → **SystemAPI.Time.DeltaTime**。
- フィルター（**WithAll** / **WithNone**）は **Query** ビルダー側へ移す。

4. Port — Entities.ForEach.ScheduleParallel → IJobEntity

ForEach を並列スケジュールしていた場合、Job 形式へは 1 対 1 でそのまま移行でき、並列性も保たれる。

4.1 After — IJobEntity

```
[BurstCompile]
public partial struct HealthRegenJob : IJobEntity
{
    public float DeltaTime;

    void Execute(ref Health health, in Regen regen)
    {
        health.Value = math.min(health.Value + regen.Rate * DeltaTime, health.Max);
    }
}

[BurstCompile]
public partial struct HealthRegenSystem : ISystem
{
    [BurstCompile]
    public void OnUpdate(ref SystemState state)
    {
        new HealthRegenJob { DeltaTime = SystemAPI.Time.DeltaTime }
            .ScheduleParallel();
    }
}
```

フィルターは Job 構造体にアトリビュートとして付ける。

```
[WithAll(typeof(Alive))]
[WithNone(typeof(Dead))]
public partial struct HealthRegenJob : IJobEntity { ... }
```

あるいはスケジュール時にクエリを渡してもよい。

```
var q = SystemAPI.QueryBuilder()
    .WithAll<Health, Regen, Alive>()
    .WithNone<Dead>()
    .Build();
new HealthRegenJob { DeltaTime = dt }.ScheduleParallel(q);
```

5. Captures and their replacements

旧来の Entities.ForEach では、.WithReadOnly や .WithDisposeOnCompletion などを使ってローカル変数をキャプチャできた。IJobEntity では、キャプチャは **Job 構造体のフィールド** として表す。何が渡されるかを Burst が正確に把握できる。


```
// BEFORE
var buffer = new NativeArray<int>(count, Allocator.TempJob);
Entities.ForEach((ref Foo f) => { /* use buffer */ })
    .WithDisposeOnCompletion(buffer)
    .ScheduleParallel();

// AFTER
[BurstCompile]
partial struct FooJob : IJobEntity
{
    [ReadOnly] public NativeArray<int> Buffer;

    void Execute(ref Foo f) { /* use Buffer */ }
}

var buffer = new NativeArray<int>(count, Allocator.TempJob);
state.Dependency = new FooJob { Buffer = buffer }.ScheduleParallel(state.Dependency);
buffer.Dispose(state.Dependency); // chained dispose replaces WithDisposeOnCompletion
```

アトリビュートへの置き換えは次のとおり。

- `.WithReadOnly(x)` → `[ReadOnly] public NativeArray<T> X;`
- `.WithDisposeOnCompletion(x)` → スケジュール後の `x.Dispose(handle)`。
- `.WithoutBurst()` → `Job` 構造体への `[WithoutBurst]`（ただし、そもそも `WithoutBurst` が必要になった原因を取り除けないか、まず検討してほしい）。
- `.WithEntityAccess()` → `Execute` に `Entity entity` パラメータを追加する。

6. When a SystemBase is still required

元の `System` が本当にマネージドフィールド（`Input System`・`UI Toolkit`・外部サービスなど）を必要としているなら、機械的に `ISystem` へ変換することはできない。選択肢は 2 つある。

1. **SystemBase のまま残す。** `OnUpdate` 内で `SystemAPI.Query` を使う。シングルスレッドになる。
2. **分割する。** マネージドワールドを橋渡しする `SystemBase` と、`Burst` 処理を担う `ISystem` に分ける。詳しくは `../DOTS Workflows/08_System - ISystem vs SystemBase.md` を参照。

7. Validation

System を移行したら、次を確認する。

1. **挙動をユニットテストする**。移行でロジックは変わらないはずだ。出力が変わったなら、リファクタリングでバグが入ったことになる。
2. **プロファイルする**。同等の処理なら、並列 `ForEach` と `IJobEntity.ScheduleParallel` の差は 5% 以内に収まるはずだ。大きく悪化したなら `Burst` が効いていない。Burst Inspector で確認する。
3. **古い System ファイルを削除する**。`#if LEGACY` のフォールバックを残してはいけない。残せば必ず腐っていく。

8. Troubleshooting

症状	原因 / 対処
warning: Entities.ForEach is obsolete	6.5 では想定内だ。コードはまだコンパイルできる。 将来のメジャーリリースで削除される前に、上記の手順で移行して警告を消しておく。
error: 'SystemAPI' does not contain 'Query'	System が <code>partial</code> になっていないか、構造体が <code>ISystem</code> を正しく実装していない。
IJobEntity はコンパイルできるが実行されない	構造体と <code>OnUpdate</code> に <code>[BurstCompile]</code> が付いていない。ソースジェネレーターは <code>partial</code> でない型をスケジュールしない。
Job が古い値を読む	<code>.Schedule*</code> を呼ぶ前にローカル変数をキャプチャしてしまっている。 フィールドはスケジュール前に Job 構造体へ設定しなければならない。
移行後にパフォーマンスが落ちる	Burst Inspector で Burst フォールバックを確認するか、 フィールドにマネージド型が紛れ込んでいないかを調べる。
移行後にコンパイル時だけ問題が出る	キャプチャしていたローカル変数をアトリビュートで置き換えられているか、 <code>ForEach</code> API でしか使えないラムダクロージャ構文が残っていないかを確認する。

Chapter 36. IAspect Deprecation — Migrating Off Aspects

1. Status — deprecated, not yet removed

IAspect は **Entities 1.4** で廃止としてマークされ、6.x 系でも廃止のまま。コンパイルも動作もまだ可能で、6.5 の API リファレンスでも有効な `IJobEntity.Execute` のパラメータ種別として載っている。とはいえコンパイラは廃止警告を出すし、Unity は**将来のメジャーリリースで削除する**と明言している。新規コードでは使わないこと。既存コードは折を見て移行すればよい。

IAspect は、複数の Component に対する「ビュー」を System が宣言し、それを `Entities.ForEach` や `IJobEntity` で単一の引数として渡せるようにするために導入された。だが実際には、コストに見合わないソース生成の層と、もう一つの型システムを増やしただけだった。

- Aspect を書くと、Component のパラメータリストを二重に書くことになる。
- デバッガが生成されたラッパーコードまでステップ実行してしまう。
- `IJobEntity` や `SystemAPI.Query` の `RefRW<T>` / `RefRO<T>` でも、余計な型を増やさずに「Component をひとまとめにして渡す」使い勝手が得られる。

API はまだ残っているので、Editor をアップグレードしてもコードが壊れることはない。この移行は、あくまで警告を解消し、2.0 に備えるための作業だ。

2. Mechanical refactor

2.1 Starting point

```
using Unity.Entities;
using Unity.Mathematics;
using Unity.Transforms;

public readonly partial struct MovementAspect : IAspect
{
    public readonly RefRW<LocalTransform> Transform;
    public readonly RefRO<Velocity> Velocity;
    public readonly RefRO<Speed> Speed;

    public void Step(float dt)
    {
        Transform.ValueRW.Position += Velocity.ValueRO.Direction * Speed.ValueRO.Value * dt;
    }
}
```

```

}

public partial struct MovementSystem : ISystem
{
    public void OnUpdate(ref SystemState state)
    {
        float dt = SystemAPI.Time.DeltaTime;
        foreach (var m in SystemAPI.Query<MovementAspect>())
            m.Step(dt);
    }
}

```

2.2 After — direct component parameters

```

using Unity.Entities;
using Unity.Mathematics;
using Unity.Transforms;

public partial struct MovementSystem : ISystem
{
    public void OnUpdate(ref SystemState state)
    {
        float dt = SystemAPI.Time.DeltaTime;
        foreach (var (transform, velocity, speed) in
            SystemAPI.Query<RefRW<LocalTransform>, RefRO<Velocity>, RefRO<Speed>>())
        {
            transform.ValueRW.Position += velocity.ValueRO.Direction * speed.ValueRO.Value * dt;
        }
    }
}

```

変更点は2つだ。

1. Aspect 型がなくなる。そのフィールドはそのまま `SystemAPI.Query` のタプル要素になる。
2. Aspect が持っていたメソッド（Step）はインライン化する。インラインにするには大きすぎるなら、同じパラメータを取る **静的ヘルパー**に切り出す。こうすれば Burst フレンドリーなまま保てる。

```

static void Step(ref LocalTransform t, in Velocity v, in Speed s, float dt)
{
    t.Position += v.Direction * s.Value * dt;
}

```

呼び出し側はこうなる。

```

Step(ref transform.ValueRW, velocity.ValueRO, speed.ValueRO, dt);

```

3. Aspects in IJobEntity

3.1 Before

```
[BurstCompile]
public partial struct MoveJob : IJobEntity
{
    public float DeltaTime;
    void Execute(MovementAspect aspect) => aspect.Step(DeltaTime);
}
```

3.2 After — components as Execute parameters

```
[BurstCompile]
public partial struct MoveJob : IJobEntity
{
    public float DeltaTime;

    void Execute(ref LocalTransform transform, in Velocity velocity, in Speed speed)
    {
        transform.Position += velocity.Direction * speed.Value * DeltaTime;
    }
}
```

`IJobEntity` では `RefRW<T>` / `RefRO<T>` ではなく、素の `ref` / `in` パラメータを使う点に注意する。
`Component` をチャック配列へのアクセスにラップする処理は、ジェネレーターが裏で行ってくれる。

4. Aspects that wrapped a DynamicBuffer

```
// BEFORE
public readonly partial struct PathAspect : IAspect
{
    public readonly DynamicBuffer<Waypoint> Path;
    public void Advance(int n) => Path.RemoveRange(0, n);
}

// AFTER – access buffer directly
foreach (var (path, e) in SystemAPI.Query<DynamicBuffer<Waypoint>>().WithEntityAccess())
{
    path.RemoveRange(0, 1);
}
```

`Job` でも `DynamicBuffer<T>` はそのまま `Execute` のパラメータとして使える。追加の手当ては要らない。

5. Cross-entity aspects

「別の Entity の Component を覗きたい」という目的で、Aspect が `ComponentLookup<T>` をラップしているケースもある。その置き換えは次のとおり。

```
// BEFORE
public readonly partial struct AITargetAspect : IAspect
{
    public readonly RefRO<Target> Target;
    [ReadOnly] public readonly ComponentLookup<Health> HealthLookup;
    public float TargetHealth =>
        HealthLookup.TryGetComponent(Target.ValueRO.Entity, out var h) ? h.Value : 0f;
}

// AFTER – declare the lookup on the system / job directly
public partial struct AISystem : ISystem
{
    private ComponentLookup<Health> _healthLookup;

    public void OnCreate(ref SystemState state)
    {
        _healthLookup = state.GetComponentLookup<Health>(true);
    }

    public void OnUpdate(ref SystemState state)
    {
        _healthLookup.Update(ref state);

        foreach (var (target, e) in SystemAPI
            .Query<RefRO<Target>>()
            .WithEntityAccess())
        {
            float hp = _healthLookup.TryGetComponent(target.ValueRO.Entity, out var h)
                ? h.Value : 0f;
            // ...
        }
    }
}
```

このパターンは Lookup を Entity 間で共有できるぶん、Aspect 版より軽量でもある。

6. When an aspect was complex

Aspect がフィールドを数多く抱え、状態をカプセル化し、込み入った継承に絡んでいるなら、それは単なる移行ではなく本格的なリファクタリングを考えるべきサインだ。巨大な Aspect を抱えた 1 つの System より、Component を介してデータを受け渡す 2 つの System に分けたほうが、コードが読みやすくなることが多い。

7. Troubleshooting

症状	原因 / 対処
コンパイル警告 IAspect is obsolete	6.5 では想定内だ。上記の手順で移行して警告を消す。 将来のメジャーバージョンで削除されるまでは、コードはそのまま動く。
type or namespace IAspect not found	IAspect がすでに削除されたバージョンを使っている。 移行を最後までやり切るしかない。フォールバックは残っていない。
Query タプルの型が多すぎてコンパイルできない	SystemAPI.Query は大きなタプルにも対応するが、幅の広すぎるクエリは 2 つに分けるか、 IJobChunk に移すほうがたいてい読みやすい。
Job が Burst コンパイルできなくなった	Aspect のメソッドが、 Burst の苦手な型をキャプチャしていた可能性がある。 Execute のパラメータを直接見直す。
そのまま移行したクロス Entity Aspect が遅い	ComponentLookup<T>.Update(ref state) は、使う前に OnUpdate で呼ばなければならない。呼び忘れると Lookup が無効になる。
DynamicBuffer の Aspect が移行できない	DynamicBuffer<T> は SystemAPI.Query の型としても IJobEntity.Execute のパラメータとしてもそのまま使える。ラッパーは要らない。
Aspect 型をアサートしていたテスト が失敗する	新しいシグネチャに合わせてテストを書き直すか削除する。 残しておく価値はない。

Appendix. Changelogs

Appendix A. Entities 1.4 → 6.5 Key Changes

`com.unity.entities` パッケージの公式 Changelog から、ECS プロジェクトを 1.4 系から 6.5 Core Package 系へ移行するときに押さえておきたい項目だけを絞り込んだまとめだ。本ページは Entities パッケージの変更だけをあえて対象にしている。

1. Version tracks

Entities 6.x は Unity Editor 6000.x に合わせてあり、Core Package として同梱される。古い Entities 1.x は Package Manager のパッケージだった。

トラック	配布	例
Entities 1.x	Package Manager (<code>com.unity.entities</code>)	公式の 6.5 Changelog 履歴に載る 1.4.2。
Entities 6.x	Editor に組み込まれる Core Package	6.4.0 / 6.5.0。

この区別はあくまでパッケージの配布とバージョン管理の話だ。無関係な UnityEngine や Editor 全体の移行を、DOTS プログラミングのドキュメントに持ち込んでよい理由にはならない。

2. Entities 6.5.0

Entities 6.5.0 の公式パッケージ Changelog のエントリはこうだ。

"Placeholder for 6.5.0"

6.5.0 は Unity 6000.5 に合わせたパッケージバージョンと考えればよい。このリリースで追加された Entities の新 API は、パッケージ Changelog には載っていない。

3. Entities 6.4.0

Entities 6.4.0 の公式パッケージ Changelog のエントリにはこうある。

"Package is now a core package embedded in Unity."

実務上の影響は次のとおり。

エリア	変更
インストール	Unity 6000.4 以降でも <code>com.unity.entities</code> は Package Manager から追加するが、Editor のバンドルからインストールされる。選ぶべきバージョンはない。
バージョン選択	Entities のバージョンは、インストール済みの Editor によって決まる。
ロックファイル	<code>packages-lock.json</code> 上で、パッケージがレジストリ依存ではなく <code>built-in / core</code> として記録される。
アップグレードワークフロー	<code>com.unity.entities</code> の固定された 1.x バージョンを置き換え（Package Manager から追加し直す）、Editor に Core バージョンへ解決させる。

詳しくは `../Migration/02_Package Manager → Core Package.md` を参照。

4. Entities 1.4.2

`com.unity.entities@6.5` に含まれる公式 Changelog 履歴には、Core Package 移行前の 1.4.2 が記載されている。

Changed

変更	重要な理由
Burst の依存関係を 1.8.24 に更新	まだ 1.4 系に留まるプロジェクト向けの依存バージョン更新。
USS クラスのサイズ指定をパーセントからピクセルへ変更	Editor UI のスタイリング上の細部。
フォントサイズのフィールドをパーセントからピクセルへ変更	Editor UI のスタイリング上の細部。

Fixed

修正	重要な理由
<code>public partial SystemBase</code> に対する誤ったソース生成	古い 1.4 のコードでソース生成エラーが出ていた場合に関係する。
<code>WithDeferredPlaybackSystem</code> 付きの <code>Entities.ForEach</code> 内で <code>SystemAPI</code> を使ったときの <code>SGICE002</code>	<code>Entities.ForEach</code> からの移行時に知っておくと役立つ。

修正	重要な理由
World シリアライズでのメモリーリーク	ランタイムと Editor の安定性に関わる。
Entity を選択したまま Play モードを抜けたあとの Inspector 初期化	Editor ツールの安定性に関わる。

5. 1.4.0 pre-release entries worth migrating for

重要な ECS API の方向性が見えてくるのは、1.4.0 のプレビュー / プレリリース履歴だ。

エリア	公式変更	移行ターゲット
イテレーション	<code>Entities.ForEach</code> を廃止としてマーク。 将来のメジャーで削除予定。	<code>IJobEntity</code> または <code>SystemAPI.Query</code> を使う。
Aspect	<code>IAAspect</code> を廃止としてマーク。 将来のメジャーで削除予定。	<code>Component</code> やクエリの API を直接使う。
オプションの Component 参照	<code>ComponentLookup.GetRefRWOptional()</code> / <code>GetRefROOptional()</code> を廃止。	<code>TryGetRefRW<T>()</code> / <code>TryGetRefRO<T>()</code> を使う。
チャンクバッファ	<code>ArchetypeChunk.GetBufferAccessorRO<T>()</code> / <code>GetBufferAccessorRW<T>()</code> を追加。	本当に必要なアクセスモードを指定する。
型なしバッファ	<code>GetUntypedBufferAccessorReinterpret<T>()</code> を追加。	変換元と変換先の要素サイズ、そしてバッファ容量が安全にエイリアスできる場合だけ使う。
System ハンドル	<code>SystemHandle</code> から <code>SystemTypeIndex</code> を取得できるようになった。	低レベルでの <code>System</code> アクセスがしやすくなる。
クエリツール	<code>Disabled</code> ・ <code>Present</code> ・ <code>Absent</code> ・ <code>None</code> の各カラム、 <code>Dependency</code> タブ、 <code>Query</code> ウィンドウでの <code>Prefab</code> の区別。	クエリの検査やデバッグがしやすくなる。
ECS 内のオブジェクト参照	<code>UnityObjectRef</code> のマニュアルドキュメントが追加された。	アンマネージドな ECS データからマネージドな <code>Unity</code> オブジェクトを参照するには <code>UnityObjectRef<T></code> を使う。

6. Migration checklist

- ☐ Unity 6000.4 以降では、DOTS の Core Package を Package Manager から追加し直し、`manifest.json` の固定された 1.x バージョンを置き換える。
 - ☐ 残っている `Entities.ForEach` を `IJobEntity` または `SystemAPI.Query` に置き換える。
 - ☐ `IAAspect` の利用を、直接の `Component` アクセスに置き換える。
 - ☐ `GetRefRWOptional` / `GetRefROOptional` を `TryGetRefRW<T>()` / `TryGetRefRO<T>()` に置き換える。
 - ☐ マネージドな Unity オブジェクト参照を見直し、適切なら `UnityObjectRef<T>` やベイク済みデータに切り替える。
 - ☐ 移行後に `Systems` ・ `Query` ・ `Entity Inspector` の各ウィンドウを再確認する。
-

7. References

- [Entities 6.5 Manual](#)
- [Entities 6.5 Changelog](#)

Appendix B. Netcode for Entities 1.4 → 6.5 Key Changes

Unity 6000.5 · Entities 6.5.0 · Netcode for Entities 6.5.0

Netcode for Entities が 1.4 系から 6.5 Core Package 系へ移る過程で積み重なった変更を、移行のために読み解く資料だ。Netcode 6.5.0 は組み込みパッケージとして Unity 6000.5 Editor に取り込まれたため、これ以降の項目はこのパッケージ Changelog ではなく Editor のリリースノート側に移っていく。

1. Version highlights

6.5.0 (2026-02-05) — built-in Editor package

エリア	変更
配布	Netcode は Unity 6000.5 から、組み込みの Editor パッケージになった。
Entities 依存関係	Changelog の行には <code>com.unity.entities 1.4.2</code> と書かれている。DOTS マニュアルが対象とする Unity 6000.5 / Entities Core Package とは混同しないこと。
Profiler	Ghost Snapshot View の TreeView ラベルに、Prefab や Component を素早く検査するためのコンテキストメニューが追加された。
Source Generator	Rider と Visual Studio で IDE 側の Source Generator 実行を無効化し、IDE のパフォーマンスが向上した。
Breaking	切断時の ECB が <code>BeginSimulationECBS</code> から <code>NetworkGroupCommandBufferSystem</code> に変わった。

1.12.0 (2026-02-01)

エリア	変更
Profiler	ティック単位のナビゲーション、検索フィールド、クライアント側の予測 / 補間ティック表示。
Entities 依存関係	1.4.2 から 1.4.4 へ更新。
廃止	ブラウザ版の Network Debugger を廃止し、Netcode Profiler に置き換え。
削除	<code>NETCODE_PROFILER_ENABLED</code> は不要になった。

エリア	変更
Breaking	GhostAuthoringComponentBaker の TransformUsageFlags の挙動が変わった。

1.11.0 (2025-12-12)

エリア	変更
Prefab List メニュー	Ghost Prefab を素早く検査できる。
GameObject Ghost	GameObject レイヤーの Ghost サポートに向けた作業が始まった。
Static Ghost 最適化	変化のないデータに対する Static Ghost Job の処理時間が大幅に短縮された。
Ghost Migration	IncludeInMigration が Ghost 以外のデータの移行にも対応した。
Bool シリアライズ	パック / アンパックいずれの bool シリアライズもコストが改善された。
API Breaking	PrefabDebugName.Name が完全に廃止された。

1.10.0 (2025-11-09)

エリア	変更
ゼロサイズコンポーネント	World 状態の保存がゼロサイズコンポーネントに対応した。
Source Generator	IDE 側の実行を無効化。6.5.0 の挙動に引き継がれている。
修正	PredictedGhostSpawnSystem のアサート、スリープモードでの警告の大量発生、AlwaysRun での予測ループの例外などを修正。

1.9.x (2025-09 to 2025-10)

エリア	変更
Ghost optimization docs	Ghost 最適化の専用ドキュメントが追加された。
Stabilization	1.8.0 の Profiler Modules と Importance Visualizer が安定版になった。

1.8.0 (2025-08-17)

エリア	変更
Importance Visualizer	PlayMode Tool に追加された。
Forced Input Latency	ClientTickRate.ForcedInputLatencyTicks が追加された。
Profiler Modules	Unity 6 以降向けの、実験的なサーバー / クライアント Profiler モジュール。
Breaking	GhostDistanceImportance のスケール関数が、baseImportance に 1000 を掛けなくなった。

エリア	変更
Transport	<code>com.unity.transport</code> を 2.5.3 に更新。

1.7.0 (2025-07-29)

エリア	変更
Host Migration	<code>ENABLE_HOST_MIGRATION</code> の <code>define</code> が削除され、Host Migration がデフォルトで有効になった。
Host Migration API	クラス名とメソッド名がリファクタリングされた。 正確な型名は最新のパッケージドキュメントやサンプルを参照する。
Serialization	Host Migration 向けに Ghost Component のシリアライズが改善された。
Transport	<code>com.unity.transport</code> を 2.1.0 に更新。

1.6.1 to 1.6.2 (2025-05 to 2025-07)

エリア	変更
Predicted ECB	<code>PredictedSimulationSystemGroup</code> 向けの <code>begin / end ECB</code> システムが追加された。
Predicted spawning	<code>EndPredictedSimulationCommandBufferSystem</code> の後ろに <code>PredictedSpawningSystemGroup</code> が追加された。
Host Migration	<code>ENABLE_HOST_MIGRATION</code> の裏に隠された実験的機能。
Reconnect tracking	<code>NetworkStreamIsReconnected</code> コンポーネントが追加された。
Snapshot history	<code>GhostSystemConstants.SnapshotHistorySize</code> をコンパイラ定義で設定できるようになった。
Relevancy + Importance	Ghost Relevancy と Importance Scaling を併用するためのファストパスに対応。

1.5.0 (2025-04-22)

エリア	変更
Thin Clients	ランタイムで Thin Client を作成するための <code>AutomaticThinClientWorldsUtility</code> が追加された。
Serialization	<code>FixedList</code> と <code>unsafe</code> な固定バッファのレプリケーションに対応。
Baselines	CPU コスト削減のための <code>GhostAuthoringComponent.UseSingleBaseline</code> が追加された。
Integer serialization	<code>byte / short</code> 専用のテンプレートを追加。
Lag Compensation	Physics History Buffer が 16 ティックから 32 ティックに拡張された。
Relay	Relay パッケージのパスが <code>com.unity.services.multiplayer v1.1.0</code> 側へ移った。

1.4.0 (2024-11-14)

エリア	変更
Send rate	GhostAuthoringComponent.MaxSendRate で Ghost の送信頻度に上限を設けられるようになった。
Chunk iteration	GhostSendSystemData.MaxIterateChunks で、ティックごとに処理するチャンク数に上限を設けられるようになった。
NetCodeConfig	Unity Transport の NetworkConfigParameters がいくつか追加された。
Snapshot ACK	ACK ヒストリの容量を設定できるようになった。
Fixes	256 ティックを超えた Snapshot ACK の不具合、物理補間のジッター、および関連する修正。

2. Cumulative breaking changes

バージョン	破壊的変更
6.5.0	切断時の ECB が BeginSimulationECBS から NetworkGroupCommandBufferSystem に変わった。
1.12.0	GhostAuthoringComponentBaker の TransformUsageFlags の挙動が変わった。
1.11.0	PrefabDebugName.Name が完全に廃止された。
1.8.0	GhostDistanceImportance の乗数の挙動が変わった。
1.7.0	Host Migration API のクラス名とメソッド名がリファクタリングされた。
1.5.0	コマンドの検証が厳格になり、ハンドシェイクのタイムアウト挙動が変わった。
1.4.0	MaxSendChunks のスコープが変わった。

3. Trends to remember

1. **Profiler 中心のデバッグへ** — 古いブラウザ版 Network Debugger は廃止された。Netcode Profiler のモジュールを使う。
 2. **Host Migration がデフォルトに** — 1.6.1 では実験的扱いだったが、1.7.0 以降はデフォルトで有効になった。
 3. **IDE のパフォーマンス向上** — Netcode の Source Generator が IDE の解析内で実行されなくなった。
 4. **シリアライズの効率化** — bool のパッキング、byte / short テンプレート、シングルベースラインのオプションにより、CPU と帯域幅の負荷が下がった。
 5. **Core Package への移行** — Netcode 6.5.0 は Unity 6000.5 に同梱される。今後の変更は Editor のリリースノート側へ移っていく。
-

4. Related docs

- `../DOTS Workflows/16_Netcode Client-Server World & Bootstrap.md`
 - `../DOTS Workflows/18_Netcode Ghost Snapshot & Synchronization.md`
 - `../DOTS Workflows/22_Netcode Ghost Optimization · Importance · Relevancy.md`
 - `../DOTS Workflows/24_Netcode Profiler & Debugging.md`
-

5. References

- [Netcode for Entities 6.5 Manual](#)
- [Netcode for Entities 6.5 Changelog](#)
- [Netcode for Entities 1.12 Changelog](#)

Reference. Glossary

Glossary

Unity 6000.5 · Entities 6.5.0 · Netcode for Entities 6.5.0

本マニュアルに登場する、説明なしには分かりにくい用語をすべて一行で定義する。クロスリンクは、各用語を詳しく扱ったドキュメントを指している。

1. Core nouns — the five things ECS is built from

用語	定義	参照
Entity	不透明な 64 ビット ID (<code>Unity.Entities</code> 名前空間の <code>struct Entity</code>)。データも挙動も持たず、 <code>Component</code> を引くためのキーにすぎない。	DOTS Workflows/03_ECS Core Concepts.md
Component	<code>Entity</code> に付与するデータ。種類は何通りかある。 §2 を参照。	DOTS Workflows/05_Component Types.md
Archetype	<code>Entity</code> が持つ <code>Component</code> 型の組み合わせ。同じ <code>Archetype</code> の <code>Entity</code> はストレージを共有する。	DOTS Workflows/03_ECS Core Concepts.md
Chunk	同じ <code>Archetype</code> を持つ <code>Entity</code> をまとめた 16 KB のブロック。 <code>Component</code> はチャンク内にカラムメジャー (SoA) で格納される。	Optimizations and Debugging/01_Chunk Layout & TypeManager.md
World	<code>Entity</code> と <code>System</code> を収めるコンテナ。ランタイムは「Default World」から始まる。	DOTS Workflows/03_ECS Core Concepts.md

2. Component kinds

種類	内容
<code>IComponentData</code>	アンマネージドな構造体コンポーネント。デフォルトの種類。
<code>IBufferElementData</code>	<code>Entity</code> に付与する <code>DynamicBuffer<T></code> の要素型。
<code>ISharedComponentData</code>	値をキーにするコンポーネント。同じ値を持つ <code>Entity</code> はチャンクのパケットを共有する。
<code>ICleanupComponentData</code>	<code>DestroyEntity</code> の後も残るコンポーネント。 <code>Entity</code> が完全に消える前に <code>System</code> がクリーンアップを行える。
Tag component	空の <code>IComponentData</code> 構造体。ゼロバイトで、クエリフィルターとしてだけ使う。

種類	内容
Chunk component	チャンク単位のリデータ (Entity ごとではなく、チャンクごとに 1 インスタンス)。
IEnableableComponent	チャンク単位のリビットマスクで有効・無効を切り替えられる Component。 構造変更を伴わない。

詳しくは DOTS Workflows/05_Component Types.md・DOTS Workflows/06_Enableable Component.md を参照。

3. Systems and jobs

用語	内容
ISystem	構造体ベースの新しい System インターフェース。Burst に対応する。 新規の System にはこちらを推奨。
SystemBase	クラスベースの旧来の System インターフェース。System がメインスレッドでのマネージドアクセスを必要とする場合だけ使う。
SystemGroup	System を順序付きで束ねるコンテナ。主なグループは InitializationSystemGroup・SimulationSystemGroup・PresentationSystemGroup。
IJobEntity	クエリに一致する Entity をイテレートする Job。ソース生成され、Burst コンパイルされる。
IJobChunk	個々の Entity ではなくチャンクをイテレートする Job。チャンク単位の集計に使う。
SystemAPI	System 内で使う静的ヘルパー群。Query<>()・GetComponentLookup<T>()・Time・GetSingleton<T>() などを提供する。

詳しくは DOTS Workflows/08_System – ISystem vs SystemBase.md・DOTS Workflows/11_IJobEntity・SystemAPI.Query.md・DOTS Workflows/12_IJobEntity vs IJobChunk.md を参照。

4. Authoring → runtime

用語	内容
Baker	SubScene の保存時やビルド時に、Authoring の GameObject と MonoBehaviour を ECS Component へ変換する。
SubScene	Authoring 用のコンテナ。中の GameObject はベイクされて Entity になり、EntityScene アセットにシリアルライズされる。
EntityScene	SubScene から生成されるベイク済みアセット。ランタイムに World へロードされる。

詳しくは DOTS Workflows/01_Baker Pattern & SubScene.md を参照。

5. Structural change safety

用語	内容
Structural change	Component の追加・削除、または Entity の生成・破棄。Entity をチャンク間で動かすためコストが高く、イテレーション中に行うのは安全でない。
EntityCommandBuffer (ECB)	イテレーション中に構造変更を記録しておき、安全な境界でまとめてプレイバックする。
EntityCommandBuffer.ParallelWriter	並列 Job 向けのスレッドセーフな ECB のバリエーション。各呼び出しは、決定論的なプレイバックのために sortKey を受け取る。
Deferred entity	ecb.CreateEntity() や Instantiate() で作成した、まだ実体のない Entity。プレースホルダーのインデックスは ECB のプレイバック時に解決される。
Sort key	並列 ECB の各コマンドに渡す整数。デフォルトは [ChunkIndexInQuery]。

詳しくは DOTS Workflows/13_Structural Change & Safety.md・DOTS Workflows/14_EntityCommandBuffer・Deferred Entity.md・DOTS Workflows/15_ParallelWriter・Deterministic Playback.md を参照。

6. Entity reference types

ECS データや Baking ワークフローで使う DOTS の参照型をまとめる。

型	内容
Entity	ECS のハンドル (Unity.Entities)。ECS World 内ではただの ID にすぎない。
EntityPrefabReference	コンテンツ / ストリーミングワークフロー向けの、ベイク済み Entity Prefab アセットへの参照 (Unity.Entities.Serialization)。
UnityObjectRef<T>	ECS 側から UnityEngine.Object を指す参照。Entity からマテリアルやメッシュなどのマネージドアセットを、ボクシングなしで参照できる。

詳しくは DOTS Workflows/04_Entity References – Entity・EntityPrefabReference・UnityObjectRef.md を参照。

7. Netcode for Entities

用語	内容	参照
Netcode for Entities	クライアント予測を備えたサーバー権威型ゲーム向けの、Unity の DOTS マルチプレイヤーネットコードレイヤー。	DOTS Workflows/16_Netcode Client-Server World & Bootstrap.md
Server World	サーバーのシミュレーションを実行し、Ghost Snapshot を送信する権威側の ECS World。	DOTS Workflows/16_Netcode Client-Server World & Bootstrap.md
Client World	Snapshot を受信し、コマンドを送信し、ローカルのゲームプレイを予測し、補間した状態を表示する ECS World。	DOTS Workflows/16_Netcode Client-Server World & Bootstrap.md
Thin Client World	負荷や接続の挙動をテストするための、軽量なダミークライアント World。	DOTS Workflows/16_Netcode Client-Server World & Bootstrap.md
NetworkStreamConnection	トランスポートの接続ハンドルを保持する、接続 Entity 上のコンポーネント。	DOTS Workflows/17_Netcode Network Connection & Approval.md
NetworkId	承認後にサーバーが割り当てる接続 ID。	DOTS Workflows/17_Netcode Network Connection & Approval.md
NetworkStreamInGame	ゲームプレイデータの送受信を有効にするコンポーネント。これがないと Snapshot もコマンドも流れない。	DOTS Workflows/17_Netcode Network Connection & Approval.md
Ghost	サーバーが権威を持ち、Snapshot を通じてクライアントにレプリケートされるネットワーク Entity。	DOTS Workflows/18_Netcode Ghost Snapshot & Synchronization.md
Snapshot	サーバーからクライアントへ送られる、シリアライズされた Ghost の状態。通常は非信頼トランスポートで送られる。	DOTS Workflows/18_Netcode Ghost Snapshot & Synchronization.md
GhostField	Component のフィールドを Ghost シリアライズの対象として印付けるアトリビュート。	DOTS Workflows/18_Netcode Ghost Snapshot & Synchronization.md
PredictedGhost	クライアント予測の対象となる Ghost を識別するコンポーネント。	DOTS Workflows/19_Netcode Prediction & Rollback.md
Simulate	現在の予測ティックで実行すべき予測 Entity を示すために、Netcode が使う Enableable なタグ。	DOTS Workflows/19_Netcode Prediction & Rollback.md
IInputComponentData	コマンドバッファの管理が自動生成される、高レベルの入力コンポーネント型。	DOTS Workflows/20_Netcode Command Stream & Input.md
ICommandData	ティックの対応付けやバッファアクセスを自分で管理する、低レベルのコマンドストリーム型。	DOTS Workflows/20_Netcode Command Stream & Input.md

用語	内容	参照
InputEvent	ジャンプや射撃のように、対象ティックでちょうど 1 回だけ登録されるべきイベント向けの、ワンショットな入力マーカー。	DOTS Workflows/20_Netcode Command Stream & Input.md
IRpcCommand	信頼性のあるワンショットな RPC メッセージ型。	DOTS Workflows/21_Netcode RPC.md
IApprovalRpcCommand	接続承認の途中でも送れる、特殊な RPC 型。	DOTS Workflows/21_Netcode RPC.md
Ghost Importance	どの Ghost を優先して Snapshot のバジェットに収めるかを決める、サーバー側の優先度スコア。	DOTS Workflows/22_Netcode Ghost Optimization ・ Importance ・ Relevancy.md
Ghost Relevancy	特定のクライアントへ Ghost をレプリケートするかどうかを決める、接続単位のフィルタリング。	DOTS Workflows/22_Netcode Ghost Optimization ・ Importance ・ Relevancy.md
Lag Compensation	クライアントの操作を、そのクライアントが知覚した時刻で検証するために、サーバーが過去のコリジョンワールドを参照する仕組み。	DOTS Workflows/23_Netcode Physics Integration & Lag Compensation.md

8. Legacy / obsolete (do NOT appear in new code)

用語	6.5 でのステータス	移行ターゲット
Entities.ForEach	1.4 以降は廃止。 警告付きでコンパイルは通る。 将来のメジャーリリースで削除予定。	SystemAPI.Query<>() または IJobEntity。 Migration/04_foreach → IJobEntity.md を参照
IAspect	1.4 以降は廃止。 警告付きでコンパイルは通る。 将来のメジャーリリースで削除予定。	Component パラメータの直接指定 (RefRW<T> / RefRO<T>)。Migration/05_IAspect Removal.md を参照
ComponentLookup.GetRefRWOptional	廃止。	TryGetRefRW。Migration/01_Entities 1.x → 6.5 Overview.md を参照

9. Supporting concepts

用語	内容
Burst	Job や [BurstCompile] を付けた構造体を対象にする、Unity のネイティブコンパイラ。ネイティブな SIMD コードを生成する。
Change version	System がチャンク内の Component カラムへ書き込むたびに進む 32 ビットのカウンタ。これにより SetChangedVersionFilter で変化のないチャンクをスキップできる。
SoA (Structure of Arrays)	フィールドごとに別々の配列にするストレージレイアウト。スカラーなイテレーションでキャッシュ効率がよい。チャンクは SoA を採用している。
RefRW<T> / RefRO<T>	SystemAPI.Query で、読み書き / 読み取り専用の意図を明示して Component にアクセスするためのラッパー型。
EnabledRefRW<T> / EnabledRefRO<T>	構造変更を起こさずに、Enableable Component の有効・無効ビットを読み書きするためのラッパー。
TypeManager	各 Component に安定した型インデックスを割り当てるランタイムのレジストリ。起動時に再構築されるため、大規模プロジェクトではその時間が体感できる。
BlobAssetReference<T>	チャンクの外に置かれた、大きく読み取り専用のデータ Blob へのポインタ。チャンク容量が膨らむのを防ぐ。

© Silly Tool Valley. 公開された範囲での閲覧と引用は許可されている。自動スクレイピング、データセットへの収録、AI 学習、AI 回答への取り込みは、書面による許可なく行うことを許可しない。